Android Developer Interview Handbook

Your Ultimate Guide to Crack Android Interviews with Confidence

One Time Purchase Lifetime free updates

Written By
Anand Gaur

MobileTech Lead@ TCS
Ex-Samsung & Tech Mahindra

500+ Real Interview Questions with Clear Explanations

Android Developer Interview Handbook
© 2025 Anand Gaur
All rights reserved.

No part of this eBook may be reproduced, stored in a retrieval system, or transmitted in any form or by any means electronic, mechanical, photocopying, recording, or otherwise without prior written permission of the author.

Copyright Notice: Unauthorized sharing or distribution of this eBook is illegal. Any digital forwarding, uploading, or reproduction without permission is a violation of the **Copyright Act, 1957** and may lead to legal consequences.

Disclaimer: The information in this eBook is provided for educational and interview preparation purposes. While every effort has been made to ensure accuracy, the author is not responsible for any errors or consequences arising from the use of this eBook.

ISBN: 978-93-343-7047-8

First Edition: July 2025
Published by: Anand Gaur
Country of Publication: India

📖 About This Handbook

Thank you for purchasing this Android Developer Interview Handbook.

My name is **Anand Gaur**, and I'm currently working as a **Mobile Tech Lead at TCS** with over **8+ years of experience** in Android development. Over the years, I've worked with renowned companies like **Samsung, Tech Mahindra**, and various high-growth startups. Throughout my journey, I've contributed to building **25+ Android apps**, solving real-world challenges, scaling performance, and mentoring teams.

I'm also actively involved in **interview panels at TCS**, where I've taken countless technical interviews. This experience gave me deep insights into what companies really expect — not just textbook answers, but practical thinking, scenario handling, and architectural understanding.

Why I Created This Handbook?

As a developer, I always found it frustrating to prepare for interviews by juggling multiple sources for questions and answers. There was no one place that covered real, relevant, and repeatedly asked questions — especially with clear explanations. That's when I decided to create **this handbook** — a **single, structured, and reliable guide** that helps every Android developer prepare smarter and faster.

Most of the questions included here are not just from theory — they are from **my personal experience**, real interviews I've appeared in, or interviews I've conducted. I've consistently documented every question I faced — especially those that challenged me — so that I could improve and help others grow too.

My Philosophy on Interview & Growth

Even when I'm not actively looking for a job, I continue giving interviews. Why? Because interviews are **not just about switching companies**, they're about understanding your **strengths, weaknesses**, and **where the industry is heading**. They help me assess how much I've grown and where I need to sharpen my skills.

In the last 8+ years, I've appeared in **1000+ interviews**. I've faced rejections too — but every rejection was fuel for my next improvement. That's the mindset I live by:

> "**Never fear rejection. Use it as feedback.**"

Created By Anand Gaur

A Note on AI & The Future

Today, many developers are worried that **AI will replace their jobs**. But let me tell you this — **AI won't replace skilled developers, it will amplify them.** If you focus on improving your logic, your fundamentals, and your ability to build real solutions — AI will become your assistant, not your competition.

Remember, critical systems (like fintech, healthcare) will **always need human oversight**, deep logic, and careful code reviews. Your ability to **think deeply, write clean architecture, and stay updated** will set you apart.

Final Words

This handbook isn't just a collection of questions — it's a **reflection of years of real experience**, effort, failures, learnings, and growth.

If you read it with the **right mindset**, and **keep improving daily**, you'll become part of the top **5% Android developers** — those who don't just write code, but **write smart, scalable, and secure solutions.**

Stay consistent. Stay curious. Keep preparing — **even if you're not switching.**
Because in the ever-changing tech world, **your only true job security is your skills.**

– Anand Gaur
Mobile Tech Lead | Mentor
🌐 LinkedIn: @anandgaur22
💻 GitHub: @anandgaur22
📖 Medium: @anandgaur2207
📸 Instagram: @tech.anandgaur
▶️ YouTube: @technicalanandgaur

Created by Anand Gaur

Contents

1. Android Core Concepts (Page 10 - 47)

Covers fundamental building blocks of Android and essential interview topics:

- Android Application Lifecycle
- Activity vs Fragment vs View
- Context: Application vs Activity
- Intents: Explicit vs Implicit
- Services, Broadcast Receivers, Content Providers
- ViewModel, LiveData, Lifecycle Aware Components
- RecyclerView, Adapters & ViewHolder Pattern
- Difference between UI Thread & Background Thread
- Handler, Looper, and MessageQueue
- SharedPreferences, SQLite, Room DB
- Data Binding vs View Binding
- Manifest Permissions, App Components
- Parcelable vs Serializable
- Process Death & State Restoration
- ANR & StrictMode
- Debugging with Logcat & Profiler

Created by Anand Gaur

- WorkManager, JobScheduler, AlarmManager
- Multi-Window Support, Doze Mode, App Standby

2. **Kotlin Concepts** (Page 48 - 106)

Focused on Kotlin language essentials, advanced concepts and best practices:

- Null Safety, Elvis Operator, Safe Calls
- Extension Functions, Lambda Expressions
- Data Classes vs Regular Classes
- Sealed Classes vs Enum
- Object vs Companion Object
- High-Order Functions, Inline Functions
- Coroutines: launch, async, suspend, delay
- Coroutine Scopes: GlobalScope, viewModelScope, lifecycleScope
- Flows vs Channels
- Exception Handling in Coroutines
- Kotlin Collections (map, filter, flatMap)
- Typealias, Destructuring Declarations
- Kotlin DSL, Inline Classes
- Kotlin Multiplatform Basics

Created by Anand Gaur

- Kotlin with Retrofit & Room
- Best Kotlin Interview Patterns

3. **Android Architecture** (Page 107 - 133)

Explores clean code practices and architecture models used in real-world projects:

- MVVM vs MVP vs MVC
- Repository Pattern
- Use Case / Interactors
- ViewModel Layer
- UI State Management
- Dependency Injection: Dagger, Hilt
- SOLID Principles in Android
- Single Source of Truth
- Clean Architecture Layers
- Jetpack Navigation Component
- Unidirectional Data Flow (UDF)
- Common Mistakes in Architecture Design

Created by Anand Gaur

4. **Jetpack Compose** (Page 134 - 157)

Modern UI Toolkit concepts covered for declarative UI in Android:

- What is Jetpack Compose
- Composable Functions, Preview
- State & State Hoisting
- Remember & MutableState
- Scaffold, Column, Row, LazyColumn
- Navigation in Compose
- Form Validation with State
- Side Effects: LaunchedEffect, rememberCoroutineScope
- Theming & Custom Composables
- Compose with ViewModel
- Lists, Lazy Grids, Pagination
- Animations & Transitions
- Modifiers & Layouts
- Compose vs XML based UI

Created by Anand Gaur

5. **Unit Testing Concepts** (Page 158 - 163)

Essential testing concepts for robust app development:

- What is Unit Testing?
- Writing JUnit Test Cases
- Testing ViewModel Logic
- Mockito, Mockk Basics
- Test Rules and Coroutines Testing
- Best Practices for Testable Architecture
- Dependency Injection for Testing
- UI Testing with Espresso (Overview)

6. **Android Security Concepts** (Page 164 - 174)

Critical concepts to secure Android apps and data:

- ProGuard, R8 & Code Obfuscation
- Secure SharedPreferences
- Encrypted Room Database
- Android Keystore System
- Network Security Configuration
- SSL Pinning
- Authentication with Biometric & Fingerprint
- App Integrity & Tamper Detection

Created by Anand Gaur

- OWASP Mobile Top 10
- Preventing Reverse Engineering
- Runtime Permission Best Practices

7. **Scenario-Based Interview Questions** (Page 175 - 208)

Advanced real-world scenario discussions & problem-solving Q&A:

- App crashes after process death – how do you restore state?
- RecyclerView lag – how would you optimize?
- How to handle API call failure in ViewModel?
- Implementing offline-first architecture
- Optimizing performance for large lists
- Security breach reported – how to identify and fix?
- Jetpack Compose form with complex validation
- Dealing with slow API responses in Coroutines
- Architecting large-scale feature module
- Handling memory leaks in legacy code
- How would you migrate XML to Compose?
- Debugging ANR issues in production
- Designing clean architecture for multi-module app

Created by Anand Gaur

8. **CI/CD, Git, and Detekt in Android** (Page 209 - 237)

Essential DevOps & Code Quality Concepts for Scalable Android Development:

- CI/CD Concepts in Android
- CI/CD Workflow Stages
- Tools Used in Android CI/CD
- Release Automation Techniques
- Multi-module Build Optimization
- Secrets and Signing Management
- CI/CD Best Practices
- Git Essentials
- Pull Request (PR) Strategies
- Git for Release Management
- Avoiding Git Mistakes
- Pre-commit Hooks for Android
- What is Detekt?
- Detekt Rule Sets
- Running Detekt
- Detekt Baseline Setup
- Custom Rules in Detekt
- Integrating Detekt in CI/CD

Created by Anand Gaur

Android Core Concepts

1) What are the core building blocks of an Android application?

Android apps are built using several essential components provided by the Android framework. These components work together to handle **UI**, **background tasks**, **user interactions**, and **data sharing**.

Core Building Blocks of an Android Application:

1. Activities

- Represents a single screen with a user interface.
- Acts as the entry point for user interaction.
- Every screen in an app is usually an Activity.

2. Fragments

- A modular piece of UI that lives inside an Activity.
- Can be reused in multiple activities.
- Useful for responsive layouts (e.g., tablets vs phones).

3. Services

- Runs background operations without user interaction.
- Useful for long-running tasks like downloading files, playing music, etc.

4. Broadcast Receivers

- Listens to system-wide or app-wide broadcast messages.
- Useful for responding to events like battery low, Wi-Fi connected, etc.

5. Content Providers

- Used to share data between apps.
- Acts as a data access interface, like a database that can be queried using URI.

Created by Anand Gaur

6. View:

- Every UI component like Button, TextView, etc.
- Created in XML or code using Jetpack Compose.

7. Layouts:

- Organize UI views. Examples: LinearLayout, ConstraintLayout, Compose Column/Row.

8. Manifest File:

- Declares all components, permissions, and metadata.
- It's like the blueprint of your app.

2) What is the intent?

Intent is a messaging object used to request an action from another app component (activity, service, or broadcast receiver). It allows communication between components and even across different applications.

There are two types of intents in Android:

1) Explicit Intent

- Used to launch a **specific component** (e.g., another activity **within the same app or in another app**, if the component name is known).

- You define the **class name** or **component name** directly.

Example:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

2) Implicit Intent

- Used to perform an action **without specifying the exact component**.

- The Android system will match the intent with the appropriate component using **intent filters**.

```
val intent = Intent(Intent.ACTION_VIEW)
intent.data = Uri.parse("https://www.google.com")
startActivity(intent)
```

Created by Anand Gaur

3) What is the Android Application Lifecycle?

The **Android Application Lifecycle** is the process your **entire app** goes through — from the moment it starts running to the moment it's closed or killed by the system.

This lifecycle is managed by the `Application` **class**.

Key Steps in the Application Lifecycle:

`onCreate()`

- Called **once** when the app is first launched
- Best place to **initialize global things** like Firebase, logging, or any SDKs

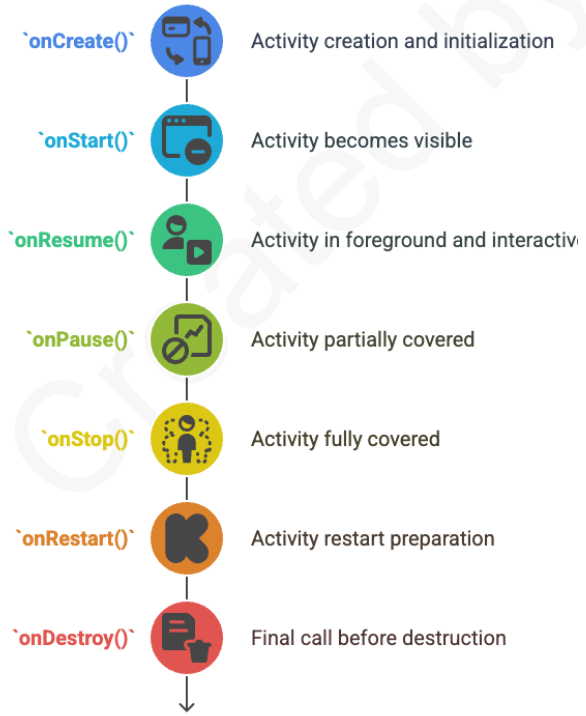
`onTerminate()`

- Called when the app is about to close (only in emulators or rare cases)
- Not reliable on real devices

`onLowMemory()` / `onTrimMemory()`

- Called when the system is running low on memory
- Use this to **free up memory** (like clearing image cache)

Activity Lifecycle Methods:



Created by Anand Gaur

4) What is the Scenario in which only onDestroy is called for an activity without onPause() and onStop()?

If **finish()** is called in the OnCreate method of an activity, the system will invoke onDestroy() method directly.

5) What is the activity A to activity B lifecycle in Android?

Let's Say Activity **A is starting Activity B by Calling StartActivity(Intent)** method then the lifecycle call be like this:-

- A onCreate()
- A onStart()
- A onResume()

Now Button click for startActivity(intent)

- A onPause()
- B onCreate()
- B onStart()
- B onResume()
- A onStop()

….. If you press the back button from Activity B then the lifeCycle call will be …..

- B onPause()
- A onRestart()
- A onStart()
- A onResume()
- B onStop()
- B onDestory()

Now one more scenario "From Activity A start Activity B by calling StartActivity(Intent) on button click and use finish() method inside onstart() method on Activity B"

- A onPause()
- B onCreate()
- B onStart()
- A onResume()
- B onStop()
- B onDestory()

Created by Anand Gaur

6) What is Context in Android?

Context gives access to system services, app resources, file system, databases, etc. It represents the current state of the application or component.

Types of Context:

1. **Application Context**

- Lifecycle is tied to the whole application.
- Use when context needs to live long (e.g., database, shared preferences).

2. **Activity Context**

- Tied to UI lifecycle.
- Use when you're dealing with UI, layouts, etc.

Example:

```
//Application context
val prefs = context.getSharedPreferences("prefs",
Context.MODE_PRIVATE)

//Activity context for Toast
Toast.makeText(this, "Hello", Toast.LENGTH_SHORT).show()
```

7) What is a Content Provider and When to Use It?

A Content Provider is used to share data between apps (like contacts, media files) in a secure and structured way.

Key Functions:

- Acts like a **database interface**.
- Uses **URI (Uniform Resource Identifier)** to access data.
- Helps apps **read/write data** in another app with permission.

Created By Anand Gaur

Use-Cases:

- Accessing phone contacts, media, SMS.
- Custom Content Providers to share app-specific data.

Example (Query Contacts):

```
val cursor = contentResolver.query(
    ContactsContract.Contacts.CONTENT_URI,
    null, null, null, null
)
```

8) How do access data using a Content Provider?

First, make sure your app has the required **read permission**.
Then, get the ContentResolver by **calling getContentResolver()** from the Context.
Use ContentResolver.query() to request the data.

This method returns a Cursor, which you can use to read data from each column using cursor methods like **getString()** or **getInt().**

9) What are the fragments?

A fragment is a UI entity attached to an Activity. Fragments can be reused by attaching to different activities. Activity can have multiple fragments attached to it. The fragment must be attached to an activity and its lifecycle will depend on its host activity.

10) What is the lifecycle of the fragment?

1. onAttach()

- Called when the fragment is first attached to the Activity.
- Good place to get a reference to the activity.

2. onCreate()

- Called when the fragment is being created.
- Initialize essential components (like variables or data you want to keep).

3. onCreateView()

- Called to create and return the UI for the fragment.
- Return a layout (View) or null if there's no UI.

4. onActivityCreated()

- This method was used to access activity-related stuff, but it's now **deprecated**.Use `onViewCreated()` instead.
- Useful for initializing components that need the activity's context (e.g., adapters).

5. onStart()

- Called when the fragment becomes visible to the user.

6. onResume()

- Called when the fragment becomes active (ready to interact).
- Ideal for starting animations or listening to events.

7. onPause()

- Called when the fragment is partially hidden (another fragment/activity comes on top).
- Good time to save temporary data or pause tasks.

8. onStop()

- Called when the fragment is no longer visible.

9. onDestroyView()

- Called to remove and clean up the UI of the fragment.

10. onDestroy()

- Final cleanup of the fragment's non-UI components.
- Might not always be called.

11. onDetach()

- Called when the fragment is detached from the activity.
- The fragment is no longer connected to the host activity.

Created by Anand Gaur

11) When should you use a fragment rather than an activity?

Use a Fragment instead of an Activity in the following situations:

1. When UI components need to be reused in multiple screens

- If a part of your layout or logic is required in different Activities, convert it into a Fragment for better reusability.

2. When creating a multi-pane UI (side-by-side views)

- Ideal for tablets or large screens where you want to show multiple sections together.
- Common in ViewPager, TabLayout, and split-screen designs.

3. When data should be retained across configuration changes

- Fragments can be set to retain instance, so they help save and restore data during screen rotations.

4. When using Navigation Component (Jetpack)

- Modern Android navigation is based on Fragments.
- Using Fragments simplifies back stack management and screen transitions.

5. When building a dynamic UI

- Fragments can be added, replaced, or removed at runtime using FragmentTransactions.
- Helps in creating flexible and adaptive UIs.

6. When lifecycle awareness is needed within a part of UI

- Each Fragment has its own lifecycle, which is useful when dealing with scoped ViewModels or lifecycle-aware components like LiveData.

12) Difference between adding/replacing fragments in the backstack?

- ***replace*** removes the existing fragment and adds a new fragment. This means when you press the back button the fragment that got replaced will be created with its onCreateView being invoked.

Created by Anand Gaur

- ***add*** retains the existing fragments and adds a new fragment which means the existing fragment will be active and they wont be in ‘paused’ state hence when a back button is pressed onCreateView is not called for the existing fragment(the fragment which was there before new fragment was added).

In terms of the fragment’s life cycle events onPause, onResume, onCreateView, and other life cycle events will be invoked in case of replace but they won’t be invoked in case of add.

13) What are retained fragments?

By default, Fragments are destroyed and recreated along with their parent Activities when a configuration change occurs. Calling ***setRetainInstance(true)*** allows us to bypass this destroy-and-recreate cycle, signaling the system to retain the current instance of the fragment when the activity is recreated.

14) Difference between FragmentPagerAdapter and FragmentStatePagerAdapter

1. FragmentPagerAdapter

- Keeps the fragment instance in memory, even if the view is destroyed.
- When the fragment becomes visible again, only the view is recreated, not the fragment itself.
- Uses more memory, especially if there are many fragments.
- Best suited for a small number of static fragments, like fixed tabs.
- Calls detach() method on the fragment, so the fragment stays alive.

Example Use-Case:
Tabs like Home, Profile, Settings.

2. FragmentStatePagerAdapter

- Destroys the fragment instance when it's not visible to the user.
- Saves only the state of the fragment (e.g., scroll position or form input).
- Uses less memory, ideal for large or dynamic fragment lists.
- Best for dynamic content or large data sets.
- Calls remove() method, so the fragment is fully removed from memory.

Example Use-Case:
Image galleries, news apps, quiz apps with many swipeable pages.

Created by Anand Gaur

15) How does the Activity respond when the user rotates the screen?

1. Activity is destroyed and recreated

- When the screen rotates (from portrait to landscape or vice versa), Android **treats it as a configuration change**.
- The **current Activity is destroyed**, and a **new instance is created** in the new orientation.

2. Lifecycle methods are called again

- The system calls the **lifecycle methods** again, just like when the activity is launched for the first time.
- Methods like *onPause(), onStop(), onDestroy()* are called on the old activity.
- Then, the new activity goes through *onCreate(), onStart(), onResume().*

3. Common mistake: Thinking onRestart() is called

- **onRestart() is NOT called** during rotation.
- It is called **only when an activity comes back from the background**, not on rotation.

16) How to prevent data from reloading or resetting when the screen is rotated?

1. Use a ViewModel to store data

- A **ViewModel** is **lifecycle-aware** and **survives configuration changes** like screen rotation.
- When the Activity is recreated due to rotation, the same ViewModel instance is reused.
- This helps you **keep your data alive** without reloading it from scratch.

2. Use onSaveInstanceState() for UI state

- onSaveInstanceState() is used to **save small UI values** like text in an EditText, scroll position, etc.
- These values are stored in a **Bundle**, which is passed back in onCreate() or onRestoreInstanceState().

Created by Anand Gaur

3. How it works together

- Use **ViewModel** to store and manage big or reusable data like API results, list items, user objects, etc.
- Use **onSaveInstanceState()** to save **temporary UI state** like:
    - EditText input
    - Selected tab
    - Scroll position

17) What are Services in Android and their types?

- A Service is an Android component used to perform long-running tasks in the background.
- It does not have a user interface.
- It can keep running even if the user switches to another app.

Example: Playing music, downloading files, tracking location in the background.

Why use a Service?

- To run background tasks **without blocking the UI**.
- Useful for tasks that need to run **continuously or independently**.
- Keeps running **even when the user leaves the app** (based on the service type).

Types of Services in Android:

a) Foreground Service

- Used for **tasks that the user must be aware of**.
- It runs in the background **but shows a notification**.
- Won't be killed easily by the system.

Example: Music player, fitness tracker.

b) Background Service

- Runs **without user interaction or notification**.
- Used for **less important tasks**.
- From Android 8 (API 26+), background services have **restrictions**.
- Recommended to use **WorkManager** for such tasks.

Example: Auto-sync, sending logs in the background.

Created by Anand Gaur

c) Bound Service

- Runs only when another component is connected to it.
- Allows two-way communication between components (like Activity and Service).
- Stops automatically when no component is bound.

Example: App component asking for real-time location updates.

18) Difference between Service and IntentService

1. Service

- Runs on the **main (UI) thread** by default.
- If you do long tasks, it can **freeze or slow down the app**.
- You must **create a separate thread** manually for background work.
- Does **not stop automatically** — you have to stop it yourself.

Use-case:
Use when you need full control or need to run tasks in parallel.

2. IntentService

- Runs on a **background thread automatically**.
- No need to create your own thread.
- Handles one task at a time (in order).
- **Stops by itself** when the task is finished.

Use-case:
Good for simple background tasks like uploading a file or saving data.

19) Difference between AsyncTask and Thread

1. Thread

- Used for **long background tasks**.
- **Cannot update UI directly** from a Thread.
- Does **not handle screen rotation** or configuration changes well.
- Hard to **cancel or manage** once started.

Use for:
Long background work with manual control.

Created By Anand Gaur

2. AsyncTask

- Used for **short background tasks** (less than a few seconds).
- Can **update the UI easily** (onPostExecute).
- Handles simple tasks with **less code**.
- Not good for **long or complex tasks**.

Use for:
Simple tasks like loading data or showing progress.

20) Difference between Service, IntentService, AsyncTask, and Thread

1. Service

- Used for **long-running background tasks**.
- **No UI** and runs even if the app is closed.
- Runs on the **main thread** by default.
- You need to **manage background threads** manually.

2. IntentService

- Subclass of Service that works on a **background thread** automatically.
- Handles **one task at a time**.
- **Stops automatically** when the task is done.
- Good for **simple background tasks**.

3. AsyncTask (Deprecated)

- Used for **short background tasks**.
- Can easily **update the UI** after task completion.
- Runs on **background thread**, but starts and ends on the **main thread**.
- Not good for **long or multiple tasks**.

4. Thread

- Used to run **any background task**.
- Runs separately from the main thread.
- Cannot update UI directly.
- You have to **handle everything manually** (start, stop, update UI).

Created by Anand Gaur

21) What are Handlers in Android?

1. Handler is used to manage communication between threads

- Handlers help in sending messages or runnables from a background thread to the main (UI) thread.
- Mainly used when you want to update UI from a background thread.

2. Works with MessageQueue and Looper

- A Handler uses a Looper and a MessageQueue to handle tasks one by one.
- It can post tasks with delay or run tasks repeatedly at fixed intervals.

3. Runs outside the Activity lifecycle

- Handlers are not tied to the Activity or Fragment lifecycle.
- So you need to manually clean them in onDestroy() to avoid memory leaks.

4. When to use Handler

- When you need to:
    - Update UI from background thread
    - Delay a task
    - Run a task repeatedly after every few seconds or minutes

22) Difference between Serializable and Parcelable in Android

1. Serializable

- It is a *standard Java interface.*
- Used to convert an object into a stream of bytes, so it can be saved or passed between activities.
- You just implement Serializable, and Java handles the rest.
- Uses reflection, which creates more objects and slows down performance.
- Creates more garbage, leading to more garbage collection.
- Easier to use but not optimized for Android.

Created by Anand Gaur

2. Parcelable

- It is an *Android-specific interface.*
- Used to serialize objects manually for passing data between components like Activity or Fragment.
- You have to write your own logic for how the object should be written and read.
- Does not use reflection, so it is faster and more memory-efficient.
- Recommended in Android development when you need better performance.

23) How would you update the UI of an Activity from a Background Service?

1. Use LocalBroadcastReceiver

- Register a **LocalBroadcastReceiver** in the Activity.
- From the Service, **send a broadcast using an Intent** with the data.
- The receiver catches the data and **updates the UI**.
- This works **only when the Activity is in the foreground**.

2. Unregister the Receiver

- Always **unregister the BroadcastReceiver** in the onStop() method.
- This helps to **avoid memory leaks** or crashes when the activity is not visible.

3. Use Handler (Optional Method)

- You can also create a **Handler in the Activity** and pass it to the Service.
- Then the Service can send messages to the Handler, and the Activity can update the UI.
- This method requires **binding the service** and is used less commonly now.

24) What is a Sticky Intent in Android?

- A **Sticky Intent** is a type of broadcast **that stays in the system** even after it has been sent.
- sticky intents are associated with the android system for the future broadcast events.
- This means **other components can receive the last broadcasted data**, even if they weren't registered when the broadcast was sent.

Created by Anand Gaur

How it works

- It is sent using sendStickyBroadcast(Intent).
- When another component calls registerReceiver() for that action, it **immediately receives the last sent intent**.

Example

- Suppose the system sends a sticky broadcast for ACTION_BATTERY_CHANGED.
- Even if your app registers **after** the broadcast was sent, you can still get the **last known battery state** immediately.

Important Note

- **Sticky broadcasts are deprecated** from API level 21 (Lollipop) for security and performance reasons.
- You should use **other communication methods** like LiveData, BroadcastReceiver with system broadcasts, or bound services.

25) What is a PendingIntent?

- A PendingIntent is a special type of Intent used to perform an action in the future.
- It allows another app or the Android system to perform an action on behalf of your app.
- Even if your app is not running, the PendingIntent can still work.

Commonly used in:

- **Notifications** – to open an activity when user taps it
- **AlarmManager** – to run something at a scheduled time
- **Broadcasts** – to send data in the future

26) What are Intent Filters?

- Intent Filters are used to tell Android which intents an activity, service, or broadcast receiver can handle.
- They are defined in the **AndroidManifest.xml** file.
- The system uses intent filters to decide which component should respond to a specific intent.
- **For example**, if a user clicks on a web link, the system checks the intent filters to find an app that can handle it.

Created by Anand Gaur

- An intent filter includes:
    - **action** – what kind of action (like view, send, etc.)
    - **category** – extra information (like default)
    - **data** – type of data (like http, tel, etc.)

27) What is a BroadcastReceiver in Android?

- A **BroadcastReceiver** is a component in Android that **listens for system-wide or app-specific broadcast messages** (called Intents).
- It helps your app **respond to events**, even if your app is not currently open.
- You can use it to listen to **system events** like:
    - Phone is charging
    - Battery is low
    - Internet connectivity changes
    - Device boot completed
- You can also create and send **custom broadcasts** within your own app.
- It does **not show any UI**, but you can use it to start a service, show a notification, or launch an activity.

28) What are Loaders in Android?

- **Loaders** are used to **load data in the background** from a data source (like a database or content provider).
- They were introduced in **API level 11 (Android 3.0)**.
- Loaders help to **avoid running long tasks on the main thread** (which can freeze the UI).
- They are commonly used with **CursorAdapters** to load data into list-based views.
- Loaders can **automatically reconnect to the last loaded data** after a configuration change (like screen rotation), so they prevent **duplicate queries**.
- Loaders are managed by **LoaderManager**, which handles the lifecycle.

29) What are Launch Modes in Android ?

In Android, **launch modes** decide **how activities are created and managed** in the back stack when you open or reopen them.

Created by Anand Gaur

• Standard (Default)

- A **new activity instance is always created**, even if it's already in the stack.
- You can have **multiple copies** of the same activity.

Example:
If current stack is: A → B → C
You launch **B** again with **standard**, new stack becomes:
A → B → C → B

Use Case: Chat screen in a messaging app (like WhatsApp)

- Suppose you're chatting with the same person and open their chat screen multiple times using different paths.
- Each time, a **new instance** of the chat screen (activity) is created.

• SingleTop

- Works like **standard**, but if the **same activity is already on top**, it won't create a new one.
- It will reuse the existing one by calling onNewIntent().

Example:
Stack: A → B → C
Launch **C** again with **singleTop** → still: A → B → C (no duplicate)
But if stack is A → B → C, and you launch **B** again with **singleTop**, new stack becomes:
A → B → C → B (because B was not on top)

Use Case: Notifications in Gmail

- You tap on a Gmail notification, and it opens the **Email Detail screen**.
- If the user is already on that same screen (top of stack), **no new instance is created**.
- Instead, the existing screen is updated with new data using `onNewIntent()`.

• SingleTask

- Only **one instance** of the activity exists at a time in the task.
- If it's already in the stack, all activities **above it are removed**, and that activity is reused.

Created by Anand Gaur

Example:
Stack: A → B → C → D
Launch **B** with **singleTask** → stack becomes:
A → B (C and D are removed)

Use Case: Splash screen or Login screen

- You log into an app, and the login activity is launched with `singleTask`.
- Now after login, you go to the main dashboard.
- If you accidentally launch the login activity again (like by deep link), **all screens above it are removed**, and you're brought back to login.

● SingleInstance

- Same as **singleTask**, but the activity is launched in its **own separate task**.
- No other activities will be in this task.

Example:
Stack: A → B → C → D
Launch **B** with **singleInstance** →

- Task 1: A
- Task 2: B (in its own task)

Use Case: Video player or call screen (like Zoom, Google Meet, or YouTube PiP)

- You open a video or a call screen that should run in a **separate task**.
- No other activity should be part of this task.
- If you press Home and come back to the app, the video/call continues in the separate task.

30) What is ConstraintLayout?

- ConstraintLayout is a **layout in Android** that lets you design **complex UIs without nesting** multiple layouts.
- It helps to create **flat and efficient layouts**, which means better performance.
- Similar to RelativeLayout, but **more powerful and flexible**.
- You position views by creating **constraints** between:
    - one view and another view
    - or a view and the parent layout
- It supports advanced features like:
    - Chains (for evenly spaced items)

Created by Anand Gaur

    - Guidelines (for alignment)
    - Barriers (for dynamic layout)

31) How to support different screen sizes in Android?

To make sure your app looks good on **all devices** (phones, tablets, foldables), follow these tips:

- **Use ConstraintLayout**
    - It helps you create flexible layouts that adjust automatically to different screen sizes.
    - Views can move, align, or stretch based on the screen size.
- **Use wrap_content and match_parent**
    - Don't hard-code fixed width/height like 200dp.
    - Let the view size itself automatically or fill available space.
- **Create alternative layouts**
    - Use different layout files for tablets and large screens.
    - Example:
        - res/layout/main_activity.xml → for phones
        - res/layout-sw600dp/main_activity.xml → for tablets (7 inch or more)
- **Use "smallest width" qualifier (sw)**
    - Android picks the layout file based on screen size.
    - Example:
        - sw600dp means screens that are at least 600dp wide (like tablets)
- **Use 9-patch images**
    - These images can stretch without breaking.
    - Great for backgrounds and buttons that need to scale.

32) What is an ANR (Application Not Responding) error, and how to prevent it?

- **ANR (Application Not Responding)** is an error shown when your app's **UI becomes unresponsive** for more than **5 seconds**.
- This usually happens when you **do heavy work on the main thread**, such as:
    - Downloading data
    - Reading large files
    - Long database operations

Created by Anand Gaur

How to prevent ANR?

- **Never block the main (UI) thread.**
  Do heavy work like API calls, file read/write, and database queries on background threads.
- **Use background tools like:**
  - AsyncTask (deprecated but still asked in interviews)
  - HandlerThread
  - IntentService (deprecated, replaced by WorkManager)
  - Coroutines (recommended now)
  - WorkManager or ExecutorService
- **For UI updates**, always come back to the **main thread** using runOnUiThread() or Main dispatcher (in Kotlin coroutines).

33) What is a Singleton Class in Android?

- A **singleton class** is a class that **creates only one object (instance)** for the entire app.
- This **single instance is shared** by all other classes that need it.

Why use a Singleton?

- To **share one object globally** (e.g., shared preferences manager, Retrofit client, database instance)
- To **save memory** and avoid creating multiple unnecessary objects

Properties of Singleton:

- Only **one instance** is created
- That instance is **accessible globally**
- Same instance is used by all parts of the app

Rules to create a Singleton class:

- Use a **private constructor** so no one else can create an object
- Create a **static reference** of the class inside it
- Add a **public static method** to return that single instance
- Make sure it's **thread-safe** (only one instance even in multi-threaded access)

Created by Anand Gaur

34) What's the difference between commit() and apply() in SharedPreferences?

commit()

- Saves data **synchronously** (happens immediately on the main thread).
- **Returns a boolean** — true if the save was successful, false otherwise.
- It can **slow down the UI** if used on the main thread (not recommended).
- Use it **only if you need to know immediately** whether the save was successful.

apply()

- Saves data **asynchronously** (in the background).
- **Does NOT return** any result (no success/failure).
- It is **faster and better for performance**.
- Changes are committed **in memory first**, then written to disk in the background.
- Safe to use on the **main/UI thread**.

35) How does RecyclerView work?

- RecyclerView is used to **show a long list or grid of items efficiently** (like messages, contacts, posts, etc.).

Problem with creating all views:

- If you have 100 items and create 100 views, it's **slow and memory-heavy**.
- Only a few items (like 10) are visible on the screen — the rest are **off-screen** and not needed immediately.

RecyclerView solution:

- RecyclerView only creates the **visible items** (like 10 views).
- As you **scroll**, it **reuses (recycles)** the views that go off-screen instead of creating new ones.
- This saves **memory** and gives a **smooth scrolling experience**.

How recycling works:

- Let's say the view for item 1 goes off-screen when you scroll.
- Instead of destroying it, RecyclerView **reuses that same view** to show item 11 (or next item).
- The data is replaced using *onBindViewHolder()* method.

Created by Anand Gaur

Key methods:

- **onCreateViewHolder()** – called **only when a new view** is needed.
- **onBindViewHolder()** – called to **bind new data** to an existing (recycled) view.
- **getItemCount()** – tells how many total items are there in the list.

36) How does RecyclerView differ from ListView?

1. ViewHolder Pattern

- **RecyclerView** uses the **ViewHolder pattern by default**, which improves performance.
- In **ListView**, using ViewHolder is optional (you have to implement it manually).

What is ViewHolder Pattern?

- It saves reference to item views so we **don't call findViewById() repeatedly**.
- This makes scrolling **faster and smoother**.

2. LayoutManager Support

- **ListView** only supports **vertical lists**.
- **RecyclerView** supports:
    - **LinearLayoutManager** – vertical/horizontal list
    - **GridLayoutManager** – grid views
    - **StaggeredGridLayoutManager** – Pinterest-style layout

Result: RecyclerView is **more flexible** than ListView.

3. Item Animations

- **RecyclerView** has **built-in animation support** (e.g., when items are added, removed, or moved).
- **ListView** doesn't support these animations well.

4. Item Decoration

- **RecyclerView** lets you add custom **dividers, spacing, and decorations** easily using ItemDecoration.
- In **ListView**, you had to do it manually or through styling.

Created By Anand Gaur

5. More Control and Performance

- **RecyclerView** gives **more control** to developers (e.g., scroll behavior, nested scrolling, custom animations).
- It is **more efficient** and modern than ListView.

37) What is AAPT in Android?

- **AAPT** stands for **Android Asset Packaging Tool**.
- It's a **build tool** used during the APK creation process.

What does AAPT do?

- It **collects and processes all app resources** (like XML files, images, strings).
- It **converts** them into a **binary format** so that Android can read and use them efficiently.
- It also **packages everything** into the final .apk file.

AAPT can:

- **View** contents of .apk, .jar, or .zip files
- **Compile** XML and resource files into binary format
- **Package** resources into .apk
- Generate R.java file that helps access resources in code

38) What is AIDL? And which data types are supported by AIDL?

- **AIDL** stands for **Android Interface Definition Language**.
- It is used for **Inter-Process Communication (IPC)** — when two different apps (or processes) need to talk to each other in Android.
- AIDL lets you define **methods** that can be called **from another process**, like a remote service.
- It **breaks complex objects into simple types** (called "marshalling") so Android OS can understand and send them across process boundaries.

Real-life Example:

If App A wants to bind to a **Service** running in App B and send/receive data — AIDL is used to define that communication.

Created by Anand Gaur

Data Types Supported by AIDL:

You can use the following data types:

- **Java primitives**: int, long, float, double, boolean, char
- **String**
- **CharSequence**
- **List** (must be typed, e.g., List<String>)
- **Map** (must be typed, e.g., Map<String, String>)
- **Parcelable** objects (custom model classes that implement Parcelable)

❗ Note: You **cannot** pass regular objects or non-serializable/parcelable data.

39) What is JobScheduler in Android?

- **JobScheduler** is an Android API used to **schedule background tasks/jobs** to run at the best time.
- It helps **save battery and system resources** by letting Android decide **when** the task should run.

What does JobScheduler do?

- Runs jobs **in your app's process**, even if your app is not running in the foreground.
- Lets the system **batch jobs together** from multiple apps — this reduces **CPU wakeups and battery use**.
- You can set **conditions** for when the job should run.

Common use-cases:

- Run a task **only when the device is charging** (e.g., backup files).
- Sync data **only on Wi-Fi** (to save mobile data).
- Do some clean-up task **every 6 hours** (even if timing isn't exact).

Created by Anand Gaur

40) What are the different storage methods in Android?

Android provides multiple ways to store **persistent data** (data that stays even after the app is closed or device restarts).

Here are the main options:

Shared Preferences

- Stores **small key-value** pairs of primitive data (e.g., String, int, boolean).
- Best for things like **login state**, **settings**, or **user preferences**.
- Data is stored **privately** in your app's internal storage.

Example: Saving **isUserLoggedIn = true**

Internal Storage

- Saves files **privately** inside your app's storage space.
- Other apps **cannot access** these files.
- Used for storing text, images, or files you don't want shared.

Example: Saving a profile image or configuration file

External Storage

- Saves files to **shared storage** like SD card or shared folders.
- Can be accessed by **other apps** (with permission).
- Good for media files (images, videos, documents) that users might want to access outside your app.

Example: Downloaded PDF, exported files

SQLite Database

- Stores **structured data** in the form of tables (like a small local database).
- Used for complex data like **contacts, user history, or cache**.
- All data is private to your app.

Example: Chat history, notes, offline records

Created by Anand Gaur

41) What is WorkManager in Android?

WorkManager is a background processing library in Android. It is used to run tasks that should be executed reliably, even if:

- The app is closed
- The phone restarts
- The task doesn't need to run immediately but must be completed

Key Features of WorkManager:

- **Guaranteed execution:** The task will run eventually, even if the device reboots or app is killed.
- **Constraints support:** You can set conditions like run only when:
    - Device is charging
    - Device is connected to Wi-Fi
    - Battery is not low, etc.
- **Task chaining:** You can schedule tasks to run one after another (Task A, then Task B, and so on).
- **Monitor task status:** You can track if a task is running, completed, or failed.
- **Backward compatible:** It works on all Android versions (API 14 and above), no need to handle version-specific code.

When to use WorkManager:
Use WorkManager for tasks that are:

- Required to run even if the app exits or device restarts
- Not time-critical but must eventually complete
- Examples: data sync, uploading logs, sending analytics

42) Difference between Picasso and Glide

Picasso

- Lightweight image loading library from Square
- Simple to use with minimal setup
- Slower when loading large images
- Suitable for basic use-cases (like profile pictures, thumbnails)
- Supports WebP format
- Does not support GIFs
- Smaller cache size

Created by Anand Gaur

- Handles memory efficiently but **not as optimized** as Glide
- No memory reuse (no BitmapPool)
- UI remains smooth and responsive

Glide

- Developed by Google; more powerful than Picasso
- Optimized for **loading large images** and handling scrolling lists
- Supports **GIFs** and **WebP** formats
- Has a **BitmapPool** – reuses image memory, reducing garbage collection
- Caches images more efficiently (both memory and disk cache)
- Better suited for **image-heavy apps** like social media or e-commerce
- Slightly larger library size than Picasso
- UI also stays smooth and responsive

43) What is View Lifecycle in Android?

The **View Lifecycle** in Android refers to the series of method calls that occur from the moment a **View** (like TextView, ImageView, or custom view) is **attached to the screen**, until it is **removed** or **destroyed**.

It's important especially when you are creating **custom views**, handling **drawing**, or optimizing **UI performance**.

Key Stages in the View Lifecycle:

1. onAttachedToWindow()

- Called when the view is attached to a window (i.e., becomes part of the screen).
- Good place to **start animations or register listeners**.

2. onMeasure(int widthMeasureSpec, int heightMeasureSpec)

- Called to **calculate the size** of the view and its children.
- You can use setMeasuredDimension(width, height) to define how big the view should be.

3. onLayout(boolean changed, int left, int top, int right, int bottom)

- Called to **position child views** inside the parent view.
- This defines where each child appears on the screen.

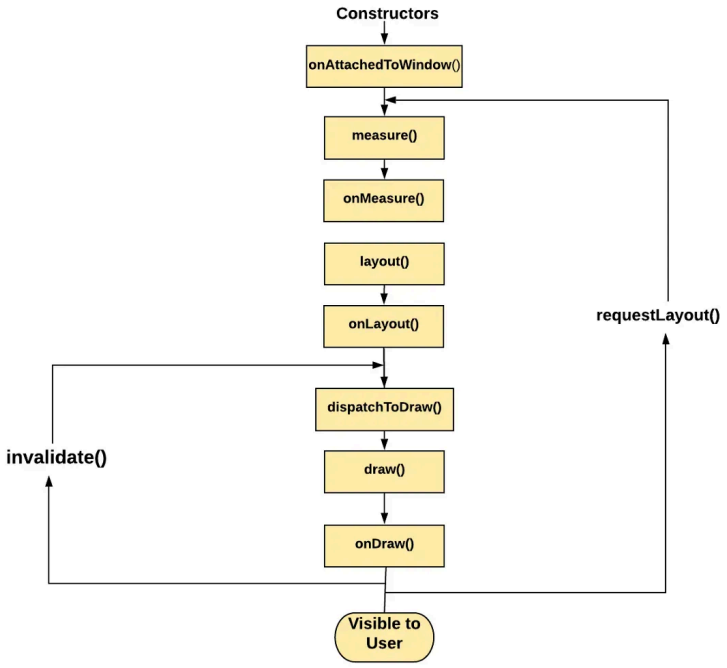
Created by Anand Gaur

4. onDraw(Canvas canvas)

- Called to **draw the content** of the view.
- You use the Canvas object to draw shapes, text, images using Paint.

5. onDetachedFromWindow()

- Called when the view is **removed** from the screen or no longer visible.
- Good place to **clean up resources** (like stopping animations or unregistering listeners).



44) What is the Zygote process in Android?

Zygote is a special system process that starts when Android boots up. It is responsible for launching new Android applications.

How it works:

- Zygote starts first and loads core libraries.
- When a new app is started, Android forks the Zygote process.
- This makes app startup faster by sharing common memory (like preloaded classes).

Key benefit: Speeds up app launch time and reduces memory usage.

45) What is Binder in Android?

Binder is the underlying **IPC (Inter-Process Communication)** mechanism in Android that allows communication between different processes.

Example:

- Activity in one app communicates with a Service in another app using Binder.

It forms the foundation of system services like LocationManager, TelephonyManager, etc.

46) What is ANR Watchdog?

The Android OS includes an ANR (Application Not Responding) watchdog thread that monitors the main thread.

If the main thread is blocked for more than **5 seconds**, the system shows an ANR dialog.

This is to prevent poor user experience due to UI freezing.

47) What is Looper, Handler, and MessageQueue?

- **Looper**: Prepares a thread (usually the main/UI thread) to run a loop and listen for messages.
- **Handler**: Sends or processes messages (Runnable or Message).
- **MessageQueue**: Queue that holds tasks/messages to be executed sequentially.

Used heavily in main thread (UI thread) and background threads for task scheduling.

Created by Anand Gaur

48) What is the main thread in Android?

The **main thread** (UI thread) is responsible for:

- Drawing UI elements
- Handling user interactions
- Updating views

Blocking this thread (e.g., doing network or DB operations) leads to ANR.

49) What is the difference between ApplicationContext and BaseContext?

- **ApplicationContext**:
  - Lives as long as the app does.
  - Use for app-level operations like Toast, DB, etc.
- **BaseContext**:
  - Internal context tied to the base implementation of a ContextWrapper (e.g., for Activities, Services).

Usually, you use `getApplicationContext()` for global use cases and `this` (Activity context) for UI-related cases.

50) What is the purpose of the AndroidManifest.xml file?

It is the configuration file that tells the Android system:

- What components your app has (Activity, Service, Receiver, etc.)
- Which permissions your app needs
- App's entry point (`<intent-filter>` for LAUNCHER)
- Minimum SDK version

Created by Anand Gaur

- Broadcast receivers, deep links, etc.

Without it, Android won't recognize or run your app correctly.

51) What happens when you press the Home button in Android?

- The current activity goes into **onPause()** and then **onStop()**.
- The app is pushed to the background.
- It is **not killed** immediately — Android may keep it in memory.

If the system needs memory, it may kill the process later.

52) What is a Task and Back Stack in Android?

- **Task**: A collection of Activities arranged in a stack (back stack).
- **Back Stack**: Works like a stack (LIFO) — last activity added is shown on top.

When a user presses back, the system pops the current activity off the stack and resumes the previous one.

53) What is the role of the Package Manager in Android?

The **PackageManager** is a system service that:

- Installs, updates, and uninstalls apps
- Grants permissions
- Queries for app info (version, permissions, etc.)
- Starts app components

Example:

```
val packageManager = context.packageManager
val info = packageManager.getPackageInfo("com.example.app", 0)
```

Created by Anand Gaur

54) What is the Dalvik Virtual Machine (DVM)?

Before Android 5.0 (Lollipop), apps used **DVM** instead of ART.

- Dalvik executed `.dex` bytecode (converted from `.class` files).
- It used **Just-In-Time (JIT)** compilation.
- Less efficient compared to ART.

Now replaced by **ART** (Android Runtime) for better performance.

55) What is Android Runtime (ART)?

ART replaced Dalvik in Android 5.0+.

- Uses **Ahead-Of-Time (AOT)** compilation at install time.
- App code is compiled into native code ahead of time, resulting in faster execution.
- Uses less battery and memory.

56) What is the difference between foreground and background processes?

- **Foreground Process**: Has UI interaction (Activity in front). Highest priority.
- **Background Process**: Not visible, doing some work (like Service). Lower priority.
- If Android runs low on memory, background processes are the first to be killed.

57) What is the role of `proguard-rules.pro`?

ProGuard shrinks, obfuscates, and optimizes your code.

Created by Anand Gaur

`proguard-rules.pro` lets you:

- Keep certain classes/methods from being removed
- Avoid obfuscation for libraries (like Retrofit or Gson)
- Prevent runtime crashes from reflection-based APIs

58) What are `.dex` files?

`.dex` (Dalvik Executable) files contain bytecode that Android uses to run your app.

- Java/Kotlin → Compiled to `.class` files → Then converted to `.dex`
- Multiple `.dex` files possible (multidex support)

59) What is Android Boot Process?

1. **Bootloader**: Initializes hardware
2. **Kernel**: Starts core system
3. **init**: Launches system daemons
4. **Zygote**: Loads Android runtime and system classes
5. **System Server**: Starts core Android services
6. **Launcher**: Displays home screen

60) What is the difference between compile, implementation, and api in Gradle?

- `compile`: Deprecated. Previously used to add dependencies.
- `implementation`: Hides the dependency from other modules. Safer and faster compilation.
- `api`: Exposes the dependency to other modules that depend on your module.

Use `implementation` unless you intentionally want to expose that dependency to other modules.

Created By : Anand Gaur

61) What are the advantages of using WorkManager?

- Runs even if the app is not running
- Automatically chooses the best way to run the task (JobScheduler, AlarmManager, etc.)
- Supports constraints (like network, charging)
- Supports retries and chaining of tasks
- Persists through device reboots (with `setPersisted(true)`)

62) What are the main types of WorkRequests in WorkManager?

- `OneTimeWorkRequest`: Executes a task only once
- `PeriodicWorkRequest`: Executes a task at regular intervals (e.g., every 15 minutes)

63) What are the Result types in WorkManager?

- `Result.success()`: Task completed successfully
- `Result.failure()`: Task failed permanently
- `Result.retry()`: Task failed but should retry later

64) What is PeriodicWorkRequest and when to use it?

It's used for running tasks repeatedly at fixed intervals.
Minimum repeat interval is **15 minutes**.
Useful for syncing data every few hours/days.

Created by Anand Gaur

65) What are the different states of Work in WorkManager?

- `ENQUEUED`
- `RUNNING`
- `SUCCEEDED`
- `FAILED`
- `BLOCKED`
- `CANCELLED`

66) What are Constraints in WorkManager and how to use them?

Constraints control when the work should run.

Example: Only run when device is charging and connected to network.

```
val constraints = Constraints.Builder()
    .setRequiresCharging(true)
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build()
val request = OneTimeWorkRequestBuilder<MyWorker>()
    .setConstraints(constraints)
    .build()
```

67) What is the difference between Worker and CoroutineWorker?

- `Worker`: Uses blocking `doWork()`

- `CoroutineWorker`: Allows suspending functions (non-blocking, better with coroutines)

68) How does WorkManager differ from JobScheduler?

WorkManager and **JobScheduler** are both used to perform background tasks, but they differ in terms of **compatibility, reliability, and flexibility**.

WorkManager:

- **Jetpack Library** introduced by Google for background task management.
- **Backwards compatible**: Works on devices running Android **API 14+**.
- Can **persist work across device reboots**.
- **Automatically chooses the best API** under the hood (JobScheduler, AlarmManager, FirebaseJobDispatcher).
- **Supports chaining, constraints, retry logic, periodic work**, and more.
- Best for **guaranteed**, deferrable tasks like syncing data, uploading logs, or scheduled notifications.

JobScheduler:

- Introduced in **API 21 (Lollipop)**.
- Only available on **Android 5.0+**.
- Does **not support backward compatibility** natively.
- Cannot easily **chain tasks** or observe work lifecycle with LiveData.
- Limited in comparison to WorkManager in terms of **features and flexibility**.

Created by Anand Gaur

Kotlin Concepts

1) What are the main features of Kotlin?

- **Concise**: Less boilerplate than Java
- **Null Safety**: Built-in null checks
- **Extension Functions**: Add functions to existing classes
- **Coroutines**: Lightweight concurrency
- **Smart Casts**: No need for explicit casting after type check
- **Data Classes**: Auto-generate equals(), hashCode(), toString(), etc.
- **Default & Named Arguments**
- **Higher-order functions & Lambdas**

2) What is the difference between `val`, `var`, and `const` in Kotlin?

In Kotlin, `val` and `var` are used to declare variables, but they behave differently:

1. `var` (Variable)

- A **mutable** variable.
- You **can change** its value after it's assigned.
- Stored in **memory at runtime**.

2. `val` (Value)

- An **immutable** variable (like `final` in Java).
- You **can assign only once**.
- Value is also stored **at runtime**, but can't be reassigned.

Created by Anand Gaur

3. `const val` (Constant)

- A **compile-time constant**.
- Can only be used **with top-level properties** or **inside objects or companion objects**.
- **Must be of a primitive type or String**, and value must be known **at compile time**.

```
val name = "Anand" // Cannot be changed later
var age = 30       // Can be updated
age = 31
```

3) What are null safety features in Kotlin?

Kotlin eliminates `NullPointerException` (NPE) by making all types **non-nullable** by default.

Types:

- **Non-nullable:** `var name: String = "Anand"` → cannot hold null
- **Nullable:** `var name: String? = null` → can hold null

Safe Operations:

- **Safe call `?.`**: Skips execution if the object is null.
- **Elvis `?:`**: Provide default value if null.
- **Not-null Assertion `!!`**: Throws Null Pointer Exception if value is null.
- **Safe Cast `as?`**: Returns null instead of throwing ClassCastException.

4) What is a `data class` in Kotlin?

A `data class` is a special class made specifically for storing data. It automatically gives you useful methods like:

Created by Anand Gaur

`toString()` – so you can print the object easily

`equals()` and `hashCode()` – to compare objects or use in HashMap/Set

`copy()` – to create a new object with some properties changed

`componentN()` – to access values using destructuring (like `val (a, b) = obj`)

Syntax:

```
data class User(val name: String, val age: Int)
```

5) What is the difference between == and === in Kotlin?

== (Structural Equality)

- Checks if **two objects have the same content**.
- Internally calls the `.equals()` method.

=== (Referential Equality)

- Checks if **two references point to the same object in memory**.
- Like checking object identity (`===` is the same as Java's `==`).

```
val a = "hello"
val b = "hello"
println(a == b)   // true (same content)
println(a === b)  // might be true (same object in memory)
```

Always use `==` unless you're doing memory-level comparison.

Created by Anand Gaur

6) What is the Elvis Operator (`?:`) in Kotlin?

The Elvis operator `?:` in Kotlin is used to return a default value when an expression is `null`.

It's a shortcut for an if-else check when you want to assign a default value to a nullable variable.

Syntax:

```
val result = someNullableValue ?: defaultValue
```

- If `someNullableValue` is not null, it will be used.
- If it is null, then `defaultValue` will be used.

7) Do we have a ternary operator in Kotlin like in Java?

No, Kotlin does not have a ternary operator (`condition ? trueValue : falseValue`) like Java.

But you don't need it — Kotlin replaces it with a simple `if-else` expression, because in Kotlin, `if` is an expression, not just a statement.

8) What is the difference between Safe Call (`?.`) and Not-null Assertion (`!!`) in Kotlin?

In Kotlin, we deal with **nullable variables** using two main operators:

Safe Call Operator (`?.`)

- Used **when a variable might be null**.
- It **safely accesses** a method or property **only if the object is not null**.
- If the variable is null, the whole expression returns **null** instead of crashing.

Created by Anand Gaur

Not-null Assertion Operator (`!!`)

- Used when you are **absolutely sure** that a variable is **not null**.
- It **forces** Kotlin to treat a nullable variable as non-null.
- If the variable **is null**, your app will **crash** with a `NullPointerException`.

9) Can we use primitive types like `int`, `double`, and `float` in Kotlin?

- No, **Kotlin doesn't use Java-style primitive types directly** (`int`, `double`, `float` etc.).

- Instead, Kotlin uses **classes** like `Int`, `Double`, `Float`, etc.

10) What is String Interpolation in Kotlin?

String Interpolation means inserting variables or expressions **directly inside a string**.

In Kotlin, you can use:

- `$variableName` to include a variable

- `${expression}` to include logic/calculation

11) What are the different visibility modifiers in Kotlin?

In Kotlin, visibility modifiers control who can access classes, functions, properties, or variables.
There are **4 main modifiers**:

- **public**:
  This is the **default** visibility modifier.
  Public declarations are **accessible from anywhere** in the code.

- **private**:
  Private declarations are only **accessible within the same file** (for top-level declarations)
  or within the **same class or object**.

Created by Anand Gaur

- **protected**:
  Protected declarations are **accessible within the same class and its subclasses**.
  They are not visible outside the class hierarchy and cannot be used for top-level declarations.

- **internal**:
  Internal declarations are **visible only within the same module**.
  A module is a group of Kotlin files compiled together — like an IntelliJ module or a Gradle module.

12) When to use the `lateinit` keyword in Kotlin?

`lateinit` means **late initialization** — it allows you to initialize a non-null variable **after** the object is created.

When to use `lateinit`:

- When **you can't initialize the variable at the time of declaration**, but you are **sure it won't be null** when accessed.

- Useful in cases like:
  - Dependency Injection
  - Initializing views in Android
  - Setting up values in unit tests (e.g., `@Before` methods)

Why use `lateinit`?

Normally, non-null properties in Kotlin must be initialized in the constructor.
But in many real-world cases (like Android or testing), it's not possible or practical.

So instead of using nullable types (`var name: String? = null`) and checking for null every time, you can write:

```
lateinit var name: String
```

And initialize it later, without worrying about null safety.

Created by Anand Gaur

Rules:

- `lateinit` only works with **`var`** (not `val`)
- Only for **non-nullable types**
- Cannot be used with **primitive types** like `Int`, `Boolean`, etc.

13) What is a `lateinit` property in Kotlin?

A `lateinit` property in Kotlin is a way to declare a **non-null variable** without initializing it right away.

Key Points:

- Normally, non-null variables must be initialized when declared.
- But sometimes (e.g., in Android, dependency injection, or testing), you may want to **assign the value later**.
- `lateinit` lets you do that — it tells the compiler: “I’ll initialize this variable later, trust me.”

Rules:

- Only works with `var` (not `val`)
- Only for **non-nullable types**
- Can't be used with **primitive types** like `Int`, `Float`, etc.
- If you try to access it before initialization, you'll get an `UninitializedPropertyAccessException`.

14) How to check if a `lateinit` variable has been initialized or not?

In Kotlin, you can check whether a `lateinit` variable has been initialized using the **`::propertyName.isInitialized`**

Created by Anand Gaur

Example:

```
lateinit var name: String
fun checkName() {
    if (::name.isInitialized) {
        println("Name is initialized: $name")
    } else {
        println("Name is not initialized yet")
    }
}
```

- You must use **:: (property reference operator)** before the variable name.
- This check only works with **lateinit var**.
- If you try to access a lateinit variable before initializing it **without checking**, it will throw UninitializedPropertyAccessException.

15) What is the difference between `lateinit` and `lazy` initialization in Kotlin?

Both `lateinit` and `lazy` are used for delayed initialization — but they are different in how and when they work.

`lateinit`:

- Used with **`var`** (mutable variable)
- Used for **non-null** variables that are **initialized later manually**
- Typically used when initialization depends on some lifecycle method (like in Android or unit tests)

```
lateinit var name: String
fun init() {
    name = "Anand"
}
```

- **No return value** like lazy; you just assign manually
- You can check with ::name.isInitialized

`lazy`:

- Used with **`val`** (read-only, immutable variable)
- The value is initialized **automatically** when it's first accessed
- Useful for **expensive operations** or **heavy calculations**

```
val name: String by lazy {
    println("Calculating name")
    "Anand"
}
```

The lambda block is called only once — the result is cached

16) What is the use of the `lateinit` modifier in Kotlin?

The `lateinit` modifier is used in Kotlin to declare **non-null variables** that will be initialized **later**, instead of at the time of declaration.

Key Points:

- It is used with **`var`** (mutable) variables.
- It is only allowed for **non-nullable** types (e.g., `String`, `View`, etc.)
- You use it when you **can't initialize the variable in the constructor** or immediately.
- Commonly used in **Android development**, **dependency injection**, or **unit testing**.

Created by Anand Gaur

17) Explain the concept of lazy initialization in Kotlin

Lazy initialization in Kotlin means **delaying the creation of a variable's value until the first time it is used**. It helps **optimize performance and memory** by avoiding unnecessary computation or loading.

Key Points:

- Used with `val` (read-only) variables.
- Declared using:

```
val variableName: Type by lazy { initializer }
```

- The code inside the `lazy` block runs **only once**, and the result is **cached** for future use.

18) What is the purpose of the `by lazy` function in Kotlin?

The `by lazy` function in Kotlin is used for **lazy initialization** — meaning the value of a property is **only calculated when it's accessed for the first time**.

Purpose:

- To **delay heavy or unnecessary computation** until it's really needed.
- To **improve performance** by avoiding the creation of unused objects.
- To **cache** the result — once computed, the value is stored and reused.

Syntax:

```
val variableName: Type by lazy {
    // initialization logic
}
```

Created by Anand Gaur

19) What is `forEach` in Kotlin?

In Kotlin, `forEach` is a **higher-order function** used to **iterate over collections** like lists, sets, or arrays — similar to the for-each loop in Java.

Purpose:

- To perform an action on **each element** of a collection.
- A **cleaner and more functional way** to loop through items.

Syntax:

```
collection.forEach { item ->
    // do something with item
}
```

20) What are companion objects in Kotlin?

In Kotlin, a companion object is used when you want to define members (like functions or variables) that belong to a class itself, not to instances of the class — similar to `static` members in Java.

Why Use Companion Objects?

- To call methods or access variables **without creating an object** of the class.
- Useful for **utility functions**, **factory methods**, or **constants** inside a class.

How to Declare a Companion Object:

```
class MyClass {
    companion object {
        val appName = "MyApp"

        fun printMessage() {
            println("Hello from companion object!")
        }
    }
}
```


Created by Anand Gaur

21) What is the equivalent of Java static methods in Kotlin?

Kotlin does not have `static` keyword like Java. But you can achieve the same functionality using these three options:

1. Companion Object

A companion object allows you to define functions and properties that belong to a class — just like static members in Java.

2. Package-Level Function

In Kotlin, you can write functions **directly inside a `.kt` file**, outside any class. These behave like static methods.

3. Object Declaration (Singleton)

If you need a singleton utility class (just one instance), you can use `object` keyword.

22) What is the use of `@JvmStatic`, `@JvmOverloads`, and `@JvmField` in Kotlin?

These annotations are used in Kotlin when you're working with **Java interoperability**, i.e., when Java code needs to call Kotlin code.

`@JvmStatic`

- Used to **expose a Kotlin method as a static method** to Java.
- Typically used inside a **companion object** or an **object declaration**.

`@JvmOverloads`

- Used to **generate overloaded methods** for Java when Kotlin functions have **default parameters**.
- Kotlin supports default values, but Java does not.

Created by Anand Gaur

`@JvmField`

- Used to expose a **Kotlin property as a field** to Java **without generating getter/setter** methods.
- Can only be used on `val` **or** `var` **declared at the top-level, in** `object`**, or inside companion objects**.

23) What is Smart Cast in Kotlin?

Smart Cast is a Kotlin feature that automatically casts a variable to a specific type after checking its type or nullability, so you don't need to do explicit casting.

Why use Smart Cast?

- Reduces boilerplate code.
- Makes code more readable and safe.
- Avoids manual type casting like in Java (`(MyType)obj`).

How it works:

If you check a variable with an `is` check or a `null` check, the compiler will smartly remember the type or nullability and cast it for you inside that block.

When smart cast doesn't work:

If the variable is **mutable** (`var`) and **can be changed** from another thread, Kotlin won't apply smart cast to avoid risk.

24) What are Kotlin Collections?

Kotlin collections are used to **store and manage groups of related data items**. They help you work with multiple values as a **single unit**—like a list of names, or a map of IDs and values.

Created by Anand Gaur

Why use Collections?

- To store and manage **multiple values**
- To **iterate, filter, sort, or modify** data easily
- To organize data like:
    - A list of tasks (List)
    - A set of unique items (Set)
    - A key-value mapping (Map)

Types of Kotlin Collections:

Kotlin provides **two types** of collections:

1. Immutable Collections (Read-only)

- You **can't modify** them (add/remove/update items).
- Safe to use when data shouldn't change.
- Examples:
    - `listOf()`
    - `setOf()`
    - `mapOf()`

2. Mutable Collections (Read + Write)

- You **can modify** them (add/remove/update items).
- Use when data changes during runtime.
- Examples:
    - `mutableListOf()`
    - `mutableSetOf()`
    - `mutableMapOf()`

Created by Anand Gaur

25) What is the difference between `map` and `flatMap` in Kotlin?

➤ **`map`:**

- Used to **transform** each item in a collection.
- Returns a **new list** with the same size, but with transformed items.
- Useful when you want to apply a function to every element.

➤ **`flatMap`:**

- Used when **each item produces a list**, and you want to **flatten** all lists into a **single list**.
- Combines mapping + flattening in one step.

26) How do you create an empty list in Kotlin?

You can create an **empty list** using the following methods:

➤ **Immutable empty list:**

```
val emptyList = listOf<String>()
```

- It's a read-only list (immutable).
- You cannot add or remove items.

➤ **Mutable empty list:**

```
val emptyMutableList = mutableListOf<String>()
```

- You can add or remove items later using `.add()`, `.remove()`, etc.

Created by Anand Gaur

27) What is a Lambda Expression in Kotlin?

A lambda expression in Kotlin is an anonymous function—a block of code that doesn't have a name and can be passed as an argument, stored in a variable, or returned from a function.

Why Use Lambda?

- To write shorter, cleaner code
- To pass behavior (like actions) as parameters
- Commonly used in functional programming and collection operations

Syntax:

```
val lambdaName: (Type) -> ReturnType = { parameter -> body }
```

28) What is a Higher-Order Function in Kotlin?

A **higher-order function** is a function that either:

- **Takes another function as a parameter**,
- **Returns a function**,
- Or **does both**.

In simple words, a higher-order function works **with functions as values** — just like it works with integers, strings, or other objects.

Why use higher-order functions?

- To write **cleaner**, **reusable**, and **modular** code
- Helpful in **callbacks**, **functional programming**, and **event handling**

Created by Anand Gaur

Real Use Cases in Android:

- **filter, map, reduce** for list/collection operations.
- **setOnClickListener** takes lambda (function) to handle button click.
- **Dependency Injection** using lambdas.
- **Asynchronous calls** using Coroutines and callbacks.

29) What is an Extension Function in Kotlin?

In Kotlin, **extension functions** let you **add new functions to existing classes** without modifying their source code or using inheritance.

This means you can extend any class (even classes from libraries like Android SDK, Java APIs, etc.) with **custom behavior**.

Syntax:

```
fun ClassName.functionName(parameters): ReturnType {
    // function body
}
```

Benefits of Extension Functions

- You can extend third-party libraries without inheritance.
- Makes code cleaner and easier to read.
- Encourages reusable utility methods.
- Useful in writing helper functions in Android apps.

Note:

- Extension functions **do not actually modify the class**. Internally, Kotlin just uses static helper functions.
- They can't access **private** or **protected** members of the class.

Created By Anand Gaur

30) What is an Infix Function in Kotlin?

An **infix function** in Kotlin is a **special type of function** that lets you **call a function without using dots (.) or parentheses ( )**, making the code look more **natural and readable** — like a sentence.

Rules to Define an Infix Function

1. It must be a **member function** or **extension function**.
2. It must have **only one parameter**.
3. It must be marked with the `infix` keyword.

Benefits of Infix Functions

- Improves **code readability**
- Makes DSL (domain-specific language) creation easier
- Great for **utility** and **helper functions**
- Looks like **natural language** when used

31) What are Inline Functions in Kotlin?

An **inline function** in Kotlin is a function where the **compiler copies the function body directly to the places where it's called**, instead of calling the function in the traditional way. This can **improve performance**, especially when **using higher-order functions** (functions that take other functions as parameters).

Why Use Inline Functions?

Normally, passing a function (lambda) creates an **object** and adds some **runtime overhead**. Using `inline` tells the compiler to **avoid creating the function object** and instead **insert the lambda code directly** — making it **faster** and **more efficient**.

Created by Anand Gaur

When to Use Inline Functions

- When your function takes **lambda parameters**
- To avoid the **overhead of lambda object creation**
- To **improve performance**, especially in **tight loops** or **small utility functions**

32) What is `noinline` in Kotlin?

In Kotlin, the `noinline` keyword is used inside an `inline` function to prevent specific lambda parameters from being inlined.

Why Use `noinline`?

When you mark a function as `inline`, all lambdas passed to it are normally inlined (code is copied). But sometimes, you don't want to inline certain lambdas — for example, if you need to:

- Store the lambda in a variable
- Pass it to another function
- Return it from the function

33) What is the use of the `reified` modifier in Kotlin?

In Kotlin, the `reified` modifier is used with type parameters inside inline functions to make the actual type available at runtime. Normally, type information is erased at runtime (called type erasure), but `reified` helps retain it for certain cases like reflection or type checks.

Key Benefits

Type Checks: You can use `is`, `as`, and `::class` with generic types.

Reflection: Get class metadata like `T::class.java`, `T::class.simpleName`, etc.

Clean Code: Avoids boilerplate and workarounds for passing `Class<T>` manually.

Created by Anand Gaur

34) What is a Sealed Class in Kotlin?

A **sealed class** in Kotlin is a **special type of class** used to represent **restricted class hierarchies**. It allows you to define a set of subclasses **in the same file**, and the compiler knows all the possible types a sealed class can have.

This is helpful when you're working with `when` expressions and you want the compiler to ensure that all possible cases are handled.

Key Points

- Sealed classes **restrict inheritance** — all subclasses must be declared in the same file.
- Useful for representing **finite state or result types**, such as Success/Failure, Loading/Error/Data, etc.
- A sealed class **cannot be instantiated directly**.
- Helps make your code more **type-safe and readable**.

35) What is a `suspend` Function in Kotlin Coroutines?

A **`suspend` function** is a special type of function in Kotlin that can be **paused and resumed** at a later time **without blocking the main thread**. It is used **within Kotlin Coroutines** to perform long-running or asynchronous tasks like network calls, database operations, or file I/O.

Key Points:

- A `suspend` function **can only be called** from another `suspend` function or from a coroutine.
- It allows the function to **suspend its execution** and **resume later**, making asynchronous code easier and more readable.
- It helps avoid **callback hell** and keeps the UI **smooth and responsive**.

Created by Anand Gaur

Where is it used?

- **Network requests** (with Retrofit + Coroutines)
- **Database operations** (Room DAO methods with suspend)
- **File reading/writing**
- **Background processing without blocking the UI**

Benefits of suspend Functions

- **Non-blocking:** Keeps the main/UI thread free.
- **Readable code:** Looks like sequential code but works asynchronously.
- Efficient: Uses lightweight coroutines instead of heavy threads.

36) What is the suspend Modifier in Kotlin?

The suspend modifier in Kotlin is used to mark a function **as suspendable**, meaning it can be **paused and resumed later** without blocking the thread. It's a key concept in **Kotlin Coroutines** for writing **asynchronous, non-blocking code** in a clean and sequential manner.

Purpose of suspend

- It lets the compiler know that this function **might take time** (e.g., a network call, database read, or file operation).
- It allows the function to **suspend its execution** and let other coroutines run, instead of blocking the thread.
- suspend functions must be called from **within a coroutine or another suspend function**.

Created By Anand Gaur

How it Works Internally

- When a `suspend` function is called, it **doesn't block the thread**.
- If it hits a long-running operation (like `delay()`), it **suspends** itself and lets other coroutines work.
- Once the operation completes, the coroutine **resumes** where it left off.

Benefits of Using `suspend`

- **Clean asynchronous code** without callbacks
- **Does not block** the main thread
- Works great with **structured concurrency**
- Can be **easily composed** with other suspend functions

37) What is the difference between `List`, `MutableList`, `ArrayList` in Kotlin?

`List` (Read-Only)

- It is **immutable**, meaning you **can't add, update, or remove** elements.
- It only allows **read operations** (like `get(index)` or `forEach`).

`MutableList` (Read and Write)

- It is a **modifiable list**.
- You can **add, remove, and update** items.

`ArrayList` (A Type of MutableList)

- `ArrayList` is a **specific implementation** of `MutableList`.
- Internally backed by a **dynamic array**, similar to Java's `ArrayList`.

Created by Anand Gaur

38) What are Scope Functions in Kotlin?

Scope functions in Kotlin are special functions that allow you to execute code within the context of an object. They help improve code readability and reduce boilerplate.

These are the **main scope functions**:

- `let`
- `run`
- `with`
- `apply`
- `also`

Each function:

- Uses a different **object reference**: `it` or `this`
- Returns a different result: either the **object itself** or the **lambda result**

1. `let`

- **Uses:** It uses `it` to refer to the object
- Returns: **lambda result**
- **Use case:** null checks, chaining, transformations

2. `run`

- **Uses:** It uses `this` to refer to the object.
- Returns: **lambda result**
- **Use case:** object configuration and result return

Created by Anand Gaur

3. `with`

- **Uses:** It uses `this` to refer to the object.
- Returns: **lambda result**
- **Use case:** call multiple methods on the same object

4. `apply`

- **Uses:** It uses `this` to refer to the object.
- Returns: **the object itself**
- **Use case:** object configuration

5. `also`

- **Uses:** It uses `it` to refer to the object.
- Returns: **the object itself**
- **Use case:** for logging, debugging, or chaining

39) What is the difference between `apply` and `also` scope functions in Kotlin?

`apply`

- **Used for:** Configuring or initializing an object.
- **Object Reference:** `this` (you can access members directly).
- **Return value:** Returns the object itself.
- **Best for:** Setting up properties during object creation.

Created by Anand Gaur

`also`

- **Used for:** Performing side effects (like logging, debugging) without modifying the object itself.
- **Object Reference:** `it`.
- **Return value:** Returns the object itself.
- **Best for:** Adding actions like logging or validation without changing object setup.

40) How do you handle exceptions in Kotlin Coroutines?

In Kotlin Coroutines, exceptions can be handled using different techniques depending on the coroutine builder (`launch`, `async`, etc.).

1. Using `try-catch` block

For `suspend` functions and `async`, we can wrap the code inside a `try-catch` block.

```
suspend fun fetchData() {
    try {
        val data = getNetworkData()
        println(data)
    } catch (e: Exception) {
        println("Error: ${e.message}")
    }
}
```

For `async`:

```
val deferred = CoroutineScope(Dispatchers.IO).async {
    // Some risky operation
    getNetworkData()
}
try {
    val result = deferred.await()
} catch (e: Exception) {
    println("Caught: ${e.message}")
}
```

2. Using `CoroutineExceptionHandler`

When using `launch`, uncaught exceptions can be handled with `CoroutineExceptionHandler`.

```
val handler = CoroutineExceptionHandler { _, exception ->
    println("Caught exception: ${exception.message}")
}
CoroutineScope(Dispatchers.IO).launch(handler) {
    throw RuntimeException("Something went wrong!")
}
```

3. SupervisorJob for Independent Child Coroutines

To prevent one coroutine failure from cancelling others:

```
val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default)
scope.launch {
    // Fails
    throw RuntimeException("Child failed")
}
scope.launch {
    // Continues to run
    println("I'm still running!")
}
```

41) What is Kotlin Coroutines?

Kotlin Coroutines is a concurrency design pattern used in Android and Kotlin applications for writing asynchronous and non-blocking code in a simple, readable, and structured way.

Why use Coroutines?

- Traditional approaches like Threads or Callbacks can make code **complex and hard to manage**.
- Coroutines let you write asynchronous code in a **sequential and easy-to-read style**.

Key Features of Coroutines:

- **Lightweight:** Multiple coroutines can run on a few threads.
- **Non-blocking:** They suspend execution without blocking the thread.
- **Structured Concurrency:** Child coroutines are automatically cancelled when the parent is cancelled.
- **Built-in support** in Kotlin standard library with `kotlinx.coroutines`.

Common Coroutine Builders:

- `launch`: Fire and forget, doesn't return a result.
- `async`: Returns a result via `Deferred`.
- `runBlocking`: Used to bridge blocking and non-blocking code (mostly in tests or main).

Use Case in Android:

- Making **network calls** using Retrofit
- Performing **database operations**
- Running **long tasks** without freezing UI

42) What is the difference between `launch` and `async` in coroutines?

- `launch`: Used when you don't expect a result. It returns a `Job`.
- `async`: Used when you want to return a result. It returns a `Deferred<T>` and is used with `await()`.

Created by Anand Gaur

43) What is `runBlocking`? When is it used?

`runBlocking` is a coroutine builder that blocks the current thread until all its child coroutines complete. It is typically used in unit tests or the `main()` function.

44) What is structured concurrency in Kotlin?

Structured concurrency ensures that new coroutines are launched in a specific scope, and when the scope is canceled, all its child coroutines are automatically cancelled too. This prevents memory leaks and dangling coroutines.

45) What is Scope in Kotlin Coroutines?

In Kotlin Coroutines, a **Scope** defines the **lifecycle** of a coroutine — it decides **how long** a coroutine should run and **when it should get cancelled**.

Why is Scope important?

Coroutines run asynchronously. Without a scope, you'd lose control over:

- When they start
- When they should stop
- Preventing memory leaks

Types of Coroutine Scopes:

1. GlobalScope

- Used to launch coroutines that live throughout the entire lifetime of the application.
- Avoid it in most cases — leads to **memory leaks** and **uncancelled coroutines**.

2. CoroutineScope

- A general-purpose scope that you can control manually.
- Used when creating custom scopes (e.g., repositories, services).

Created by Anand Gaur

3. lifecycleScope (Android-specific)

- Tied to the **Activity** or **Fragment** lifecycle.
- Automatically cancels coroutines when the lifecycle ends.

4. viewModelScope (Android-specific)

- Tied to the **ViewModel** lifecycle.
- Automatically cancels all coroutines when ViewModel is cleared.

46) What is the difference between `CoroutineScope` and `GlobalScope`?

- `CoroutineScope`: Tied to the lifecycle of a component (e.g., Activity, ViewModel). Automatically cancels coroutines when the lifecycle ends.
- `GlobalScope`: Not tied to any lifecycle, Coroutines run forever unless you cancel them manually.

47) What is the use of `withContext()` in Coroutine?

`withContext()` is used to switch the coroutine context (e.g., from `IO` to `Main`) within a coroutine without creating a new coroutine.

48) What are Dispatchers in Kotlin Coroutines?

In Kotlin Coroutines, **Dispatchers** define **which thread** or **thread pool** the coroutine should run on. They help control the **execution context** of coroutine — whether it runs on the **main/UI thread**, a **background thread**, or a **custom thread pool**.

Common Dispatchers in Kotlin:

1. `Dispatchers.Main`

- Runs on the **main/UI thread**.
- Used for tasks like updating UI elements.

Created by Anand Gaur

- Only available in Android or JVM with UI.

2. `Dispatchers.IO`

- Optimized for **I/O operations** like:
    - Reading/writing files
    - Making network calls
    - Accessing database

3. `Dispatchers.Default`

- Optimized for **CPU-intensive work** like:
    - Sorting
    - Complex calculations
    - Large data processing

4. `Dispatchers.Unconfined`

- Starts the coroutine in the **current thread**, but may resume it on a different thread.
- **Advanced use case**, not recommended for UI or predictable threading.

5. Custom Dispatchers

You can create a **custom thread pool** using `Executors` and convert it to a coroutine dispatcher:

```
val myDispatcher =
Executors.newSingleThreadExecutor().asCoroutineDispatcher()
CoroutineScope(myDispatcher).launch {
    // custom background task
}
```

Created by Anand Gaur

49) What is `supervisorScope` in coroutines?

In `supervisorScope`, the failure of one child does not cancel the others. It helps in running multiple independent tasks where failure of one should not affect others.

50) How to cancel a coroutine?

You can call `.cancel()` on the `Job` returned by `launch` or `async`.

51) How do coroutines differ from threads?

- Coroutines are **lightweight**, and thousands of coroutines can run on a few threads.
- Threads are **heavyweight**, and creating too many can affect performance.
- Coroutines are managed by the **Kotlin runtime**, not the OS.

52) What is a Flow in Kotlin?

In Kotlin, a **Flow** is an asynchronous stream of data that **emits multiple values sequentially** — one after another — **over time**.

It's a part of **Kotlin Coroutines** and is used for handling **reactive, asynchronous data streams**, like data coming from a network or database.

Why use Flow?

- To **emit multiple values** over time (unlike `suspend` functions, which return only one value).
- It's **cold** by default – meaning the code inside flow { } doesn't run until it is collected.
- It supports powerful operators like `map`, `filter`, `collect`, `combine`, etc.
- Supports **cancellation**, **backpressure**, and **exception handling**.

Created By Anand Gaur

53) What are cold and hot flows?

In Kotlin **Flow**, the way data is produced and consumed can be categorized into **Cold Flow** and **Hot Flow** based on when the data starts emitting and how many consumers can receive it.

Cold Flow

- **Definition**: A **Cold Flow** starts emitting values **only when it's collected**.
- Every time a new collector subscribes, the flow restarts from the beginning.
- Think of it like a **YouTube video**: each viewer watches from the beginning.

Hot Flow

- **Definition**: A **Hot Flow** emits values **regardless of collectors**.
- It starts emitting immediately, and **collectors receive values only from the point they start collecting**.
- Think of it like a **Live TV broadcast**: you only see what's airing **now**.

Types of Hot Flow:

- `StateFlow`: Emits the latest state, useful for UI state.
- `SharedFlow`: Used for one-time events (like navigation).

54) What is `StateFlow` vs `LiveData`?

- `StateFlow` is Kotlin's Flow-based observable state holder.
- Unlike LiveData, `StateFlow` is fully Kotlin-based, works in background threads, and has no lifecycle awareness by default.

55) What are `Job`, `Deferred`, and `CoroutineScope`?

- `Job`: Represents a cancellable unit of work.
- `Deferred<T>`: A job that returns a result with `await()`.

Created by Anand Gaur

- `CoroutineScope`: A scope that controls coroutine lifecycle.

56) What is `supervisorScope` in Kotlin Coroutines?

In Kotlin Coroutines, `supervisorScope` is a special coroutine scope that **prevents failure of one child coroutine from cancelling other child coroutines.**

It is useful when you want to isolate failures among multiple coroutines.

Why use `supervisorScope`?

By default, in a regular coroutine scope (like `coroutineScope`), **if one child fails, all siblings are cancelled.**
But in `supervisorScope`, **other coroutines continue to run** even if one fails.

57) Explain the concept of functional programming in Kotlin?

Functional Programming (FP) is a style of programming where you build your programs using **pure functions**, **immutability**, and **function composition**, instead of changing state or using objects that maintain internal state.

Kotlin supports both **object-oriented** and **functional** programming features.

Key Concepts of Functional Programming in Kotlin:

1. Pure Functions

- A pure function returns the **same output for the same input** and has **no side effects** (like modifying a global variable or reading a file).

2. Immutability

- Data is **not changed**, instead, new copies are created when changes are needed.

3. Higher-Order Functions

- You can **pass functions as parameters**, return them, or store them in variables.

Created By Anand Gaur

4. Lambda Expressions

- Lambdas are anonymous functions you can write inline.

5. Function Composition

- You can combine small functions to build complex logic.

6. Collections & Functional Methods

- Kotlin provides **functional operations** like `map`, `filter`, `reduce`, etc., on collections.

Benefits of Functional Programming in Kotlin:

- Easier to **test and debug**
- More **readable and concise**
- Helps in writing **thread-safe** code
- Promotes **reusability** and **modularity**

51) Explain the Concept of Delegates in Kotlin

In Kotlin, **delegates** allow you to hand over (or "delegate") the responsibility of property access and management to another object. This helps **reuse logic**, avoid boilerplate, and **write cleaner, more maintainable code**.

A delegate is an object that handles the **get() and set()** logic for a property.

Why Use Delegation?

- Reuse common logic (e.g., lazy initialization, observable changes)
- Avoid repetitive code (e.g., SharedPreferences access)
- Separation of concerns (encapsulate property behavior)

Created By Anand Gaur

52) What is the purpose of the `by` keyword in Kotlin?

In Kotlin, the `by` keyword is **used for delegation** — a design pattern that allows an object to **delegate behavior or property access** to another object.

53) Why Use Sealed Classes with `when`?

- The `when` expression becomes smart with sealed classes.
- Since the compiler knows all possible subclasses, you don't need an `else` block in `when`.

54) What is a Primary Constructor in Kotlin?

In Kotlin, a **primary constructor** is a concise way to **declare and initialize class properties** directly in the class header.

It simplifies boilerplate code by allowing you to define constructor parameters and property declarations in one place.

55) What's a `const` in Kotlin? How does it differ from a `val`?

In Kotlin, both `const` and `val` are used to **declare read-only variables**, but they are **very different in behavior, usage, and compilation time**.

`val` (Runtime Constant)

- `val` is a **read-only variable**.
- Its value is assigned **at runtime**, when the object or class is initialized.
- It **can hold dynamic values**, like values from function calls, constructors, etc.

Created by Anand Gaur

`const` (Compile-Time Constant)

- `const` is used to **declare constants at compile time**.
- It **must be a `String` or primitive type** (`Int`, `Boolean`, `Double`, etc.).
- It **must be assigned a value known at compile time** (no function calls or dynamic values).
- It can **only be declared at the top-level** or inside an `object`.

56) What is the Difference Between `init` Block and `constructor` in Kotlin?

In Kotlin, the **primary constructor** and the **`init` block** often work together during **object initialization**, but they **serve different roles**. Let's break it down clearly:

What is a `constructor` in Kotlin?

- A **constructor** is used to **pass parameters** when creating an object.
- Kotlin has:
  - **Primary constructor** (declared in the class header)
  - **Secondary constructors** (using the `constructor` keyword inside the class body)

What is an `init` block in Kotlin?

- The `init` block is a **special initializer block** that runs **immediately after the primary constructor**.
- It's used for **additional logic**, like validation, logging, or initializing values based on constructor parameters.

Created by Anand Gaur

57) Explain the Concept of Generics in Kotlin?

Generics in Kotlin allow you to **write reusable, type-safe code**. They enable classes, interfaces, and functions to operate on **different types** while still being **type-safe at compile time**.

Why Use Generics?

Imagine you want to create a class or function that works with **any data type** (Int, String, custom class, etc.). Instead of rewriting the logic for each type, **generics let you define it once**.

58) Explain the Concept of `typealias` in Kotlin

`typealias` is a **keyword** in Kotlin used to **give an alternative (shorter or more readable) name** to an existing type — especially for complex or nested types.

It's **not creating a new type**, but just **a shorthand** for an existing one to improve **code readability**.

Limitations of `typealias`

- You **cannot use `typealias` to create a truly new type**.
- It **won't add type-safety**.

59) What is the Difference Between Invariance, Covariance, and Contravariance in Kotlin Generics?

In Kotlin (just like in Java), **generics** define how one generic type relates to another — particularly whether a type like `List<String>` is a subtype of `List<Any>`.

But in Kotlin, **type variance** gives you more control with `in`, `out`, and the default invariant behavior.

Let's break it down deeply but simply:

Created by Anand Gaur

1. Invariance (No Variance - Default Behavior)

By default, **generic types are invariant** in Kotlin.
This means:

> `List<String>` is **NOT** a subtype of `List<Any>`, even though `String` is a subtype of `Any`.

Use-case:

Use **invariant** generics when both reading and writing operations occur and strict type matching is required.

2. Covariance – `out` Keyword (Producer)

Use `out` when you **only read** from a generic type and **don't modify** it.

3. Contravariance – `in` Keyword (Consumer)

Use `in` when you **only consume** values into a generic type (i.e., you're only inserting values).

60) Explain the Concept of Inline Classes in Kotlin

An **inline class** is a **special type of class** in Kotlin that **wraps a single value** but **does not incur additional memory overhead** at runtime.

It's used to **avoid object allocation** by **replacing the class with its underlying value** at compile-time, making it **lightweight and efficient**.

Why Use Inline Classes?

- Avoid unnecessary heap allocations
- Add type-safety to primitives (like wrapping an `Int` as `UserId`)
- Prevent mixing up similar-looking types (like `UserId` vs `ProductId`)
- Boost performance for small, frequently used data

Created by Anand Gaur

61) Explain the Concept of Property Delegation in Kotlin

In Kotlin, **property delegation** allows you to **delegate the logic of getting and setting a property** to another object.

This means that instead of writing boilerplate code for `get()` and `set()` operations, you can **outsource this logic** using the `by` keyword.

Why Use Property Delegation?

- Reuse property logic across multiple classes
- Write cleaner, DRY (Don't Repeat Yourself) code
- Add advanced behavior like **lazy loading**, **observable changes**, **automatic storage**, etc.

62) What is the purpose of the `const` modifier in Kotlin?

In Kotlin, the `const` modifier is used to **declare compile-time constants** — values that are **known and fixed at compile time**, not at runtime.

It is mainly used for defining **static constant values** that don't change and can be inlined wherever used.

Syntax:

```
const val API_BASE_URL = "https://api.example.com"
```

63) What is the difference between extension functions and member functions in Kotlin?

Extension Function

An **extension function** is written **outside the class**, but it **adds functionality** to an existing class without modifying its code.

Member Function

A **member function** is a function that is written **inside a class**. It is a part of the class and can access all properties, including private ones.

Created by Anand Gaur

64) Explain the concept of property access syntax in Kotlin

In Kotlin, properties (like variables in a class) are accessed using a **simple, clean syntax** that hides the underlying getter and setter methods.

This feature is known as **Property Access Syntax**.

65) Explain the concept of the `this` expression in Kotlin

In Kotlin, the `this` keyword refers to the **current object** — the instance of the class where the code is running.

You use `this` when:

- You want to refer to the current class's properties or methods.
- You have a name conflict between a parameter and a property.
- You want to refer to the outer class in a nested or inner class.

66) Explain the Concept of Default Arguments in Kotlin

In Kotlin, **default arguments** allow you to assign default values to function parameters.

This means **you can call a function without passing all arguments**, and the missing ones will automatically take their default values.

Why Use Default Arguments?

- Reduces method overloading (no need to create multiple versions of the same function).
- Makes function calls simpler and more readable.
- Useful when some parameters are optional.

Created by Anand Gaur

67) Explain the Concept of Function References in Kotlin

In Kotlin, **function references** allow you to **pass functions as values** — just like variables.
This is part of **Kotlin's functional programming** capabilities.

You can refer to a function **without calling it**, using the `::` operator.

68) What is the Purpose of the downTo Keyword in Kotlin?

In Kotlin, `downTo` is used to create a **range that counts backward** (in descending order).

It is the **opposite of `..`**, which creates an ascending range.

69) What is a Closure in Kotlin?

A **closure** in Kotlin is a function (usually a lambda or anonymous function) that **captures variables from its surrounding scope**, even **after the scope has ended**.

It allows the function to **remember and access variables** that were in scope **when the function was created**.

Why Closures are Useful in Kotlin:

- Used heavily in **lambdas**, **callbacks**, **coroutines**, and **higher-order functions**.
- Enables creation of **stateful functions** without global variables.
- Makes functional programming patterns possible.

70) Explain the Concept of the `until` Keyword in Kotlin?

In Kotlin, the `until` keyword is used to create a **range that excludes the end value** — it's a **half-open range**.

It is commonly used in loops when you want to **iterate from a start value up to, but not including, an end value**.

Created by Anand Gaur

71) Explain the Concept of Extension Properties in Kotlin?

Extension properties in Kotlin allow you to **add new properties** to existing classes **without modifying their source code**.

Just like **extension functions**, extension properties help make classes more powerful, especially when working with third-party or built-in types.

72) Explain the Concept of Sealed Interfaces in Kotlin?

In Kotlin, **sealed interfaces** are like **sealed classes** — they restrict which types can implement them. This allows **exhaustive `when` statements**, and better control over a type hierarchy.

A **sealed interface** can only be implemented by classes or objects that are **defined in the same file** where the interface is declared.

This makes the type hierarchy **closed** (sealed), so the compiler knows **all possible implementations** at compile time.

73) Explain the Concept of Function Composition in Kotlin?

Function Composition means **combining two or more functions** to form a new function, where the **output of one function becomes the input of another**.

This concept is inspired by **mathematics and functional programming** and helps write **clean, reusable, and modular code**.

74) What is the Difference Between `first()` and `firstOrNull()` in Kotlin?

Kotlin provides two useful extension functions to **retrieve the first element** from a collection or sequence:

- `first()`
- `firstOrNull()`

Although they seem similar, **the main difference lies in how they behave when the collection is empty or no matching item is found**.

Created by Anand Gaur

`first()` – Throws Exception if No Match

This function returns the **first element** of the collection (or the first element that matches the predicate), but:

- If the collection is **empty**, or
- No element matches the condition (when used with predicate),

👉 it **throws a `NoSuchElementException`**.

`firstOrNull()` – Returns `null` if No Match

This function also returns the **first element**, but:

- If the list is **empty**, or
- No element matches the condition,

👉 it returns **`null`** instead of throwing an exception.

75) Explain the Concept of `crossinline` in Kotlin

In Kotlin, the `crossinline` modifier is used with **lambda parameters of inline functions** to **prevent non-local returns** from the lambda.

First, What is a Non-local Return?

Normally, in an `inline` function, if a lambda is **not marked with `crossinline`**, the lambda can use a `return` statement to exit **not just the lambda itself**, but also the **calling function**. This is called a **non-local return**.

What Does `crossinline` Do?

When you **add `crossinline`** to a lambda parameter, it tells the compiler:

❗ "You cannot use a `return` statement inside this lambda to exit the calling function."

It restricts the lambda to only **local returns**.

Created by Anand Gaur

When to Use `crossinline`?

Use `crossinline` when:

- You are **passing a lambda** to another scope (like a thread, coroutine, or another function object).
- You want to **prevent non-local returns**, because they would cause compile errors or break the program flow.

Example:

- `Runnable { }`
- `CoroutineScope.launch { }`
- `View.setOnClickListener { }`

In these contexts, Kotlin **cannot handle non-local returns** because the lambda might be called later or in a different thread.

76) What is the use of `requireNotNull` function in Kotlin?

In Kotlin, the `requireNotNull()` function is used to **check that a value is not `null`** at runtime, and **throw an `IllegalArgumentException`** if it is.

It is commonly used for **validating input arguments** or asserting that a variable **must not be null**, especially when working with nullable types.

When to Use `requireNotNull`?

- To ensure required **arguments are not null**.
- To **convert nullable types to non-nullable** after a safe check.
- During **early validation** of inputs.

Created by Anand Gaur

77) Explain the concept of Top-Level Functions in Kotlin?

In Kotlin, **top-level functions** are functions that are **declared outside any class, object, or interface**. Unlike Java (where functions must be inside a class), Kotlin allows you to write functions **directly in a `.kt` file**, making the code more concise and modular.

Key Points:

1. No Class Needed:
   You don't need to wrap top-level functions in a class. Kotlin compiles these into a class named after the filename (e.g., `UtilsKt.class` for `Utils.kt`).

2. Can be Used Like Utility Functions:
   Ideal for utility methods like `log()`, `validateInput()`, `calculateSum()`, etc.

3. Supports Visibility Modifiers:
   You can make top-level functions `public`, `private`, or `internal`.

4. Can be Imported with `import`:
   You can import top-level functions into other files:

78) What is the purpose of the `@JvmName` annotation in Kotlin?

The `@JvmName` annotation in Kotlin is used to **change the name of the generated Java method or class** when Kotlin code is compiled to JVM bytecode.

By default, Kotlin compiles top-level functions and properties into static methods inside a class named after the file (e.g., `UtilsKt.class` if your file is `Utils.kt`). These method names are based on the Kotlin function names, but sometimes you may want to **customize the name for better Java interoperability**.

Created by Anand Gaur

79) What is the difference between infix and regular functions in Kotlin?

In Kotlin, **infix functions** allow you to call a function **without using dots (`.`) and parentheses (`()`)**, making the syntax cleaner and more readable. They are just syntactic sugar over regular functions but have specific rules.

Regular Function:

A regular function is called using the usual syntax:

```
fun multiply(a: Int, b: Int): Int {
    return a * b
}
val result = multiply(4, 5)  // Output: 20
```

Infix Function:

An **infix function** allows the function to be called **between the receiver and the parameter**, like a natural language sentence.

```
infix fun Int.times(x: Int): Int {
    return this * x
}
val result = 4 times 5  // Output: 20
```

80) What is the difference between sealed classes and abstract classes in Kotlin?

Sealed classes and abstract classes in Kotlin are both used for inheritance, but they serve different purposes and offer different capabilities, especially when it comes to controlling subclassing and enabling smart type checking.

1. Sealed Class:

A **sealed class** is used when you want to **restrict the types** that can inherit from it. All subclasses **must be defined in the same file** as the sealed class itself.

Key Features:

- Helps **model restricted hierarchies** (known and limited types).
- Useful in `when` expressions — **exhaustiveness is guaranteed** (no `else` needed).
- Subclasses must be defined **in the same file**.
- Can be **abstract**, **data**, **object**, or **regular classes**.
- Cannot be instantiated directly.

2. Abstract Class:

An **abstract class** is a class that **cannot be instantiated**, and can have **abstract members** (functions or properties that must be implemented in subclasses).

Key Features:

- Designed for **inheritance and partial implementation**.
- Can have **abstract and non-abstract members**.
- Subclasses can be in **any file**.
- Does **not enforce** an exhaustive `when` check.
- More flexible in terms of architecture (open to external subclassing).

81) How do you handle concurrency in Kotlin using `synchronized` blocks?

In Kotlin, you can handle concurrency using `synchronized` blocks just like in Java. This is useful when you want to **ensure that only one thread accesses a block of code at a time**, preventing **race conditions** and ensuring **thread safety**.

Created by Anand Gaur

Why Use `synchronized`?

When multiple threads try to read or write shared data at the same time, it can lead to unexpected behavior or crashes. `synchronized` blocks **serialize access** to a critical section of code, making it safe to use shared resources.

82) What are `Pair` and `Triple` in Kotlin?

In Kotlin, `Pair` and `Triple` are simple data structures used to **store and return two or three values together** from a function or expression **without creating a custom class**.

They are part of the Kotlin standard library and are **immutable** (read-only).

Limitations:

- `Pair` and `Triple` are great for **temporary, quick data holders**.
- But for more complex or meaningful data, it's better to use **data classes** with named properties for better readability.

83) What are Labels in Kotlin?

In Kotlin, **labels** are identifiers followed by the `@` symbol (e.g., `myLabel@`). Labels are primarily used to **control the flow of loops and lambdas**, especially when you want to **break, continue**, or **return from a specific block or loop**.

They help you improve **readability and precision** when working with **nested loops** or **lambdas with return statements**.

84) What are the benefits of using a Sealed Class over Enum in Kotlin?

In Kotlin, both **Sealed Classes** and **Enums** are used to represent a fixed set of types or values. However, **Sealed Classes offer more flexibility and power** than enums, especially when you need to associate complex data with each type.

Created by Anand Gaur

Key Benefits of Sealed Class over Enum:

1. **Can Hold Different Data Types**

- Sealed classes allow each subclass (object or data class) to hold different types and amounts of data.

- Enums can't do this — they can only hold fixed values and limited data via constructor params.

2. **Supports Exhaustive `when` Checks Without Else**

- With **sealed classes**, the compiler **knows all subclasses** at compile-time and forces you to handle all cases in a `when` expression.

3. **Can Have Hierarchical Structure**

Sealed classes support inheritance, allowing **more structured and modular designs**.

4. **Allows Object + Class Mix**

You can mix `object` (singleton) and `data class` inside a sealed class. Enums don't allow such flexibility.

Enum Limitations:

- Can only hold primitive types or fixed constructor arguments.

- Can't easily hold different types per enum constant.

- Not as flexible for modeling **complex states** or **event flows**.

85) How are Coroutines different from Threads in Kotlin?

Coroutines and Threads are both used for **concurrent or parallel execution**, but they work **very differently** in terms of performance, resource usage, and how they are managed.

Created by Anand Gaur

Key Differences Between Coroutines and Threads

1. Lightweight vs Heavyweight

- **Thread:**
  - Heavyweight. Creating too many threads can lead to memory overhead and performance issues.
  - Each thread takes up a significant amount of memory (~1 MB stack space per thread).
- **Coroutine:**
  - Lightweight. You can create thousands of coroutines without performance problems.
  - They run within a few threads and are managed by the Kotlin coroutine dispatcher.

2. Blocking vs Suspending

- **Thread**:
  - Runs code **synchronously**.
  - Uses **blocking operations** — if one thread sleeps or waits, it holds system resources.
- **Coroutine**:
  - Uses **suspending functions** (like `delay()`), which **do not block** the thread.
  - The thread remains free for other coroutines/tasks while one coroutine is waiting.

3. Scalability

- **Thread**:
  - Not scalable for thousands of tasks.

Created by Anand Gaur

    - Each thread adds overhead.

- **Coroutine**:
    - Highly scalable. Can handle **many concurrent tasks efficiently**.

4. **Management & Structure**

- **Thread**:
    - Managed by the **OS**.
    - Needs manual handling: `start()`, `join()`, and error handling.
- **Coroutine**:
    - Managed by the **Kotlin runtime & Dispatchers**.
    - Supports structured concurrency (like using `viewModelScope`, `lifecycleScope`).
    - Easier cancellation, timeout, and error handling.

5. **Error Handling**

- **Thread**:
    - Exceptions may crash the app unless handled carefully.
- **Coroutine**:
    - Safer error handling using `try-catch`, `CoroutineExceptionHandler`, and structured scopes.

86) What is the open keyword in Kotlin used for?

In Kotlin, **classes and their members (functions, properties) are `final` by default**, meaning they **cannot be inherited or overridden**. The `open` keyword is used to explicitly allow a class or member to be **subclassed or overridden**.

Created by Anand Gaur

Purpose of `open`:

- **Allow inheritance** of a class.
- **Allow overriding** of a function or property in a subclass.

87) What is the difference between Array and List in Kotlin?

Array:

- Fixed size, cannot grow or shrink
- Mutable elements (can change values)
- Better performance for indexed access
- Backed by Java arrays

List:

- Dynamic size (for MutableList)
- Immutable (List) or Mutable (MutableList) versions
- More functional programming features
- Better API with functional methods

88) What is the difference between `is` and `as` operators in Kotlin?

`is` operator (Type Check):

- Checks if an object is of a specific type
- Returns boolean (true/false)
- Safe operation, never throws exception

`as` operator (Type Cast):

- Explicitly casts an object to a specific type
- Throws ClassCastException if cast fails
- Use `as?` for safe casting (returns null if fails)

Created by Anand Gaur

89) What is the difference between `internal` and `protected` visibility modifiers?

`internal`:

- Visible within the same module
- Module = set of Kotlin files compiled together
- Can be accessed from any class in the same module

`protected`:

- Visible within the same class and its subclasses
- Cannot be used for top-level declarations
- Inheritance-based visibility

90) What is the difference between `fold` and `reduce` in Kotlin?

`fold`:

- Takes an initial value
- Can work with empty collections
- Return type can be different from collection type

`reduce`:

- No initial value
- Cannot work with empty collections (throws exception)
- Return type must be same as collection element type

91) What is the difference between `suspend` and `inline` functions?

`suspend` functions:

- Can be paused and resumed
- Used for asynchronous operations
- Must be called from coroutines or other suspend functions

Created by Anand Gaur

`inline` functions:

- Function body is copied to call site at compile time
- Eliminates function call overhead
- Useful for higher-order functions

92) What is the difference between `==` and `equals()` in Kotlin?

`==` operator:

- Calls `equals()` method internally
- Null-safe comparison
- More convenient syntax

`equals()` method:

- Direct method call
- Can be overridden in classes
- Not null-safe by default

93) What is the difference between `filter` and `filterNot` in Kotlin?

`filter`:

- Keeps elements that match the predicate
- Returns elements where predicate is true

`filterNot`:

- Removes elements that match the predicate
- Returns elements where predicate is false

94) What is the difference between `takeIf` and `takeUnless`?

`takeIf`:

- Returns the object if predicate is true, null otherwise

Created by Anand Gaur

- Useful for conditional assignments

`takeUnless`:

- Returns the object if predicate is false, null otherwise
- Opposite of takeIf

95) What is the difference between `listOf` and `arrayListOf`?

`listOf`:

- Creates an immutable list
- Returns List<T> interface
- Cannot add/remove elements

`arrayListOf`:

- Creates a mutable ArrayList
- Returns ArrayList<T> (specific implementation)
- Can add/remove elements

96) What is the difference between `Any` and `Any?` in Kotlin?

`Any`:

- Non-nullable root of Kotlin type hierarchy
- Equivalent to `Object` in Java
- Cannot hold null values

`Any?`:

- Nullable version of Any
- Can hold any value including null
- True top type in Kotlin

Created by Anand Gaur

97) What is the difference between `Unit` and `Nothing` in Kotlin?

`Unit`:

- Represents "no meaningful value"
- Equivalent to `void` in Java
- Has a single instance
- Default return type for functions

`Nothing`:

- Represents "never returns normally"
- Used for functions that always throw exceptions
- Has no instances
- Bottom type in Kotlin type hierarchy

98) What is the difference between `return` and `return@label`?

`return`:

- Returns from the nearest enclosing function
- Non-local return in lambdas (if inline)

`return@label`:

- Returns from specific labeled block
- Local return from lambda

99) What is the difference between `break` and `continue` with labels?

`break@label`:

- Breaks out of the labeled loop
- Useful in nested loops

Created by Anand Gaur

`continue@label`:

- Continues to next iteration of labeled loop
- Skips remaining code in current iteration

100) What is the difference between `if` and `when` expressions?

`if` expression:

- Binary condition checking
- Returns a value based on condition
- Can have multiple else-if branches

`when` expression:

- Multi-way branching
- More readable for multiple conditions
- Supports pattern matching

101) What is the difference between `vararg` and `Array` parameters?

`vararg`:

- Variable number of arguments
- Can be called with multiple separate arguments
- Automatically creates an array

`Array` parameter:

- Fixed array parameter
- Must pass an array explicitly
- More explicit about expecting an array

Created by Anand Gaur

102) What is the difference between `Elvis` operator and `Safe call` operator?

Safe call operator (`?.`):

- Safely accesses properties/methods on nullable objects
- Returns null if object is null

Elvis operator (`?:`):

- Provides default value when expression is null
- Returns left side if not null, right side if null

103) What are the different ways to launch coroutines in Kotlin?

Kotlin provides several ways to launch coroutines:

- **launch()**: Launches a new coroutine without returning a result. Typically used for fire-and-forget tasks.
- **async()**: Launches a coroutine and returns a Deferred object, which you can use to get the result later.
- **runBlocking()**: Used to run a coroutine on a blocking thread (often used in tests or the main function).
- **GlobalScope.launch()**: Used for global scope coroutines, which run for the entire app lifecycle. This is generally discouraged for structured concurrency.\

104) What is the difference between yield() and Thread.sleep() in coroutines?

- **yield():** It is a suspending function that tells the coroutine dispatcher to give up the current thread and allow other coroutines to run. It doesn't block the thread but allows the scheduler to switch to another coroutine that may be ready to run.
- **Thread.sleep():** It is a blocking function that halts the entire thread for the specified duration, preventing any other tasks from executing on that thread.

While Thread.sleep() can cause performance issues by blocking the thread, yield() provides a cooperative suspension mechanism that does not block the thread.

Created by Anand Gaur

Android Architecture Concepts

1) What is Android Architecture?

- It defines a way to structure code into layers.
- Helps separate UI, data, and business logic.
- Makes the code easy to maintain, test, and scale.

2) What is MVVM Architecture?

- MVVM stands for **Model-View-ViewModel**.
- **Model**: Manages data (e.g., from API or database).
- **View**: UI layer (Activity, Fragment, or Compose).
- **ViewModel**: Holds UI data and business logic, survives screen rotation.
- Helps reduce code in Activity/Fragment.

3) What is ViewModel?

- Part of Android Architecture Components.
- Stores UI-related data across configuration changes.
- Provides data to the View using LiveData or StateFlow.
- Doesn’t contain references to View (Activity/Fragment).

4) What is LiveData?

- Lifecycle-aware observable data holder.
- UI observes LiveData to get automatic updates.
- Prevents memory leaks as it only updates when the UI is active.

Created by Anand Gaur

5) What is the difference between LiveData and StateFlow?

- **LiveData** is lifecycle-aware, works well with XML-based UIs.
- **StateFlow** is not lifecycle-aware, works better with Kotlin Coroutines and Jetpack Compose.
- StateFlow is a part of Kotlin Flow and used for modern reactive UIs.

6) What is Repository in MVVM?

The Repository is responsible for fetching data. It abstracts the data sources (API, Room database, Firebase, etc.) from the ViewModel. This separation makes it easy to manage and test data logic.

7) What are UseCases in Clean Architecture?

- A UseCase contains **a single specific business logic** (e.g., GetUserDetails).
- Keeps the ViewModel clean by handling complex logic inside it.
- Lies in the **domain layer** in Clean Architecture.
- Reusable and testable units of code.

8) What is Clean Architecture?

- Divides app into **three layers**:
    - **Presentation**: ViewModel, UI
    - **Domain**: Business logic (UseCases)
    - **Data**: API, Room, Repositories
- Makes code **modular, testable, and maintainable**.
- Helps in scaling large applications.

Created by Anand Gaur

9) What is Room in Android Architecture?

- Room is a library that provides an easy way to use SQLite.
- Uses annotations to create and query databases.
- Helps persist data locally (offline storage).
- Works with ViewModel and LiveData for reactive updates.

10) What are the benefits of using ViewModel, LiveData, and Room?

- **ViewModel** keeps data during screen rotation.
- **LiveData** automatically updates UI with data changes.
- **Room** provides easy, safe database operations.
- All three are lifecycle-aware and prevent memory leaks.
- Improve app stability, maintainability, and performance.

11) What is DataBinding in Android Architecture?

- Allows you to **bind UI components directly to data sources** in XML.
- Reduces boilerplate code (like `findViewById()` or `setText()`).
- Works well with ViewModel and LiveData for reactive UI updates.
- Helps keep XML and business logic loosely coupled.

Created by Anand Gaur

12) What is the difference between DataBinding and ViewBinding?

- **ViewBinding**: Only helps access views easily (no logic in XML).
- **DataBinding**: Lets you add expressions and bind LiveData/variables in XML.
- Use ViewBinding for simple layouts, DataBinding for complex reactive UIs.

13) What is Dependency Injection in Android?

- A technique to **provide dependencies from outside**, instead of creating them inside classes.
- Makes code more testable and loosely coupled.
- Popular libraries: **Hilt**, **Dagger**, and **Koin**.

14) What is Hilt and why do we use it?

- **Hilt** is Android's official DI library built on top of Dagger.
- Provides a simple way to inject dependencies in ViewModels, Activities, Fragments, etc.
- Reduces boilerplate and manages scopes automatically.
- Great for large apps with many dependencies.

15) What is the Single Source of Truth in MVVM or Clean Architecture?

- Means all UI data should come from **one place only** — usually the Repository.
- Avoids bugs, keeps data consistent, and simplifies debugging.

Created by Anand Gaur

16) Why should the ViewModel not reference a Context directly?

- ViewModel survives configuration changes.
- If it holds a reference to Context, it may cause memory leaks.
- Use **Application context via AndroidViewModel** if really needed.

17) What is the role of UseCase in Clean Architecture?

- Each UseCase focuses on **one business operation** (like FetchUserDetails).
- Keeps ViewModel clean and focused only on presenting data.
- Makes business logic reusable across different screens/modules.

18) What are the common mistakes developers make in MVVM?

- Putting logic in Activities or Fragments instead of ViewModel.
- Making ViewModel too heavy with too many responsibilities.
- Skipping Repository layer and calling network/database directly from ViewModel.
- Not handling LiveData properly (e.g., multiple observers or memory leaks).

19) What is MediatorLiveData and when to use it?

- **MediatorLiveData** observes multiple LiveData sources.
- Useful when you want to **combine data from multiple LiveData** in one place.
- For example, combining two LiveData values into one UI update.

Created by Anand Gaur

20) How do you handle errors in ViewModel and Repository?

- Use **sealed classes** or **Result wrapper** (Success, Error, Loading).
- Repository returns `Result<Success/Error>` to ViewModel.
- ViewModel observes it and updates the UI accordingly.

21) What is Clean Architecture and why do we use it in Android?

- Clean Architecture separates your code into **layers**:
  - **Presentation (UI)** → Activity/Fragment/ViewModel
  - **Domain** → Business logic (UseCases)
  - **Data** → API/database operations
- It helps you:
  - Keep the app **scalable**, **testable**, and **easy to maintain**
  - Avoid tight coupling between UI and logic
- **Example:** If you want to switch from REST API to GraphQL, only the Data layer changes, not the business logic or UI.

22) Why do we need a Repository in MVVM or Clean Architecture?

- The **Repository** acts as a **single source of truth** for your data.
- It manages data from multiple sources like API, database, cache, etc.
- ViewModel only talks to the Repository, not to the network or Room directly.
- This improves **separation of concerns** and makes unit testing easier.

Created by Anand Gaur

23) What is the difference between Domain Model and UI Model?

- **Domain Model**: Pure business logic representation (used in UseCases).
- **UI Model**: What your screen or UI needs (includes UI-related fields like isSelected, formattedDate).
- You should map Domain Model to UI Model inside ViewModel to keep UI logic separate from business logic.

24) What is the role of UseCases in Clean Architecture?

- Each UseCase should do **one thing only** (like `GetUserProfile()` or `SubmitOrder()`).
- Keeps your business logic reusable, testable, and clean.
- ViewModel calls UseCase, and UseCase interacts with Repository.
- This avoids bloated ViewModels and mixes of UI + business logic.

25) What is the difference between MVP and MVVM?

- **MVP**: View talks to Presenter, Presenter talks back to View.
- **MVVM**: View observes ViewModel via LiveData/StateFlow, no direct call-back.
- **MVVM** is more lifecycle-aware, reactive, and Jetpack-friendly.
- With Jetpack Compose, MVVM works more naturally with `State` and `Flow`.

26) What is a Shared ViewModel and when do you use it?

- Shared ViewModel is used when **two or more fragments need to share data**.
- Works best with `activityViewModels()` delegate in Jetpack.
- **Example:** Master-Detail screens, where selection in one fragment updates UI in another.

Created by Anand Gaur

27) What are the advantages of using sealed classes with ViewState or Result wrapper?

- Sealed classes allow you to define a **fixed set of states** like:

```
sealed class UiState {
    object Loading : UiState()
    data class Success(val data: List<User>) : UiState()
    data class Error(val message: String) : UiState()
}
```

- Helps manage UI state clearly from ViewModel.
- Prevents runtime errors because compiler enforces all cases in `when`.

28) How do you handle multiple API calls in ViewModel using Kotlin Flow or Coroutines?

- Use `combine`, `zip`, or `flatMapLatest` to run parallel or dependent calls.
- Use `viewModelScope.launch` to safely launch coroutines in ViewModel.
- Handle errors using `catch {}` block or `runCatching`.

Example:

```
viewModelScope.launch {
   val user = async { repository.getUser() }
   val posts = async { repository.getPosts() }

   // Wait for both
   val userData = user.await()
   val postData = posts.await()
}
```

29) What is a ViewModel Factory and why is it needed?

- When your ViewModel takes constructor parameters (like UseCase, Repository), Android's default ViewModelProvider can't handle it.
- You need to create a **custom ViewModelFactory** to pass those parameters.
- If you're using **Hilt**, it handles ViewModel injection automatically with `@HiltViewModel`.

30) What is Unidirectional Data Flow (UDF) and why is it useful in MVVM?

- UDF means:
  - **Event** comes from UI → ViewModel
  - ViewModel processes → Emits new **State**
  - UI **observes and reacts** to that State
- Helps manage screen behavior clearly, especially in Compose or modern XML.
- Makes the architecture **predictable**, easier to test and debug.

31) What is Dependency Injection and why do we use it?

- Dependency Injection (DI) is a design pattern where a class receives its dependencies from an external source rather than creating them itself.
- This improves:
  - **Code reusability**
  - **Testability** (easy to mock dependencies)
  - **Loose coupling** between components

Created by Anand Gaur

32) What is Hilt in Android and how is it different from Dagger?

- **Hilt** is a DI library built on top of Dagger, optimized for Android development.
- Dagger requires a lot of boilerplate (modules, components).
- Hilt simplifies this by:
  - Automatically generating required components
  - Providing predefined scopes for Android classes (Activity, Fragment, ViewModel, etc.)
  - Integrating seamlessly with Jetpack libraries like ViewModel, WorkManager, Navigation

33) What are the key annotations used in Hilt?

- `@HiltAndroidApp`: Initializes Hilt and triggers code generation. Add to the Application class.
- `@AndroidEntryPoint`: Marks Android classes (Activity, Fragment, Service) where you want to inject dependencies.
- `@Inject`: Used on constructors or fields to request dependencies.
- `@Module` + `@InstallIn`: Defines how to provide objects (dependencies).
- `@Provides`: Tells Hilt how to create an instance of a dependency manually.
- `@Singleton`, `@ActivityScoped`, `@ViewModelScoped`, `@ServiceScoped`: Define the lifespan of dependencies.

Created by Anand Gaur

34) What is the use of @HiltAndroidApp?

- Added to the Application class.
- Sets up the base Hilt component (ApplicationComponent).
- It allows Hilt to perform dependency injection across the entire app.

```
@HiltAndroidApp
class MyApplication : Application()
```

35) What is @AndroidEntryPoint and where should it be used?

- Marks Android framework classes (Activity, Fragment, ViewModel, etc.) as injection targets.
- Required on every class where you want to inject dependencies using `@Inject`.

```
@AndroidEntryPoint
class MainActivity : AppCompatActivity() {
    @Inject lateinit var userRepository: UserRepository
}
```

36) What is a Hilt Module and why do we use it?

- A **Hilt Module** is a class where you define how to **provide objects manually**.
- Use it when you can't use `@Inject` on constructor (like Retrofit, Room, third-party classes).

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    fun provideRetrofit(): Retrofit { ... }
}
```

37) How do you inject a ViewModel using Hilt?

- Use `@HiltViewModel` on your ViewModel.
- Inject dependencies in the constructor.
- Use `by viewModels()` or `hiltViewModel()` in Compose.

```
@HiltViewModel
class MyViewModel @Inject constructor(
    private val repo: MyRepository
) : ViewModel()
```

38) What is the difference between SingletonComponent and ActivityRetainedComponent?

- `SingletonComponent`:
  - Lives as long as the Application
  - Best for global dependencies (Retrofit, Room, etc.)
- `ActivityRetainedComponent`:
  - Lives across configuration changes (like ViewModel)
  - Used for `@ViewModelScoped` dependencies

39) What is ViewModelScoped in Hilt?

- Ensures the dependency lives **as long as the ViewModel**.
- Useful when multiple UseCases or helpers share common logic/state

```
@Provides
@ViewModelScoped
fun provideUseCase(): MyUseCase = ...
```

Created by Anand Gaur

40) How do you inject dependencies in WorkManager with Hilt?

- Hilt has special support for **WorkManager** via `@HiltWorker`.
- Use `HiltWorkerFactory` to provide dependencies.

41) Can Hilt be used in multi-module projects?

Yes, but:

- Each module must declare its own `@Module` and `@InstallIn`.
- Dependencies from different modules must be exposed properly.
- Use `@Binds` or `@Provides` in interface modules for abstraction.

42) What is the difference between `@Inject constructor` and `@Provides`?

- `@Inject constructor`: Hilt automatically understands how to create the object (used with your own classes).
- `@Provides`: You manually tell Hilt how to create the object (used with third-party classes or complex setup).

43) What is the use of @Binds in Hilt?

- Used to bind an **interface to its implementation**.
- Reduces boilerplate when the instance is created via constructor.

44) How do you test classes that use Hilt?

- Use `@HiltAndroidTest` for integration tests.
- Use `@UninstallModules` to replace production modules with fake/test ones.
- Use `@BindValue` or `@TestInstallIn` to inject mocks/fakes.

Created By Anand Gaur

45) Common mistakes to avoid while using Hilt

- Forgetting `@AndroidEntryPoint` on Fragment/Activity/View.
- Injecting into a class not supported by Hilt without proper `EntryPoint`.
- Using the wrong scope (`@Singleton` inside ViewModel).
- Mixing Dagger and Hilt manually (not recommended).

46) How does the ViewModel work internally?

The `ViewModel` is part of Android's **Architecture Components**. It is designed to **store and manage UI-related data** in a lifecycle-conscious way. Here's a **simple explanation** of how it works under the hood:

1. Survives Configuration Changes

- When a configuration change occurs (e.g., screen rotation), the **Activity or Fragment is destroyed and recreated**.
- But the **ViewModel survives**, meaning it is **not destroyed**, so your data doesn't get lost.

Internally, the `ViewModel` is **stored in a special ViewModelStore**, which is tied to the **lifecycle of the Activity/Fragment**.

2. Tied to Lifecycle

- Each **Activity or Fragment** has a `ViewModelStoreOwner`.
- When you call:

```
ViewModelProvider(this).get(MyViewModel::class.java)
```

- It checks the `ViewModelStore` to see if a ViewModel already exists.
    - If yes, it returns the **existing instance**.
    - If no, it creates a **new instance** using a **ViewModelProvider.Factory**.

Created by Anand Gaur

3. ViewModelStore & ViewModelProvider

- `ViewModelStore`: A container that holds `ViewModel` instances.
- `ViewModelProvider`: Responsible for getting or creating `ViewModel` instances and storing them in `ViewModelStore`.

4. ViewModel Lifecycle

- **Created** when first requested.
- **Retained** across configuration changes.
- **Cleared** when the `Activity` or `Fragment` is **finished or destroyed permanently** (e.g., when the user presses back).

When the ViewModel is no longer needed, the system calls:

```
onCleared()
```

You can override this to clean up resources (e.g., close database, cancel coroutines).

5. Used with LiveData / StateFlow

- ViewModel holds **LiveData** or **StateFlow** to expose data to the UI.
- Since ViewModel survives configuration changes, it ensures that **UI doesn't reload data unnecessarily** after rotation or other events.

6. Memory Safety

- ViewModel **doesn't hold reference to UI elements** (like Activities or Views) to avoid memory leaks.
- It should only store **UI-related data**, not UI components.

47) What is Single Activity Architecture and why use it?

Most modern apps use a **single Activity** as the host, and swap multiple **Fragments** or **NavHost** within it.

Benefits:

- Simplifies navigation (back stack is managed by NavController)

Created by Anand Gaur

- Clear separation: Activity handles system-level logic; Fragments handle UI
- Easier dependency injection and lifecycle handling

48) What is Navigation Component and how does it fit MVVM?

The Navigation Component manages in-app navigation and replaces manual Fragment transactions.

How it fits MVVM:

- You define nav actions in XML Graph
- ViewModel emits navigation events (via LiveData or Flow)
- View (Fragment/Activity) observes these and calls `findNavController().navigate(...)`, keeping the ViewModel free from UI logic

49) What is DataStore and why is it preferred over SharedPreferences?

DataStore is Jetpack's modern data storage solution (preferences or typed). It's:

- **Asynchronous** (uses Coroutines & Flow, non-blocking I/O)
- **Typed-safe** with Proto DataStore
- Handles **errors gracefully**
  Works well inside the MVVM architecture via dependency injection

50) What is Mediator in MVVM using Flow?

Analogous to `MediatorLiveData`, **Mediator Flow** combines or switches between multiple Flows.

Use-case:

```
val combined = flow1.combine(flow2) { a, b -> /* merge */ }
```


Created by Anand Gaur

ViewModel exposes `combined` as StateFlow or LiveData to the UI—UI just collects it.

51) How do you handle one-time navigation or toast events in MVVM?

Common pattern: You wrap such events using `Event` or `SharedFlow` instead of plain LiveData.
Example:

```
private val _navigate = MutableSharedFlow<Unit>()
val navigate = _navigate.asSharedFlow()
```

UI collects the flow once and handles the event—no duplicate toasts or unintended navigation on rotation.

52) What is Paging 3 and how does it integrate with MVVM?

Paging 3 helps load large lists in chunks using Flow.
Architecture:

- Repository builds `Pager<..., MyData>`
- ViewModel exposes a flow using `cachedIn(viewModelScope)`
- UI (Compose or RecyclerView) collects the flow and submits it to `PagingDataAdapter`

53) What is FragmentFactory and when to use it?

`FragmentFactory` lets you inject dependencies into Fragments via constructors. When using DI (e.g., Hilt or manual DI), `FragmentFactory` ensures Fragments are created with required args rather than using no-arg constructors and `arguments` bundle.

Created by Anand Gaur

54) How does the `SavedStateHandle` help in ViewModel?

`SavedStateHandle` allows ViewModel to **persist data across process death** (not just config changes).

Use-case:
Storing form input, filter selection, or scroll position.

```
class MyViewModel(savedStateHandle: SavedStateHandle) : ViewModel() {
    val query = savedStateHandle.getLiveData<String>("query")
}
```

55) What is a `StateHolder` ViewModel pattern?

A **StateHolder ViewModel** owns the entire UI state in a single data class.

Example:

```
data class UiState(val isLoading: Boolean, val data: List<User>)
```

The ViewModel updates this `UiState` immutably and exposes it as `StateFlow` or `LiveData` → making UIs reactive and predictable.

56) Why avoid putting logic in the `onCleared()` method?

While `onCleared()` is meant for cleanup (canceling jobs), putting business logic or saving data here is risky because:

- It's not guaranteed to be called on process kill.
- It's better to use lifecycle-aware scopes and `SavedStateHandle` or `onStop()` callbacks in the View/UI layer.

56) What is the purpose of `viewModelScope.launch {}` in Android?

It's a coroutine scope tied to the ViewModel lifecycle.

- Automatically cancelled in `onCleared()`

Created by Anand Gaur

- Prevents memory leaks
- Ideal for launching network/db tasks from ViewModel

57) How to avoid multiple API calls on screen rotation?

By using `viewModelScope` and `LiveData`/`StateFlow` inside ViewModel which survives rotation.
Don't initiate API calls inside `onCreate()` or `onViewCreated()` every time—observe the ViewModel instead.

58) When would you avoid using LiveData?

- When working with **non-UI layers** like Repository or UseCase
- When you need **cold streams** or backpressure
- When using **Jetpack Compose**, prefer **Flow/StateFlow** instead

59) What is Dagger 2?

Dagger 2 is a **fully static, compile-time dependency injection (DI) framework** for Java and Android, developed by Google. It helps you manage and provide dependencies across your application **without writing boilerplate code manually**.

Key Points:

- **Dependency Injection** is a design pattern where dependencies (like API clients, Repositories, etc.) are provided to a class rather than the class creating them itself.
- **Dagger 2** generates code at **compile time**, which makes it **faster and more efficient** than reflection-based frameworks (like Guice).
- It helps in writing **modular, testable, and maintainable** code.

How it Works:

1. **@Inject** — Used to tell Dagger where and how it should provide dependencies.
    - **Example:** Constructor injection inside ViewModel, Repository, etc.

Created by Anand Gaur

2. **@Module** — A class that provides objects which can't be constructed using `@Inject`.
    - You define functions with `@Provides` inside it.

3. **@Provides** — Used inside `@Module` classes to tell Dagger how to create instances.

4. **@Component** — It connects all modules and injection targets. Think of it as the bridge between your objects and the dependencies they need.

5. **@Singleton** — Tells Dagger to create only one instance and reuse it wherever required.

60) How ViewModel handles state?

The `ViewModel` class in Android Jetpack is designed to store and manage UI-related data in a lifecycle-conscious way. This means the `ViewModel` survives configuration changes like screen rotations and provides a central place to handle UI logic.

Why Use ViewModel for State Management?

- Activities and Fragments are short-lived and get recreated on events like orientation changes.
- Keeping UI state in `ViewModel` prevents loss of data and reduces boilerplate code to restore the UI.

61) What is the Repository pattern in Android?

The Repository pattern is a design pattern that abstracts the data sources and provides a clean API for the rest of the application to retrieve or modify data. It acts as a mediator between different data sources (like local databases, remote APIs, or cache) and the ViewModel/UseCase layers.

Why Use Repository Pattern?

- Decouples the data layer from the presentation layer.
- Makes code easier to maintain and test.
- Promotes separation of concerns.
- Simplifies switching or combining data sources.

Created by Anand Gaur

62) What is the Use Case layer in Clean Architecture?

In Clean Architecture, the Use Case layer (also referred to as the Interactor layer) encapsulates the **business logic** of the application. It defines and executes specific actions or user operations by interacting with repositories or other domain objects.

Purpose:

- Acts as the **middle layer** between UI/ViewModel and the data layer (Repository).
- Keeps business rules **independent of frameworks**, databases, or UI.
- Improves **testability** and **separation of concerns**.

Benefits:

- Clear **separation of concerns** – UI doesn't need to know how data is fetched.
- Logic is **independent** of UI or Android Frameworks.
- Easy to write **unit tests** for business logic.
- Encourages **reusability** across different layers (e.g., same use case used in multiple screens).

63) How to structure modules in Clean Architecture

In Clean Architecture, modularization is crucial for enforcing separation of concerns, improving code reusability, and simplifying testing and maintenance. Each layer of the architecture is separated into different modules to isolate responsibilities.

Recommended Module Structure:

```
- app/ (UI Module)
- domain/ (Business Logic)
- data/ (Repositories, API, DB)
- core/ (Common utilities, base classes)
```

Benefits of Modular Clean Architecture:

- Improved build times with Gradle parallelization.
- Independent testing of layers (you can mock data layer while testing domain).
- Better scalability for large teams and large projects.
- Clear separation makes onboarding and refactoring easier.

Structuring your app this way aligns with SOLID principles and makes your codebase robust, scalable, and easier to maintain.

Created by Anand Gaur

64) What are the Benefits and drawbacks of Clean Architecture?

Clean Architecture offers a structured and scalable way to organize code by separating concerns into layers. While it's a powerful architectural approach, it comes with both strengths and challenges.

Benefits:

1. **Separation of Concerns:**
   - Clear division between UI, business logic, and data layers.
   - Makes it easier to understand, manage, and modify code.
2. **Testability:**
   - Business logic is isolated from Android framework classes, enabling easy unit testing of UseCases and Repositories.
3. **Maintainability:**
   - As your app grows, code becomes easier to scale and refactor due to well-defined boundaries.
4. **Reusability:**
   - Logic in the domain layer can be reused across different platforms (e.g., Android, iOS via KMM).
5. **Flexibility and Independence:**
   - You can change data sources (e.g., switch from REST API to GraphQL or Room to Realm) without affecting other layers.
6. **Team Collaboration:**
   - Large teams can work on different layers independently with fewer merge conflicts.
7. **Adaptability for Cross-platform Solutions:**
   - Clean Architecture aligns well with Kotlin Multiplatform and multi-module Android projects.

Drawbacks:

1. **Initial Complexity:**
   - Steep learning curve for new developers.
   - Understanding boundaries, module dependencies, and abstract layers requires experience.
2. **Boilerplate Code:**
   - May introduce more interfaces, abstractions, and classes than simpler architectures.

Created by Anand Gaur

3. **Overhead for Small Projects:**
    - For small apps or quick prototypes, Clean Architecture might be overkill.
    - Simpler architectures like MVVM alone may suffice.
4. **Slower Initial Development:**
    - Setting up module structure, DI (e.g., Hilt/Dagger), and layering may delay MVP delivery.
5. **Too Abstract If Not Applied Properly:**
    - Developers may introduce unnecessary layers or interfaces that add no value but increase complexity.

65) Why use Dependency Injection in architecture?

Dependency Injection (DI) is a design pattern that enables objects to receive their dependencies from external sources rather than creating them internally. In Android architecture, DI simplifies code structure, improves modularity, and enhances testability.

Core Idea:

Instead of a class instantiating its own dependencies, they are provided (injected) by another class or framework.

Why It Matters in Android:

1. **Decoupling Components:**
    - DI promotes separation of concerns. A `ViewModel` shouldn't know how to construct a `Repository`. It just needs to use it.
    - This makes components more reusable and easier to test.
2. **Improved Testability:**
    - You can inject mock or fake implementations of dependencies during testing.
    - For example, testing a `LoginUseCase` with a mock API service.
3. **Easier Code Maintenance:**
    - When dependencies are injected, switching implementations (e.g., Retrofit API to GraphQL or Firebase) requires fewer code changes.
4. **Cleaner and Scalable Architecture:**
    - In large-scale apps, managing dependencies manually becomes messy.
    - DI frameworks like Hilt and Dagger 2 automate this process efficiently.

Created by Anand Gaur

66) How to implement event-based architecture?

Event-based architecture (EBA) is a software design pattern where components communicate by producing and responding to events. Instead of direct method calls, components emit events and listen for them asynchronously. This promotes loose coupling and enhances flexibility and scalability.

Core Concepts:

- **Event Producer (Publisher):** Component that triggers an event (e.g., ViewModel emitting a UI state).
- **Event Consumer (Subscriber):** Component that listens and responds to events (e.g., UI observing changes).
- **Event Bus (optional):** A centralized medium to pass events (e.g., using LiveData, Flow, or third-party libs like RxJava or EventBus).

Why Event-Based Architecture in Android?

- Decouples components like ViewModel, UseCase, and UI.
- Makes system more reactive.
- Enhances testability (no tight coupling between sender and receiver).

66) What is Room and why is it preferred over SQLite directly?

Room is an abstraction layer over SQLite that provides:

- Compile-time verification of SQL queries
- Easy DAO interface for database access
- Integration with LiveData, Flow, Coroutines
- Cleaner and safer API compared to raw SQLite

67) What are the core components of Room?

1. **Entity** – Represents a table.
2. **DAO (Data Access Object)** – Interface that contains query methods.
3. **Database** – Abstract class annotated with `@Database` that connects everything.

Created by Anand Gaur

68) How does Room work with LiveData and Flow?

- Room automatically updates `LiveData` or `Flow` when the database changes.
- Useful for building reactive UI.

69) What is the purpose of @Insert, @Update, and @Delete in DAO?

These annotations tell Room to generate SQL behind the scenes:

- `@Insert` – Adds a new row
- `@Update` – Modifies existing row based on primary key
- `@Delete` – Deletes a row

Room figures out the query automatically.

70) What is a TypeConverter in a Room?

Room only supports primitive types by default.
To store custom types (like `Date`, `List<String>`, etc.), you use a `TypeConverter`.

71) How does Room handle one-to-many and many-to-many relationships?

- **One-to-Many**: Use a foreign key and a POJO with embedded relationships using `@Relation`.
- **Many-to-Many**: Use a third table (junction) and define cross-reference.

```
data class UserWithBooks(
  @Embedded val user: User,
  @Relation(
    parentColumn = "userId",
    entityColumn = "ownerId"
  )
  val books: List<Book>
)
```

Created by Anand Gaur

72) What is a Room Migration and when do you need it?

When your schema changes (e.g., adding/removing columns), you must **define a migration** to update existing databases without data loss.

```
val MIGRATION_1_2 = object : Migration(1, 2) {
  override fun migrate(database: SupportSQLiteDatabase) {
      database.execSQL("ALTER TABLE user ADD COLUMN age INTEGER")
  }
}
```

73) What happens if you don't handle migration?

The app will crash with a `IllegalStateException: Room cannot verify the data integrity`.

Use `.fallbackToDestructiveMigration()` in dev only (it **wipes user data**).

74) How to handle Room operations on background threads?

Room allows DB access only from a background thread.

Use:

- `suspend` functions with coroutines
- `Flow` to stream changes
- `Executor` or `viewModelScope.launch(Dispatchers.IO)` to call DAO safely

75) What is the use of `@Embedded` in Room?

Used to flatten a nested object into the table schema.

Example:

```
data class Address(val city: String, val state: String)

@Entity
data class User(
  @PrimaryKey val id: Int,
  @Embedded val address: Address
)
```


Created by Anand Gaur

76) How to update only specific fields in Room?

You can write a custom `@Query` to update only one or two fields:

```
@Query("UPDATE user SET name = :name WHERE id = :id")
suspend fun updateName(id: Int, name: String)
```

Avoid using `@Update` if partial update is needed.

Created by Anand Gaur

Jetpack Compose Concepts

1) What is Jetpack Compose?

Jetpack Compose is Android's modern UI toolkit that lets you build UI using Kotlin code instead of XML.

- It's **declarative**, meaning you describe what the UI should look like, and the system updates it automatically when the data changes.
- It replaces traditional XML + View-based UI system.
- Offers **less boilerplate**, better state handling, and **Kotlin-first approach**.

2) What is a Composable function?

A Composable is a special Kotlin function marked with `@Composable` that describes part of the UI.

Example:

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name")
}
```

You can call one composable inside another to build complex UIs.

3) What is recomposition in Jetpack Compose?

Recomposition is when Compose **redraws** parts of the UI because data/state has changed.

- Only the part of UI where data changed is recomposed.
- Compose optimizes this to avoid redrawing everything.

Example: If you update a `count` value shown in a `Text`, only that `Text` composable will recompose.

4) What is State in Compose?

State holds data that changes over time and triggers recomposition.

You can use `remember` and `mutableStateOf`:

```
val count = remember { mutableStateOf(0) }
```

When `count.value` changes, any UI that depends on it will update automatically.

5) What is remember and rememberSaveable?

- `remember` stores state during recomposition but **resets on configuration changes** (like rotation).
- `rememberSaveable` stores state across recomposition **and configuration changes** using `Bundle`.

Use `rememberSaveable` for things like text input or selection state that should survive screen rotation.

6) What is Modifier in Jetpack Compose?

Modifier is used to **modify or decorate** a composable — like setting padding, background, size, click behavior, etc.

Example:

```
Text(
  text = "Hello",
  modifier = Modifier
    .padding(16.dp)
    .background(Color.Yellow)
)
```

Modifiers are chained and read from **left to right**.

7) What is a Scaffold in Jetpack Compose?

Scaffold is a layout component that provides basic structure like:

- TopBar
- BottomBar
- FloatingActionButton
- Drawer
- SnackbarHost

Example:

```
Scaffold(
    topBar = { TopAppBar(title = { Text("Home") }) },
    floatingActionButton = { FloatingActionButton(onClick = {}) {
Text("+") } }
) {
    // Content
}
```

Useful for material design layouts.

8) What is SideEffect in Jetpack Compose?

`SideEffect` is used to run code **after** successful recomposition.

Use it to update things outside Compose like logging, analytics, etc.

Example:

```
SideEffect {
    Log.d("Compose", "Recomposition done")
}
```

It runs on every successful recomposition.

9) What is LaunchedEffect in Jetpack Compose?

`LaunchedEffect` is used to launch coroutines in Compose safely.
It's lifecycle-aware and cancels the coroutine if the composable leaves the composition.

Example:

```
LaunchedEffect(key1 = userId) {
    viewModel.loadUser(userId)
}
```

It only runs when the key changes or first time it's launched.

10) What is derivedStateOf in Compose?

`derivedStateOf` is used to create a **computed value** that only updates when its dependencies change.

Example:

```
val filteredList = remember {
    derivedStateOf { itemList.filter { it.isSelected } }
}
```

Improves performance by avoiding unnecessary recomposition.

11) What is recomposition skipping?

Compose avoids redrawing unchanged composables during recomposition.
Only those that **depend on changed state** will be redrawn.

This is an optimization feature that improves performance.

12) What is rememberCoroutineScope used for?

This is used to launch coroutines from a composable scope (like button click).

13) How does Navigation work in Jetpack Compose?

Jetpack Compose has its own navigation system.

Steps:

1. Add `navigation-compose` dependency.
2. Use `NavHost` to define routes/screens.
3. Navigate using `navController.navigate("route")`.

Example:

```
NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen() }
    composable("details") { DetailScreen() }
}
```

14) What is CompositionLocal in Jetpack Compose?

`CompositionLocal` provides **implicit dependencies** (like theme, context, or current user) to the subtree.

Use `LocalContext.current` to access the `Context` inside Compose.

You can also define your own:

```
val LocalUser = compositionLocalOf<User> { error("No user found") }
```

Created by Anand Gaur

15) How do you create a custom Composable?

You define a Kotlin function and annotate it with `@Composable`.

Example:

```
@Composable
fun CustomButton(text: String, onClick: () -> Unit) {
    Button(onClick = onClick) {
        Text(text)
    }
}
```

Reusable and follows Compose lifecycle.

16) What is a Slot API in Compose?

A Slot API lets you pass composables into other composables.

For example:

```
@Composable
fun CustomCard(content: @Composable () -> Unit) {
    Card {
        content()
    }
}
```

You can pass any UI to `CustomCard`.

17) What is Keys in LazyColumn?

When using `LazyColumn`, using keys improves scroll position and recomposition tracking.

Example:

```
LazyColumn {
    items(items = userList, key = { it.id }) { user ->
        Text(user.name)
    }
}
```

18) How do you show Dialogs in Jetpack Compose?

You can use `AlertDialog`:

```
if (showDialog) {
    AlertDialog(
        onDismissRequest = { showDialog = false },
        title = { Text("Title") },
        text = { Text("Message") },
        confirmButton = { Button(onClick = { showDialog = false }) {
Text("OK") } }
    )
}
```

19) How do you handle performance in Jetpack Compose?

- Use `remember` and `derivedStateOf` to reduce recompositions
- Use `LazyColumn` for long lists
- Use `key` in `items()` to track items uniquely
- Avoid recreating heavy objects inside `@Composable`
- Use proper scoping for coroutines

21) How do you manage state in Jetpack Compose?

In Jetpack Compose, state is managed using:

- `remember`: stores state across recompositions
- `mutableStateOf`: makes state observable
- `rememberSaveable`: saves state across configuration changes
- `ViewModel`: preferred for long-living state across configuration changes

Example:

```
val counter = remember { mutableStateOf(0) }
Text(text = "Count: ${counter.value}")
```

22) How do you handle TextField input in Compose?

Use `TextField` with a `remember` state:

```
var text by remember { mutableStateOf("") }

TextField(
    value = text,
    onValueChange = { text = it },
    label = { Text("Enter name") }
)
```

Use `rememberSaveable` if you want it to persist through configuration changes.

23) What is a Preview in Jetpack Compose?

Preview allows you to see how your Composable will look in Android Studio **without running the app**.

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    Greeting(name = "Compose")
}
```

You can preview multiple UIs with different parameters.

24) How do you create a LazyColumn in Compose?

`LazyColumn` is used for vertical scrolling lists:

```
LazyColumn {
    items(itemsList) { item ->
        Text(text = item)
    }
}
```

It only renders visible items — optimized like RecyclerView.

25) What is `produceState` in Jetpack Compose?

`produceState` bridges **suspending data sources** (like APIs) with Compose state.

Example:

```
val data by produceState(initialValue = null) {
    value = fetchDataFromApi()
}
```

Used when fetching data directly in Composables (not recommended for business logic-heavy flows — use ViewModel instead).

26) What is `rememberUpdatedState` in Compose?

This helps to access the **latest lambda or value** in asynchronous code, such as when using `LaunchedEffect`.

Used to avoid capturing old values.

27) What is `snapshotFlow`?

`snapshotFlow` converts Compose `State` into a `Flow`.

```
val state = remember { mutableStateOf(0) }
LaunchedEffect(Unit) {
    snapshotFlow { state.value }
        .collect { value -> println("State changed to $value") }
}
```

Useful for combining Compose with Kotlin Flow.

28) What is the difference between `remember` and `rememberSaveable`?

- `remember` survives recomposition
- `rememberSaveable` survives recomposition **and** configuration changes (like rotation)

Use `rememberSaveable` for input forms or user-interactive components.

29) How do you animate UI in Jetpack Compose?

Use built-in APIs like:

- `animateDpAsState`
- `animateColorAsState`
- `updateTransition`
- `AnimatedVisibility`

Created by Anand Gaur

30) What is Composition in Jetpack Compose?

Composition is the process of building the UI tree by calling Composable functions.

It runs when a Composable enters the screen or needs to update due to state change.

31) How is Lifecycle handled in Jetpack Compose?

Compose uses `LifecycleOwner` internally, especially with `LaunchedEffect`, `DisposableEffect`, and `rememberCoroutineScope`.

Also works well with ViewModel and `lifecycleScope`.

32) How do you call a ViewModel in a Composable?

Use `viewModel()` inside a Composable:

```
val vm: MainViewModel = viewModel()
val data by vm.state.collectAsState()
```

If using Hilt, prefer `hiltViewModel()`.

33) What is the role of `DisposableEffect` in Compose?

`DisposableEffect` is used to **set up and clean up** side effects when a Composable enters or leaves composition.

```
DisposableEffect(Unit) {
    startSomething()
    onDispose {
        stopSomething()
    }
}
```

34) How do you pass arguments between screens in Navigation Compose?

You define arguments in `composable` route:

```
DisposableEffect(Unit) {
    startSomething()
    onDispose {
        stopSomething()
    }
}
```

Navigate using:

```
navController.navigate("profile/123")
```

35) Can you use Compose with existing XML screens?

Yes. You can:

- Use `ComposeView` inside XML layout
- Use `AndroidView` inside Compose to embed a View-based UI

This helps migrate apps screen-by-screen.

36) What is the lifecycle of a Composable?

Composable lifecycle is tied to the **Composition**.

Steps:

1. Enters composition → function runs
2. State changes → recomposition happens
3. Leaves composition → cleaned up (use `DisposableEffect`)

Created by Anand G

37) What are the key differences between XML and Jetpack Compose?

XML	Jetpack Compose
Declarative *in structure*, but relies on imperative code to update UI	Fully declarative – UI automatically updates when data changes
Uses `findViewById()`	No need for view binding
UI and logic are separate	UI and logic together in code
Manual state update	Automatic recomposition
More boilerplate	Less boilerplate

38) What are common performance issues in Compose?

- Unnecessary recompositions
- Forgetting to use `remember`
- Re-creating lists without `key`
- Passing large objects without optimization

Use tools like **Compose Tracing** and `RecomposeHighlighter` for optimization.

39) How do you integrate Paging 3 with Jetpack Compose?

1. Create a `PagingSource` to load paged data.
2. Use `Pager` to generate a `Flow<PagingData<T>>`.
3. Collect the data using `collectAsLazyPagingItems()`.
4. Display using `LazyColumn`.

Created by Anand Gaur

40) What is `LazyPagingItems`?

`LazyPagingItems<T>` is a wrapper for paged data in Compose. It allows efficient pagination within `LazyColumn` and provides methods like:

- `itemCount`
- `loadState` (to check loading/error)
- `retry()` (to re-trigger failed loads)

41) How do you handle loading states in Paging 3 with Compose?

Use `loadState` from `LazyPagingItems`:

```
val items = viewModel.items.collectAsLazyPagingItems()
when (val state = items.loadState.refresh) {
    is LoadState.Loading -> CircularProgressIndicator()
    is LoadState.Error -> Text("Error: ${state.error}")
}
```

Handle `append` and `prepend` similarly.

42) How do you retry failed Paging 3 loads in Compose?

Use `items.retry()`:

```
Button(onClick = { items.retry() }) {
    Text("Retry")
}
```

This retries both refresh and append/prepend failures.

43) How can you optimize Paging 3 with Jetpack Compose?

- Use `key` in `items()` to avoid recomposition:

```
items(items, key = { it.id }) { item -> ... }
```

- Debounce API requests using `RemoteMediator` and `PagingConfig`.
- Use placeholders for smooth scrolling if supported.

44) What is `RemoteMediator` in Paging 3 and how is it used with Compose?

`RemoteMediator` helps you fetch data from network and store in DB.

It supports **offline-first architecture**.

Compose doesn't need to care whether data is from network or DB — it just observes `PagingData`.

45) How would you build a dashboard with multiple data sources loading simultaneously?

- Use `viewModelScope.launch { }` and launch parallel coroutines.
- Use `combine()` to merge multiple Flows (e.g., User Info + Stats + Notifications).
- Use sealed UI states to track loading and errors.
- Render sections with independent loading indicators if needed.

46) How do you manage large UI state in Compose without performance issues?

- Break Composables into small reusable functions.
- Use `remember` and `derivedStateOf` to avoid recomposition.
- Use `snapshotFlow` to track state changes efficiently.
- Avoid passing large state objects; prefer immutability and unidirectional data flow.

Created by Anand Gaur

47) How would you design a form with dynamic fields using Compose?

Steps:

- Maintain form state in ViewModel as a list of `FormField` objects.
- Render fields using `when (field.type)` inside `LazyColumn`.
- Track validation via `StateFlow` or `Map<String, String?>` for errors.
- Use `rememberSaveable` for preserving input across configuration changes.

48) You need to show real-time stock prices in a list. How do you implement this?

Use:

- `StateFlow` or `Flow` emitting updates from WebSocket or API.
- Use `collectAsState()` or `snapshotFlow()` to render changes.
- Apply `key = { item.id }` in `LazyColumn` to minimize recompositions.
- Optionally use `LaunchedEffect` with `ticker` delay for simulated live updates.

49) How do you manage Compose navigation in a multi-module project?

- Use **Navigation Compose** and define routes in feature modules.
- Expose navigation APIs using sealed routes or string constants.
- Use shared NavGraphBuilder extension functions for modular screens.
- Avoid tight coupling — inject NavController only where needed.

Created by Anand Gaur

50) How do you test a Composable that uses ViewModel and StateFlow?

Steps:

- Use `createComposeRule()` and `AndroidComposeTestRule`.
- Provide fake ViewModel using Hilt or manual ViewModel factory.
- Expose UI state via `StateFlow` or test doubles.
- Use `composeTestRule.onNodeWithText()` for assertions.

51) How would you implement offline caching with Jetpack Compose + Paging 3?

- Use Room + RemoteMediator + PagingSource
- Cache API response to DB
- In Compose, just observe PagingData — no change in UI code
- Sync policies: on startup, on scroll, or periodically using WorkManager

52) How do you show paginated search results in Compose?

Steps:

- Use `SearchViewModel` with `Flow<PagingData<Item>>`
- When user types, debounce input (e.g., with `debounce(500)`)
- Re-trigger Pager based on query
- Use `LazyPagingItems` to show results

Created by Anand Gaur

53) How does animation work in Jetpack Compose?

Jetpack Compose uses **declarative animation APIs**, meaning you describe *what* the animation should do, and Compose takes care of *how* it's done.

You don't animate views or properties directly like in XML; instead, you animate values and the UI recomposes when those values change.

53) What are the common animation APIs in Jetpack Compose?

- `animate*AsState` (e.g., `animateDpAsState`, `animateColorAsState`)
- `updateTransition`
- `AnimatedVisibility`
- `Crossfade`
- `rememberInfiniteTransition`
- `Animatable` (for manual control)
- `Modifier.graphicsLayer()` and `Modifier.animateContentSize()`

54) What is `Animatable` and when should you use it?

Use `Animatable` when you need **low-level control** over animations, such as interrupting or chaining animations.

55) Can Compose animations be interrupted or canceled?

Yes, when using `Animatable`, you can:

- Interrupt current animation
- Use `coroutineContext.isActive`
- Call `stop()` or chain `animateTo()` calls

Created by Anand Gaur

56) How do you debug recompositions caused by animations?

Use:

- `Modifier.recomposeHighlighter()` to visually highlight recompositions
- `Log.d()` inside Composable scope
- Avoid placing `LaunchedEffect` or `remember` in places that get recomposed unnecessarily

57) Can you combine multiple animations in Compose?

Yes, use `updateTransition` or `LaunchedEffect` with `Animatable` for coordination.

Or trigger multiple `animate*AsState` together.

58) How do you animate a swipe gesture?

1. Use `Modifier.pointerInput` + `detectDragGestures`
2. Update offset using `Animatable`
3. Animate to target or snap back

59) How would you handle complex state in a large Jetpack Compose screen?

In a large screen with many states (text inputs, toggles, loading flags), it's best to:

- Use a **single UI state class** in ViewModel to hold all state fields.
- Expose it via **StateFlow or LiveData**.
- Use **immutable state objects** to avoid bugs caused by mutable state.
- Use **UI events** (one-time actions like navigation or showing Snackbar) as

Created by Anand Gaur

`SharedFlow` or `Channel`.

60) How would you manage recomposition performance in Jetpack Compose?

Jetpack Compose recomposes when state changes, so avoid unnecessary recomposition by:

- Making **Composable functions as small and reusable** as possible.
- Using `remember` and `derivedStateOf` to avoid recalculating values.
- Using `key()` block in LazyColumn to scope recomposition.
- Using `SnapshotFlow` to convert state into Flow and debounce changes if needed.
- Use tools like `Recompose Highlighter` or `Layout Inspector`.

61) What approach would you use for form validation in Compose (e.g., login/signup)?

Use the following strategy:

- Keep all input fields as part of a `UiState` object.
- Validate inside ViewModel when input changes.
- Use helper functions to return error messages.

62) How do you manage one-time events in Compose, like showing Snackbar or navigation?

Use **SharedFlow** or **Channel** to send one-time events from ViewModel.

62) How would you integrate Compose into an existing XML-based project?

Use the `ComposeView` in XML:

Created by Anand Gaur

```
<androidx.compose.ui.platform.ComposeView
    android:id="@+id/compose_view"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

Then in the Activity/Fragment:

```
composeView.setContent {
    MyComposable()
}
```

You can migrate screen by screen or use Compose inside dialogs, toolbars, etc.

63) How do you share state between multiple composables in Jetpack Compose?

Two main ways:

- **Hoist state** to a shared parent and pass it down
- **ViewModel** (shared ViewModel in the case of multiple screens/fragments)

64) How do Coroutines and Jetpack Compose work together?

Jetpack Compose is designed to work well with Coroutines:

- Compose is **main-safe**, meaning you can call suspend functions inside `LaunchedEffect`.
- You can use **StateFlow**, **SharedFlow**, and `collectAsState()` to observe coroutine-based data.
- Since Compose is declarative, it reacts automatically to the updated state.

Created by Anand Gaur

65) What is LaunchedEffect and how is it different from launch in coroutines?

- `LaunchedEffect` is a **Compose-specific coroutine launcher**.
- It launches a coroutine tied to the **lifecycle of the Composable**.
- It gets **canceled automatically** when the key parameter changes or the Composable leaves the composition.

66) How do you perform side effects like navigation or Toast in Compose using Coroutines?

Use `LaunchedEffect` to perform one-time side effects like:

- Showing Snackbar
- Navigating to another screen
- Sending analytics events

65) How do you handle long-running tasks in Composables without blocking UI?

Use `LaunchedEffect`, `rememberCoroutineScope`, or `viewModelScope` to launch suspend functions without blocking the UI.

66) How do you combine Compose state and Coroutine Flow effectively?

Expose UI state from ViewModel as `StateFlow` or `MutableStateFlow` and use `collectAsState()` in Composable.

```
val uiState = viewModel.uiState.collectAsState()
Text(uiState.value.message)
```

This ensures that every time the Flow emits a new value, Compose automatically recomposes.

Created by Anand Gaur

67) How do you prevent recomposition when collecting Flows in Compose?

Use:

- `collectAsStateWithLifecycle()` from Lifecycle Compose for lifecycle safety.
- `remember` or `derivedStateOf` to cache and prevent recomposing on unchanged values.

68) What are the risks of launching coroutines directly in Composables?

- Coroutine may get **canceled prematurely** if not launched in the right scope.
- It can lead to **memory leaks or crashes** if not tied to lifecycle.
- Use `LaunchedEffect` or `rememberCoroutineScope` for Composables.
- Prefer `viewModelScope` for long-running business logic.

69) How do you ensure your coroutine-based Compose app is lifecycle-aware?

- Use `collectAsStateWithLifecycle()` for Flows.
- Use `LaunchedEffect` for coroutine logic that depends on Composables.
- Avoid `GlobalScope` or unscoped launches.
- Always tie coroutines to `viewModelScope`, `rememberCoroutineScope`, or a `LifecycleOwner`.

Created by Anand Gaur

70) How do you handle pagination with Jetpack Compose and Coroutines?

- Use `LazyColumn` + `rememberLazyListState()`.
- Monitor scroll position.
- When user scrolls near the end, launch coroutine to load more.

Created by Anand Gaur

Unit Testing Concept

1) What is Unit Testing in Android?

Unit testing is the practice of testing **individual components or functions** in isolation to ensure they behave correctly.

- In Android, we typically use **JUnit** for unit testing.
- Unit tests run on the **JVM** and are **fast** because they don't require a device/emulator.

2) **What is the difference between Unit Tests and Instrumentation Tests in Android?**

Unit Test	Instrumentation Test
Runs on JVM	Runs on a real device/emulator
Fast	Slower due to UI/device interaction
Tests logic in isolation	Tests integration, UI, and end-to-end
Uses JUnit/Mockito	Uses Espresso, UI Automator, etc.

3) Which tools/libraries are used for Unit Testing in Android?

- **JUnit** – Base library for writing tests.
- **Mockito / MockK** – For mocking dependencies.
- **Truth / AssertJ / Hamcrest** – Assertion libraries.
- **Robolectric** – Allows you to run Android SDK code in JVM unit tests.
- **Turbine** – For testing Kotlin Flow.
- **Kotlin Test DSL** – For idiomatic Kotlin test writing.

Created by Anand Gaur

4) How do you test ViewModel in Android?

- ViewModels are easy to test because they don't depend on Android Framework.
- You can write plain JUnit tests and verify outputs by observing `LiveData` or `StateFlow`.

```
@Test
fun testViewModelReturnsUser() = runTest {
    val user = User("Anand")
    whenever(repo.getUser()).thenReturn(user)
    viewModel.loadUser()
    assertEquals(user, viewModel.user.value)
}
```

5) How to test LiveData in Unit Tests?

Use `getOrAwaitValue()` extension or observe synchronously using `observeForever()`.

6) How do you test Kotlin Coroutines in Android?

Use `runTest {}` from `kotlinx-coroutines-test` to control coroutine execution.

7) How do you test Flows in Android ViewModel?

Use `Turbine` or collect manually.

8) How do you mock dependencies in Unit Tests?

Use **Mockito** or **MockK** to mock dependencies.

9) What is Robolectric and when should you use it?

- Robolectric lets you run Android SDK code on JVM without a device/emulator.
- Great for testing Android classes like `Activity`, `Context`, `SharedPreferences`, etc.

10) How do you test Repository that depends on API and Room?

Use fake/dummy implementations or mock them:

```
class FakeApi : Api {
    override suspend fun getData(): List<String> = listOf("A", "B")
}
class FakeDb : Dao {
    override suspend fun insert(data: List<String>) { /* do nothing */ }
}
```

Use these fakes in the Repository test instead of real API or Room.

11) How do you test Hilt-injected ViewModel or Repository?

Use:

- `@HiltAndroidTest`
- `@UninstallModules` to replace modules
- `@BindValue` or `@TestInstallIn` for test dependencies

12) What is Espresso and when do you use it?

Espresso is used for **UI testing** in Android.

- Used for verifying UI elements, clicks, and navigation.
- Runs on device/emulator (Instrumentation test).

13) What is IdlingResource in Espresso?

Espresso needs to wait for background tasks (like API calls) to finish.

Use **IdlingResource** to tell Espresso when your app is idle:

```
IdlingRegistry.getInstance().register(myIdlingResource)
```

Created by Anand G

14) What is the role of testDispatcher and testScope in coroutine testing?

- `StandardTestDispatcher` allows you to manually control coroutine execution.
- `runTest` + `advanceUntilIdle()` helps simulate delays/timeouts.

15) What is Integration Testing in Android?

Tests the interaction between components like:

- Repository + Room
- ViewModel + Repository
- API + Parsing logic

Uses real or fake implementations, often involves Hilt or Retrofit + Room.

16) How do you write testable code in Android?

- Avoid hard dependencies, use interfaces and DI.
- Move logic out of Activities/Fragments into ViewModels or UseCases.
- Use Repository pattern.
- Avoid using Context in ViewModel directly.

17) How do you test Jetpack Compose UI?

Use `ComposeTestRule` and `createAndroidComposeRule`.

Created by Anand Gaur

18) What's the difference between mocking and faking in tests?

- **Mocking**: Simulating the behavior using libraries (Mockito, MockK).
- **Faking**: Writing lightweight dummy implementations manually.
- Fakes are better for integration or logic testing.

19) What is the test pyramid in Android?

1. **Unit Tests** (most tests) – Fast, logic-only
2. **Integration Tests** – Moderate
3. **UI Tests** (least) – Slow, but important for end-to-end flow

20) What is code coverage and how do you measure it in Android?

Code coverage measures the percentage of code executed by your test suite.

- Use **Jacoco** or **Gradle test coverage plugins**
- Aim for meaningful coverage, not 100%
- Focus more on **critical paths**, not on getters/setters

21) How do you test a Composable function in Jetpack Compose?

Use `ComposeTestRule` with `createComposeRule()` or `createAndroidComposeRule()`.

22) How to test user interactions like button clicks in Compose?

Use `onNodeWithText()`, `onNodeWithTag()` and perform actions:

Created by Anand Gaur

23) What layers should be unit tested in Clean Architecture?

You should unit test:

- **UseCases** – Core business logic
- **Repositories** – When using fakes/mocks
- **ViewModels** – With mocked repositories
- Skip UI logic in Activities/Fragments – use UI tests instead

24) What are common testing mistakes in Clean Architecture?

- Testing only ViewModel and skipping UseCases
- Not mocking repositories or using real API/DB in unit tests
- Mixing UI + logic in ViewModel (makes it hard to test)
- Not using coroutine test libraries like `runTest` or `TestDispatcher`

25) What are test doubles and which ones do you use in Clean Architecture?

- **Mocks** – fake behavior verification (Mockito/MockK)
- **Fakes** – lightweight real implementation (e.g., fake DB, API)
- **Stubs** – return predefined data

Use:

- Mock for ViewModel
- Fake for integration test (e.g., Repository + DB)

Created by Anand Gaur

Android Security Based

1) How can you securely store sensitive data in an Android app?

You should never store sensitive data (like passwords or tokens) in plain text. Instead:

- Use **EncryptedSharedPreferences** for small data like tokens.
- Use **Android Keystore** to store cryptographic keys securely.
- Avoid storing sensitive info in internal or external storage.

2) What is Android Keystore and why is it used?

Android Keystore is a secure container that helps store cryptographic keys. These keys can be used for encryption, decryption, or signing without exposing them directly to the app.

It ensures that:

- Keys cannot be extracted.
- Operations happen in secure hardware (if available).
- Your app remains safe even if rooted.

3) What are common security risks in Android apps?

Some common risks:

- Storing data in plain text.
- Using HTTP instead of HTTPS.
- Hardcoding API keys in code.
- Not validating inputs (leading to injection attacks).
- Using outdated libraries with vulnerabilities.

Created by Anand Gaur

4) How can you protect your API keys in Android?

- Don’t hardcode keys in code or strings.xml.
- Use **BuildConfig** with Gradle to store API keys.
- Store keys on the server and use **token-based auth**.
- Use **NDK (native C++)** for critical keys (not fully secure but harder to reverse).

5) How can you prevent reverse engineering of your APK?

- Use **ProGuard** or **R8** to obfuscate the code.
- Remove unused code and classes.
- Avoid storing logic or secrets in the app.
- Sign APKs with release keystore.
- Monitor unauthorized APKs using Play Store Console.

6) What is the use of ProGuard/R8 in Android?

ProGuard (now replaced by R8) is a tool that:

- **Minifies** code (removes unused code).
- **Obfuscates** names (changes class/method names to random characters).
- Makes it harder for attackers to reverse engineer the app.

7) How can you secure communication between app and server?

- Always use **HTTPS (SSL/TLS)** to encrypt data in transit.
- Use **certificate pinning** to verify the server.
- Avoid logging sensitive data (e.g., tokens or passwords).
- Use secure authentication methods like **OAuth2** or JWT.

Created By Anand Gaur

8) What is certificate pinning?

Certificate pinning is a technique where you **hardcode your server's public certificate or key** in the app. It ensures:

- The app only trusts your server.
- Prevents man-in-the-middle (MITM) attacks, even if a fake certificate is installed.

9) What is the difference between Internal and External Storage in Android?

- **Internal Storage**: Private to the app. Other apps can't access it. More secure.
- **External Storage**: Shared across apps. Can be accessed easily. Less secure.

Always prefer **Internal Storage** for storing sensitive files or data.

10) How to protect user credentials in an Android app?

- Never store raw passwords.
- Use **OAuth** or server-side login mechanisms.
- If needed, use **EncryptedSharedPreferences** to store tokens.
- Always use secure, hashed formats (like bcrypt) for passwords (on server side).

11) How can you detect if a device is rooted?

- Check for presence of `su` binary.
- Look for packages like `com.noshufou.android.su`.
- Use system calls or third-party libraries to detect root.
- Remember: Rooted device = lower security.

Created by Anand Gaur

12) How do you secure data when app goes in background?

- Clear sensitive data from memory (e.g., hide screen with FLAG_SECURE).
- Use `FLAG_SECURE` on Activities to prevent screen capture:

```
window.setFlags(WindowManager.LayoutParams.FLAG_SECURE,
WindowManager.LayoutParams.FLAG_SECURE)
```

- Encrypt local cache or temporary files.

13) What is the use of Network Security Config in Android?

It's an XML file used to define security policies for network connections.

- You can block clear-text traffic.
- Add trusted certificates.
- Customize HTTPS handling.

Defined in `res/xml/network_security_config.xml` and referenced in `AndroidManifest.xml`.

14) How to prevent WebView vulnerabilities?

- Disable JavaScript if not needed.
- Don't allow `file://` URLs.
- Never load dynamic or user-generated HTML.
- Validate URLs before loading.
- Use `WebViewClient.shouldOverrideUrlLoading()` to control navigation.

Created by Anand Gaur

15) How do you protect against SQL Injection in Android?

- Use **Room**, **SQLiteDatabase with parameterized queries**, or ORM tools.
- Never build raw SQL strings using user input.

16) How can you securely log in using biometric authentication?

- Use `BiometricPrompt` API.
- Store authentication state/token after successful fingerprint/face scan.
- Always have a secure fallback like PIN or password.

17) What is Obfuscation and why is it important?

Obfuscation makes your code harder to read or reverse-engineer.

Tools like ProGuard or R8:

- Rename methods/variables.
- Strip debug info.
- Remove unused code.

This prevents attackers from easily reading your logic or stealing code.

18) How to secure Web API responses from tampering?

- Use HTTPS for all communication.
- Add **HMAC or JWT** signatures for responses.
- Validate responses using checksums or tokens.
- Avoid exposing sensitive data in responses.

Created by Anand Gaur

19) Can you store data in SQLite securely?

Yes, by encrypting the database using libraries like:

- SQLCipher
- Room + custom encryption logic

Also, store minimal sensitive data locally if possible.

20) What is the best practice to handle user session securely?

- Store session tokens securely (EncryptedSharedPreferences).
- Use short expiry and refresh tokens.
- Invalidate tokens on logout or timeout.
- Don’t store session in memory if it contains sensitive info.

21) What is Android Keystore and why is it used?

- Android Keystore stores cryptographic keys securely inside hardware.
- Apps can use the keys for encryption/decryption without accessing the raw key.
- It helps protect data even if the device is rooted or stolen.

22) How can you ensure secure communication between app and server?

- Always use HTTPS (SSL/TLS).
- Avoid plain HTTP or unencrypted requests.
- Implement certificate pinning for extra security.

Created by Anand Gaur

23) How can you secure login and authentication in Android?

- Never store passwords locally.
- Use OAuth2 or token-based authentication (JWT).
- Use HTTPS for all communication.
- Implement proper session timeout and re-authentication.

24) How do you prevent screenshot capture of sensitive screens?

- Use `WindowManager.LayoutParams.FLAG_SECURE` on sensitive screens.
- Prevents screenshots and screen recording by other apps.
- Useful for OTP, payment, or profile screens.

25) How do you prevent data leakage from Android logs?

- Avoid logging sensitive information like tokens, passwords, or card details.
- Use `BuildConfig.DEBUG` to disable logs in release builds.
- Remove or limit logging in production code.

26) How can you prevent app tampering?

- Check the signature of the APK at runtime.
- Validate package integrity using SHA checksum.
- Use Play Integrity API or SafetyNet Attestation for more protection.

Created by Anand Gaur

27) What is SafetyNet and how does it help?

- SafetyNet checks device integrity and app environment.
- Detects root access, emulator, tampering, etc.
- Helps in deciding whether to allow the app to run or restrict access.

28) How do you protect exported components (Activity, Receiver, etc.)?

- Always set `android:exported="false"` if component is internal.
- Use permissions if component must be accessed by other apps.
- Prevent unauthorized access through intent filters.

29) How do you secure third-party SDKs or libraries in your app?

- Only use trusted and actively maintained libraries.
- Keep them updated to latest versions.
- Review permissions and data they collect or send.

30) How do you secure deep links and prevent unauthorized access via them?

- Use intent filters carefully.
- Don’t allow sensitive screens to open via implicit intents or unguarded deep links.
- Always validate input (like tokens, IDs) before displaying data.
- Add authentication checks in deep-linked destinations.

Created by Anand Gaur

31) How do you protect against Tapjacking attacks?

- Use `filterTouchesWhenObscured=true` in XML or `setFilterTouchesWhenObscured(true)` in code.
- This prevents touches if another app overlays your UI (like chat heads).
- Useful for payment or authentication screens.

32) What are pending intent hijacking vulnerabilities and how do you prevent them?

- If `PendingIntent` is created with a mutable flag, other apps may change it.
- Always use `FLAG_IMMUTABLE` when creating `PendingIntent` unless mutation is needed.

33) How do you detect code tampering in runtime?

- Use integrity checks: compare app's SHA or hash at runtime with known valid values.
- Use `PackageManager` to verify installer or signature.
- Use Play Integrity API for additional protection against tampering.

34) How do you encrypt data in Android using AES securely?

- Generate key using Android Keystore.
- Use AES/GCM/NoPadding (GCM is secure and provides integrity).
- Avoid hardcoding keys or IVs in code.
- Example: `Cipher.getInstance("AES/GCM/NoPadding")`

Created by Anand Gaur

35) What's the role of Network Security Configuration in security hardening?

- Allows you to restrict cleartext traffic (http).
- You can pin SSL certificates.
- Helps secure domain-level networking rules without code changes.

36) How do you prevent screen overlay attacks?

- Check for `TYPE_APPLICATION_OVERLAY` windows.
- On sensitive screens, add `WindowManager.LayoutParams.FLAG_SECURE`
- Warn users if an overlay is detected during sensitive input.

37) How do you protect SharedPreferences from unauthorized access?

- Use `EncryptedSharedPreferences` introduced in AndroidX.
- Avoid using `MODE_WORLD_READABLE` or `MODE_WORLD_WRITEABLE` (now deprecated).
- Don't store sensitive tokens in unencrypted form.

38) What is dynamic code loading and is it safe?

- Dynamic code loading (e.g., loading `.dex` or `.so` files from the network) is risky.
- Can be exploited to inject malicious code.
- Google Play blocks such behavior unless justified.
- Always avoid it or validate code signatures at runtime.

Created by Anand Gaur

39) How can you protect against MITM attacks in public WiFi?

- Always use HTTPS (TLS).
- Implement certificate pinning.
- Validate server identity and avoid redirecting to untrusted domains.
- Use HSTS headers in backend to enforce HTTPS.

40) How do you test your Android app for security issues?

- Use **Mobile Security Framework (MobSF)** for static and dynamic analysis.
- Use **Burp Suite** to test API vulnerabilities.
- Enable ProGuard/R8 and check obfuscation results.
- Use **Google Play Pre-launch report** to detect known vulnerabilities.

Created by Anand Gaur

Scenario Based Questions

1) How do you handle configuration changes (like screen rotation) in Android without losing data?

ViewModel stores UI-related data across configuration changes.

When screen rotates:

- Activity/Fragment is destroyed and recreated.
- ViewModel is not destroyed.
- ViewModel retains the data and passes it again to the UI.

Example:
In a profile screen, if user scrolls halfway and rotates the screen, without ViewModel the screen will reload from start.
But with ViewModel, the profile data and scroll position can be restored smoothly.

2) You have two API calls that must run in parallel and update UI when both complete. How do you implement this?

Use Kotlin Coroutines with `async` and `await`.

```
viewModelScope.launch {
    val userDeferred = async { api.getUser() }
    val postsDeferred = async { api.getPosts() }
    val user = userDeferred.await()
    val posts = postsDeferred.await()
    _uiState.value = Success(user, posts)
}
```

This way, both calls run in parallel and UI updates after both are done.

Created by Anand Gaur

3) You need to fetch data from both local Room database and network. How do you design this?

Use **Repository** with a fallback logic:

1. First try Room DB (cached data).
2. If data is old/missing, fetch from API.
3. Save new data in Room.

This ensures:

- Fast response (local DB)
- Always fresh data (network)

4) A user opens an app with no internet. How do you show offline data?

Use Room as the local cache.

- Repository checks connectivity.
- If offline, fetch from Room.
- If online, fetch from API and update Room.

Show “You’re offline” toast/snackbar while loading cached data.

5) In MVVM, who should handle click events and why?

The **ViewModel** should handle logic, not the Activity/Fragment.

- UI calls `viewModel.onLoginClicked()`
- ViewModel checks input, performs API call
- Emits success/error state via LiveData or StateFlow

Keeps code testable and follows separation of concerns.

Created by Anand Gaur

6) In Jetpack Compose, how do you preserve scroll position when user navigates back?

Use `rememberLazyListState()` in Composable:

```
val listState = rememberLazyListState()
LazyColumn(state = listState) { ... }
```

When navigating back to the list screen, it retains scroll position.

7) You observe LiveData in Fragment but UI doesn't update. What could be the issue?

Possible reasons:

- You're observing LiveData **after** setting it (missed update).
- The observer is not **active** (lifecycle is not STARTED or RESUMED).
- You used `observe()` instead of `observe(viewLifecycleOwner)`.

Always observe LiveData after setting up UI and use `viewLifecycleOwner`.

8) You have multiple fragments that need shared data. What's the best solution?

Use a **Shared ViewModel** scoped to Activity:

```
val viewModel: SharedViewModel by activityViewModels()
```

Both Fragment A and Fragment B access same ViewModel.

Use this when:

- You want to pass selected item from List Fragment to Detail Fragment.
- You need consistent form data across steps.

Created by Anand Gaur

9) In a search screen, every character triggers an API call. How to avoid unnecessary calls?

Use **debounce operator** with Kotlin Flow:

```
searchFlow
    .debounce(300)
    .distinctUntilChanged()
    .flatMapLatest { query ->
        repo.search(query)
    }
```

This waits 300ms after user stops typing before firing API.

10) You use Coroutines in ViewModel and app crashes with `CancellationException`. What went wrong?

You probably launched coroutines in wrong scope.

Use `viewModelScope` for UI-bound tasks:

```
viewModelScope.launch {
    // safe coroutine
}
```

If you used `GlobalScope` or forgot to cancel manually, it causes leaks or crashes.

11) How do you handle one-time events like navigation or toast messages in ViewModel?

Use **Channel** or **SharedFlow** for one-time events instead of StateFlow or LiveData.

12) How do you maintain Compose state when navigating between screens?

Use `rememberSaveable` or store state in ViewModel.

`rememberSaveable` saves state across recompositions and configuration changes.

```
val name = rememberSaveable { mutableStateOf("") }
```

Use ViewModel if you want to share state across multiple Composables or survive screen re-creation.

13) You want to combine two StateFlows (e.g., search query and filters) to fetch data. How do you do it?

Use `combine()` operator from Kotlin Flow.

```
val searchResults = combine(searchQuery, selectedFilter) { query,
filter ->
    return@combine repository.search(query, filter)
}.stateIn(...)
```

This emits new results whenever either `searchQuery` or `selectedFilter` changes.

14) How to pass large data (like a model object) between Compose screens?

Avoid passing large data through navigation arguments directly.

Instead, store the data in a Shared ViewModel, then retrieve it on the next screen.

```
navController.navigate("details")
```

On details screen:

```
val viewModel: SharedViewModel = hiltViewModel()
val user = viewModel.selectedUser
```

This avoids serialization issues and is more efficient.

15) How to trigger recomposition when data changes in ViewModel?

Expose data as `StateFlow`, `LiveData`, or use `mutableStateOf`.

16) You use remember { mutableStateOf() } in a Composable, but it resets on navigation. Why?

Because `remember` does not survive configuration changes or recomposition.

Use `rememberSaveable` instead, which stores value using SavedStateHandle internally.

```
val text = rememberSaveable { mutableStateOf("") }
```

Or keep UI state in ViewModel for better separation and persistence.

17) Your app fetches user data and posts on the same screen. Both APIs are independent. How will you optimize the API calls in ViewModel?

Use `coroutineScope.async` to fetch them in parallel, reducing wait time.

```
viewModelScope.launch {
    val userDeferred = async { repository.getUser() }
    val postsDeferred = async { repository.getPosts() }

    val user = userDeferred.await()
    val posts = postsDeferred.await()

    _uiState.value = UiState(user, posts)
}
```

This approach is better than calling APIs sequentially and reduces screen loading time.

18) In Compose, you navigate from a list screen to a detail screen. The detail screen needs full user data. How do you pass it without hitting API again?

Store selected user data in a SharedViewModel or a local cache before navigation.

In ListScreen:

```
viewModel.selectUser(user)
navController.navigate("details")
```

In DetailScreen:

```
val user = sharedViewModel.selectedUser
```

Avoid passing large objects through navigation arguments. It's better to hold state in ViewModel.

19) Your screen state resets when you rotate the device. How do you persist the state in Jetpack Compose?

Use `ViewModel` with `StateFlow` or `rememberSaveable`.

Best practice is to keep the state in ViewModel:

```
data class UiState(val name: String, val age: Int)

private val _state = MutableStateFlow(UiState())
val state: StateFlow<UiState> = _state
```

This ensures the state survives recompositions and configuration changes.

20) You are building a quiz app. You need to show a timer countdown on screen. How do you implement this in Compose using ViewModel?

Use `Kotlin Flow` and `delay` in ViewModel:

```
val countdown = flow {
    for (i in 30 downTo 0) {
        emit(i)
        delay(1000)
    }
}.stateIn(viewModelScope, SharingStarted.Eagerly, 30)
```

In Composable:

```
val timer by viewModel.countdown.collectAsState()
Text("Time left: $timer")
```

This ensures smooth countdown behavior even during recomposition.

21) You want to display a list using Compose that updates when data changes. Data comes from Room database. How would you do that?

Expose a Flow from DAO:

```
@Dao
interface TaskDao {
    @Query("SELECT * FROM tasks")
    fun getAllTasks(): Flow<List<Task>>
}
```

Collect it in ViewModel:

```
val tasks = taskDao.getAllTasks()
```

In Composable:

```
val tasks by viewModel.tasks.collectAsState()
LazyColumn {
    items(tasks) { task ->
        Text(task.title)
    }
}
```

This keeps UI reactive with real-time database changes.

22) You have a feature where network may be slow or offline. You want to show cached data first and then refresh from network. How do you do that?

Use **Offline-first** pattern with Room and Retrofit:

```
val dataFlow = flow {
    emit(room.getData()) // Emit cached
    val remoteData = api.getData()
    room.insert(remoteData)
    emit(remoteData) // Emit fresh
}
```

This provides instant UI update using local data and syncs fresh data later.

23) You're making two dependent API calls: one fetches user token, the second uses that token to fetch user profile. How do you implement this using Coroutines?

Use `suspend` functions in a `viewModelScope.launch` block:

24) In your app, you need to schedule background sync every 6 hours. How would you do this reliably?

Use `PeriodicWorkRequest` with `WorkManager`:

```
val workRequest = PeriodicWorkRequestBuilder<SyncWorker>(6,
TimeUnit.HOURS).build()
WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "SyncWork",
    ExistingPeriodicWorkPolicy.KEEP,
    workRequest
)
```

This will run even in Doze Mode and handles system restarts.

25) In a large app with multiple screens, how do you share common error messages or snackbar events from ViewModel to UI without triggering multiple times?

Use `SharedFlow` or a single-consumption wrapper:

```
private val _snackbar = MutableSharedFlow<String>()
val snackbar = _snackbar.asSharedFlow()

suspend fun showSnackbar(message: String) {
    _snackbar.emit(message)
}
```

This is a clean way to emit one-time events that UI can collect without duplication.

26) Your app needs to ask for multiple permissions (e.g., location, camera). How do you handle permission logic cleanly across screens?

Use `ActivityResultContracts.RequestMultiplePermissions()` with a central `PermissionManager` class.
Abstract permission logic away from UI so it's testable and reusable.

27) You want to cancel a network request if the user leaves the screen. How would you implement this in ViewModel?

Use `viewModelScope` which is automatically canceled when the ViewModel is cleared:

```
viewModelScope.launch {
    try {
        val response = repository.getData()
        _state.value = response
    } catch (e: Exception) {
        _state.value = Error(e.message)
    }
}
```

If the screen (and ViewModel) is destroyed, the coroutine is cancelled.

28) How do you handle a large Room database migration without blocking the main thread or crashing the app on older devices?

Use the `Room.databaseBuilder()` with `.fallbackToDestructiveMigration()` **only during early development**.

For production apps, implement a custom `Migration` class:

```
val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE User ADD COLUMN last_login INTEGER")
    }
}
```

Pass this to `addMigrations()` to avoid data loss and ensure smooth upgrade paths.

29) You need to update only one field in a Room table without overwriting the whole object. How do you do that efficiently?

Create a custom DAO method for that field:

```
@Query("UPDATE user SET is_active = :isActive WHERE id = :userId")
suspend fun updateUserActiveState(userId: Int, isActive: Boolean)
```

Avoid updating the entire entity if you're modifying just a single field.

30) How do you prevent duplicate API requests when a user rapidly presses a button?

Use a flag (`isLoading`) or debounce logic with Kotlin Coroutines:

```
var isLoading = false

fun onSubmit() {
    if (isLoading) return
    isLoading = true
    viewModelScope.launch {
        try {
            val response = api.submitData()
        } finally {
            isLoading = false
        }
    }
}
```

Alternatively, use debounce from Flow when dealing with text inputs or repeated events.

31) You have a foreground service that uploads files. How do you ensure it continues even if the app is killed or removed from recent tasks?

Start the service as a **foreground service** using `startForeground()` with a persistent notification:

```
startForeground(NOTIFICATION_ID, buildNotification())
```

Also, set proper flags in the service declaration in the manifest and request **POST_NOTIFICATIONS** permission on Android 13+.

32) Your app fetches paginated data from API and also stores it locally. How do you keep local cache in sync with remote using Paging 3?

Use **RemoteMediator**:

```
@OptIn(ExperimentalPagingApi::class)
val pager = Pager(
    config = PagingConfig(pageSize = 20),
    remoteMediator = MyRemoteMediator(api, database),
    pagingSourceFactory = { database.dao().getData() }
).flow
```

`RemoteMediator` handles syncing between local Room DB and remote API.

33) You need to retry a failed network request with exponential backoff. How would you do that in Coroutines?

Use retry logic with delay inside a loop:

```
suspend fun fetchDataWithRetry() {
    var attempt = 0
    while (attempt < 3) {
        try {
            repository.fetchData()
            return
        } catch (e: IOException) {
            delay((2.0.pow(attempt) * 1000L).toLong()) // exponential backoff
            attempt++
        }
    }
}
```

You can also use `retryWhen` with Kotlin Flow.

34) Your app has a RecyclerView with infinite scrolling. How do you avoid multiple API calls being triggered at the same time?

Use a flag to prevent duplicate triggers:

```
var isLoading = false

fun loadMoreItems() {
    if (isLoading) return
    isLoading = true
    viewModelScope.launch {
        repository.loadNextPage()
        isLoading = false
    }
}
```

Combine this with `addOnScrollListener` and Paging 3 for best results.

35) How do you manage state in Jetpack Compose when working with dynamic form fields like checkboxes or radio groups?

Use a `MutableStateMap` or a list of custom data class holding selection:

```
val formState = remember { mutableStateMapOf<String, Boolean>() }
Checkbox(
    checked = formState["terms"] == true,
    onCheckedChange = { formState["terms"] = it }
)
```

36) You need to trigger a sync API only once after app install or user login. How would you do that?

Use `DataStore` or `SharedPreferences` flag:

```
if (!prefs.getBoolean("sync_done", false)) {
    // Call API
    prefs.edit().putBoolean("sync_done", true).apply()
}
```

37) In your project, some screens use XML and some use Compose. How do you mix them safely?

You can use:

- `ComposeView` inside XML
- `AndroidView` inside Compose

38) Your app is crashing due to memory leaks. How do you debug and solve it?

1. Use **LeakCanary** to detect retained objects.
2. Ensure you are not keeping references to:
    - Context inside ViewModel
    - View references in long-lived classes
3. Use `WeakReference` if needed.
4. Observe lifecycle and unsubscribe observers on `onStop/onDestroy`.

39) Your app has a large JSON file to parse offline. What's the best way to parse it efficiently on the user's device?

Use `kotlinx.serialization` or `Gson/Moshi` with background threading:

- Read the file using `Dispatchers.IO` to avoid blocking the UI.
- If the file is in assets:

```
val json = context.assets.open("data.json").bufferedReader().use {
it.readText() }
val data = Json.decodeFromString<List<MyModel>>(json)
```

Use streaming if file is very large (e.g., Moshi's `JsonReader` or Gson's `JsonReader`).

40) You want to support offline-first functionality using Room + Remote API. How would you architect it?

1. Query from Room first.
2. If empty or stale, fetch from API.
3. Save to Room for future use.

Created by Anand Gaur

41) Your app's RecyclerView with large images scrolls slowly. How do you improve performance?

- Use **Glide** or **Coil** with proper caching.
- Use **DiffUtil** in your Adapter.
- Avoid nested scrollables inside items.
- Reuse views properly, and avoid heavy layout nesting.

Preload images using `RecyclerView.OnScrollListener`.

42) How would you handle a situation where the app crashes only on some devices or Android versions?

- Use **Crashlytics** or **Play Console** to track device models and Android versions.
- Implement **feature flags** or conditional logic using:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
   // New logic
} else {
   // Fallback
}
```

- Use **fallback resources** (like different layouts or drawables) for older devices.
- Log and isolate the crash with custom tags.

Created By Anand Gaur

43) In Jetpack Compose, you notice recomposition is happening more often than expected. What could be the cause and how do you fix it?

Causes:

- Using mutable state directly in Composables without hoisting.
- Passing changing parameters like lambda/functions.
- Not using `remember` or `rememberUpdatedState`.

Fixes:

- Use `remember { mutableStateOf(...) }` and hoist state.
- Use `key` and `derivedStateOf` to limit recomposition.
- Profile recompositions using the **Layout Inspector**.

44) Your team is facing ANR (Application Not Responding) issues. How do you debug and fix them?

- Use **StrictMode** and **systrace** to find blocking code.
- Common causes:
    - Heavy disk or network operation on the main thread.
    - Synchronous API or Room DB calls on main thread.
- Move to background threads:

```
viewModelScope.launch(Dispatchers.IO) {
   val result = repository.getData()
}
```

- Watch for long operations in `BroadcastReceiver`, `Service`, or `onCreate()` of Activity.

Created By Anand Gaur

45) How would you handle a large dataset in RecyclerView for smooth scrolling?

Scenario:

You have an app that displays thousands of items in a list using RecyclerView. Users report lag when scrolling. How do you improve the performance?

To improve performance and ensure smooth scrolling in a `RecyclerView` handling large datasets, follow these optimizations step by step:

1. Use Paging Library (Recommended)

When you're dealing with thousands of items, **don't load all at once**. Use **Paging 3** library to load data in chunks:

```
val pager = Pager(PagingConfig(pageSize = 20)) {
    YourPagingSource()
}.flow.cachedIn(viewModelScope)
```

- Supports **lazy loading**.
- Efficient with both local DB (Room) and network (Retrofit).
- Reduces memory usage drastically.

2. Use ViewHolder Pattern Properly

Even though `RecyclerView` uses ViewHolder by default, make sure:

- ViewHolder holds references to views instead of calling `findViewById()` every time.
- Avoid unnecessary view binding in `onBindViewHolder()`.

3. Avoid Heavy Operations in onBindViewHolder()

- Do not load images or perform computation in `onBindViewHolder`.
- Offload operations like image resizing or formatting outside or ahead of time.

Created by Anand Gaur

4. Use DiffUtil with ListAdapter

If you're updating lists frequently (e.g., filters or sorting), use `ListAdapter` with `DiffUtil`:

```
class MyAdapter : ListAdapter<Item, MyViewHolder>(ItemDiffCallback())
{ ... }
```

This avoids redrawing the entire list and only updates changed items.

5. Use Efficient Image Loading (Glide/Coil/Picasso)

When loading images:

- Use `Glide` or `Coil` with placeholder, caching, and resizing:

6. Enable RecyclerView Optimizations

- Set fixed size if size doesn't change:

```
recyclerView.setHasFixedSize(true)
```

- Use a `LinearLayoutManager` instead of a complex custom layout unless needed.

7. Avoid Nested RecyclerViews

If your UI has nested scrollable lists, consider flattening them. Or use:

- `ConcatAdapter` instead of nesting.
- `RecyclerView.RecycledViewPool` to share views across nested adapters.

Created by Anand Gaur

44) How would you manage configuration changes in an Android app?

Scenario:

Your app crashes when the screen orientation changes. How do you handle configuration changes like rotation or language updates?

In Android, a **configuration change** (like screen rotation, language change, keyboard availability) causes the **Activity to be destroyed and recreated** by default. If you haven't managed your data properly, it leads to **crashes or data loss** during this recreation.

To properly handle configuration changes and prevent crashes:

1. Use ViewModel to Retain UI-Related Data

- ViewModel survives configuration changes.
- Keeps your UI data intact across screen rotations.

```
class MyViewModel : ViewModel() {
    val userData = MutableLiveData<User>()
}
```

2. Use SavedStateHandle to Persist Data Across Process Death

- SavedStateHandle is lifecycle-aware and survives not just rotation but also process death.

```
class MyViewModel(savedStateHandle: SavedStateHandle) : ViewModel() {
    val name = savedStateHandle.getLiveData<String>("name")
}
```

You can restore data easily on recreation.

3. Persist UI State Using onSaveInstanceState() for Small Data

Use this when not using ViewModel or when saving UI-specific things like scroll position or selected tab.

4. Avoid Storing UI References in ViewModel or Static Fields

Never store references to `Context`, `Activity`, `View`, or `Binding` inside a ViewModel. This causes memory leaks during configuration changes.

5. Handle Configuration Changes Manually (Not Recommended)

You can handle config changes manually using:

```
<activity android:configChanges="orientation|screenSize" />
```

Not recommended unless you're building custom views, games, or camera apps.

6. Retain Non-UI Data Using Retained Fragments (Legacy Approach)

Before ViewModel existed, we used retained fragments to hold non-UI data:

```
setRetainInstance(true)
```

Now, it's deprecated. ViewModel is the better replacement.

7. Localization Changes Handling

When the language changes dynamically, you must:

- Recreate the Activity
- Reload strings from updated `Context`
- Avoid caching old Context or strings

Use `ContextWrapper` or libraries like `AppLocale` to switch languages properly.

45) How would you implement offline data synchronization in your app?

Scenario:

You need to develop a feature where users can work offline, and data syncs automatically when the network is available.

Created by Anand Gaur

To implement offline data sync, we follow a **“store locally → sync later”** approach using a combination of:

- Local database (Room)
- Network monitoring (ConnectivityManager or NetworkCallback)
- Background tasks (WorkManager)
- Conflict resolution logic (optional)

1. Use Room Database for Offline Storage

All user actions (e.g., form fill, draft save, task update) should be saved locally using Room.

2. Insert Data Locally First

Whenever the user performs an action:

```
val newTask = Task(id = UUID.randomUUID().toString(), title = "Write
notes", isSynced = false)
taskDao.insert(newTask)
```

Don’t try to push data to the server immediately if offline.

3. Detect Network Availability

Use `ConnectivityManager.NetworkCallback` to monitor internet connectivity.

4. Use WorkManager for Reliable Background Sync

WorkManager is the **best tool** for deferred background tasks that should run even after app restart or device reboot.

5. Optional: Conflict Resolution

If the same data was modified both locally and on the server, define your conflict strategy:

- **Last write wins**
- **Timestamps-based**

- **Manual merge**

Example: Compare `updated_at` timestamp fields and keep the latest.

6. Optional: Schedule Periodic Sync

Use `PeriodicWorkRequest` to sync at regular intervals, even if connectivity status doesn't change.

46) How would you implement a feature to detect user inactivity?

Scenario:
You want to log out users after 5 minutes of inactivity for security reasons.

To detect user inactivity and log them out after 5 minutes, follow these steps:

1. Define What Counts as "Inactivity"

User inactivity can be defined as:

- No touch events (taps, swipes, scrolls)
- No keyboard input
- No screen interaction (app is idle in foreground)

2. Use a Centralized Timer Handler

Create a **singleton or base activity** that listens for user interaction and resets a timer.

3. Advanced: Use Foreground Services or AlarmManager

If you want the timer to run even when the app goes to background for some time, consider:

- **AlarmManager** for scheduled logout
- **WorkManager** if you want to show notification before logout
- **Foreground service** (not recommended for simple logout cases)

Created by Anand Gaur

47) How would you test your app for edge cases like low memory or slow network?

Scenario:

Your app crashes in low-memory conditions or fails to handle slow network scenarios.

To make your app reliable, you should simulate and test **low memory** and **slow network** scenarios during development and QA. Here's how:

1. Testing for Low Memory Conditions

a. Use Android Studio's Profiler

- Open Android Profiler → Monitor **Memory usage** while using the app.
- Look for memory leaks, high object retention, or GC (Garbage Collection) spikes.

b. Simulate Low Memory via Developer Options

- Enable **"Don't keep activities"** under Developer Options.
    - This forces activities to be destroyed and recreated, mimicking memory pressure.
- Use **Background process limit** to reduce the number of background processes.

c. Override `onTrimMemory()` and `onLowMemory()`

- Implement these callbacks in Activities, Fragments, or Application class to gracefully release resources.

```
override fun onTrimMemory(level: Int) {
    super.onTrimMemory(level)
    if (level == TRIM_MEMORY_UI_HIDDEN) {
        // Release bitmaps, cache, etc.
    }
}
override fun onLowMemory() {
    super.onLowMemory()
    // Clear cache or release large resources
}
```

Created by Anand Gaur

d. Use LeakCanary

- Add LeakCanary to detect memory leaks in Activities, Fragments, ViewModels, etc.

2. Testing for Slow Network Conditions

a. Use Network Link Conditioner or Throttle Tool

- On macOS, use **Network Link Conditioner**.
- In Android Emulator:
  `Extended Controls → Cellular → Set Network Speed = Edge/3G/4G`.

b. Use Charles Proxy or Postman Proxy

- Simulate delays in API response (e.g., 5–10 sec delay).
- Verify app behavior during timeouts or delayed responses.

c. Set Retrofit Timeout

Use appropriate timeout settings in your API client.

3. Best Practices for Both Cases

a. Use Paging Library for Large Lists

- Automatically handles memory efficiently and supports loading in chunks.

b. Avoid Memory Leaks

- Never keep a strong reference to `Context` in ViewModel or static variables.
- Unregister listeners, observers, or callbacks properly.

c. Gracefully Handle Failures

- Use sealed classes or `Result<T>` pattern in your Repository to emit loading, success, and error states.

4. Testing Tools to Consider

Created by Anand Gaur

- **LeakCanary** – Memory leaks
- **StrictMode** – Detects accidental disk/network calls on main thread
- **Stetho** – Debugging network/database
- **Firebase Crashlytics** – Real-time crash reporting
- **Monkey Test** – Random UI interactions under stress

48) How would you handle background tasks without draining the battery?

Scenario:

You need to run a periodic background task without affecting battery life.

To handle background tasks efficiently without draining the battery, I would follow best practices and choose the right APIs provided by Android. Here's a structured approach:

1. Use WorkManager for Battery-Optimized Background Tasks

- **WorkManager** is the recommended API for deferrable, guaranteed background work.
- It is **battery-aware** and follows **Doze mode** and **App Standby restrictions**.
- **Why WorkManager?**
    - Automatically chooses **AlarmManager**, **JobScheduler**, or **Firebase Job Dispatcher** under the hood depending on the device's API level.
    - Respects system optimizations like Doze and Battery Saver.
    - Survives app restarts and reboots (if enabled).

2. Use Constraints to Save Battery

Always define constraints to avoid running background work under poor conditions.

Example constraints:

Created by Anand Gaur

- `setRequiresCharging(true)` → Runs only when device is charging
- `setRequiresBatteryNotLow(true)` → Skips if battery is low
- `setRequiredNetworkType(NetworkType.UNMETERED)` → Uses Wi-Fi

This avoids running background work when battery or network conditions are not optimal.

3. Avoid Using Handlers, Services, or AlarmManager for Periodic Tasks

- `AlarmManager` or foreground services drain battery when misused.
- **ForegroundService** should only be used for time-sensitive or user-visible tasks (e.g., tracking, audio playback).
- Avoid using **Handlers with long delays** or **custom threads** for periodic work — they aren't battery aware.

4. Use Coroutine Dispatchers Wisely

- Use `Dispatchers.IO` for background I/O operations (network, DB).
- Cancel coroutines when `ViewModel.onCleared()` or `LifecycleOwner` is destroyed.

5. Schedule Background Tasks Smartly

- Don't run unnecessary jobs frequently.
- Use **Exponential Backoff** for retries to avoid battery drain from repeated failures.

6. Use Firebase Cloud Messaging (FCM) for Push-Based Updates

If background tasks are for syncing data, **use FCM push notifications** to trigger sync only when needed — this is more battery efficient than polling.

Created by Anand Gaur

49) How would you implement a multi-language app in Android?

Scenario:
The app should support English, Hindi, and Spanish. Users should be able to switch languages without restarting the app.

Answer:

- Store strings in `res/values`, `res/values-hi`, `res/values-es` folders
- Use `LocaleHelper` or `AppCompatDelegate.setApplicationLocales()` (Android 13+)
- For older versions, recreate activities after setting the locale

```
val locale = Locale("es")
Locale.setDefault(locale)
val config = context.resources.configuration
config.setLocale(locale)
context.createConfigurationContext(config)
```

50) How do you implement deep linking in your Android app?

Scenario:
Clicking a link like `myapp://user/123` should open the user's profile in the app.

Answer:

- Define **intent-filters** in the manifest
- Use **NavController** with Jetpack Navigation for Compose or XML

51) How would you handle push notifications when the app is in the background or killed?

Scenario:
Your app uses Firebase Cloud Messaging (FCM), but users report that they miss notifications when the app is not open.

Answer:

- Use **FirebaseMessagingService** to receive messages even in background or killed states.
- For **data-only payloads**, ensure they're marked as `priority: high` and handled in `onMessageReceived()`.
- For **notification messages**, Android system handles the display when the app is in background.
- For foreground, handle manually and show a local notification:

```
override fun onMessageReceived(remoteMessage: RemoteMessage) {
    if (remoteMessage.notification != null) {
        // Show custom notification
    }
}
```

- Use **Notification Channels** for Android 8+.
- Ensure **background restrictions** are handled (battery optimizations, Doze Mode).

Created By Anand Gaur

52) How would you implement a custom error handling mechanism across the app?

Scenario:
Your app uses many API calls, and you want consistent error handling and messages.

Answer:

- Create a **sealed class** for API result:

```
sealed class ApiResult<out T> {
    data class Success<T>(val data: T): ApiResult<T>()
    data class Error(val message: String): ApiResult<Nothing>()
    object Loading : ApiResult<Nothing>()
}
```

- Wrap every network call inside a `try-catch` and return `ApiResult.Error`.

- In ViewModel, observe this and show UI accordingly.

```
when (val result = repository.getData()) {
    is ApiResult.Success -> // Show data
    is ApiResult.Error -> // Show error message
}
```

This avoids crashing and creates a standard UX.

53) How would you monitor crashes and performance issues in production?

Scenario:
You want real-time insights into user crashes and slow screens.

Answer:

- Integrate **Firebase Crashlytics** for crash reporting.

- Use **Firebase Performance Monitoring** to track slow screens, network latency.

- Log custom keys and non-fatal exceptions for better diagnostics.

54) How would you implement a secure login using biometric + PIN fallback?

Scenario:
Your app requires strong user authentication.

Answer:

- Use **BiometricPrompt API** to authenticate with fingerprint/face.
- Provide a fallback dialog to allow manual PIN input if biometric fails.
- Store the PIN securely using EncryptedSharedPreferences.
- On successful login, issue a secure token or grant access.

55) How would you create a paginated list using Paging 3 library?

Scenario:
You have an API that returns data in pages and you need to display it efficiently.

Answer:

- Create a `PagingSource` that loads data from API:

```
class MyPagingSource(...) : PagingSource<Int, Item>() {
    override suspend fun load(params: LoadParams<Int>): LoadResult<Int,
Item> {
        ...
        return LoadResult.Page(data, prevKey, nextKey)
    }
}
```

Connect with `Pager` in the repository:

```
val pager = Pager(PagingConfig(pageSize = 20)) {
    MyPagingSource()
}.flow
```

- Collect this flow in ViewModel and observe in UI.
- Use `LazyPagingItems` in Jetpack Compose or `PagingDataAdapter` in XML.

57) How do you support multiple screen sizes and resolutions?

Scenario:
Your app UI looks broken on tablets or small-screen devices.

Answer:

- Use **ConstraintLayout** or **Jetpack Compose** with `Modifier.weight`, `BoxWithConstraints`, etc.
- Define layouts in `layout-sw600dp/` for tablets.
- Use **dp and sp** units always.
- Avoid hard-coded widths/heights. Use `match_parent`, `wrap_content`, or relative constraints.
- Use `WindowSizeClass` for Compose to create responsive layouts.

58) How do you handle crash reporting in production?

Use Crashlytics or Bugsnag:

- Add custom logging using `Crashlytics.log("...")`
- Log non-fatal exceptions manually:

Created by Anand Gaur

CI/CD, Git, and Detekt Based Questions

1) What is CI/CD in Android?

CI/CD in Android development refers to **Continuous Integration and Continuous Delivery/Deployment**, a set of practices that **automate the building, testing, and delivery of Android applications**.

- **Continuous Integration (CI)** means that developers regularly push code to a shared repository (like GitHub), and every push automatically **triggers a build and test**. This helps catch errors early.

- **Continuous Delivery (CD)** means that once code is tested and validated, it can be automatically packaged (APK or AAB) and delivered to testing environments (like Firebase App Distribution).

- **Continuous Deployment** goes one step further and automatically **publishes the app to production** like Google Play once it passes all quality checks.

CI/CD improves team collaboration, reduces manual errors, and speeds up release cycles.

2) Why is CI/CD important in Android development?

CI/CD helps in:

- **Faster development cycles** by automating build and testing.

- **Early bug detection** due to frequent code integration and automated tests.

- **Better team collaboration**, as code is constantly merged and verified.

- **Reduced manual work** — no need to manually run tests, generate APKs, or upload to Play Store.

- **Consistent builds** because the process is scripted and version-controlled.

Created by Anand Gaur

3) Which tools are commonly used for CI/CD in Android?

Some commonly used CI/CD tools are:

- **GitHub Actions** – Integrated with GitHub, good for open-source and personal projects.
- **Bitrise** – Android and iOS friendly, no setup needed, GUI-based.
- **Jenkins** – Powerful, but needs self-hosting and manual setup.
- **CircleCI** or **GitLab CI** – Useful for medium to large teams.
- **Firebase App Distribution** – For distributing builds to testers.

4) What is Git in Android development?

Git is a **version control system** that allows Android developers to **track changes in code, collaborate with others**, and manage code history.

In Android projects:

- Git helps you to **save code checkpoints** (commits).
- You can create **branches** to work on features without affecting main code.
- Git supports **pull requests** and **code reviews**.
- You can roll back to **older working versions** if something breaks.

5) What are common Git commands used in Android projects?

Here are some essential Git commands:

- `git clone` – Download project from repository.
- `git status` – See current changes.

Created by Anand Gaur

- `git add .` – Add all changes to be committed.
- `git commit -m "message"` – Save changes locally.
- `git push origin branch-name` – Upload code to GitHub.
- `git pull` – Fetch and merge latest code from others.
- `git checkout branch-name` – Switch to another branch.

These commands help you manage code effectively in a team.

6) What is Git branching and why is it important?

Branching allows you to **create a separate line of development** from the main project.

For example:

- `main` or `master` – production-ready code.
- `feature/login` – working on login screen.
- `bugfix/crash` – fixing a crash.

Branching is important because:

- You can work on new features **without breaking main code**.
- It allows **multiple developers to work in parallel**.
- You can test and review code before merging it.

Created by Anand Gaur

7) What is a Pull Request (PR) in Git?

A Pull Request (PR) is when you ask to **merge your code changes into a main branch**.

It is used to:

- **Review code** before merging.
- **Discuss issues** or improvements in the code.
- **Run automated checks** (like tests or lint) before merging.

PRs help in **maintaining code quality** and **collaboration** among developers.

8) What is Detekt in Android?

Detekt is a **static code analysis tool** for **Kotlin** projects. It helps you **find code issues** like:

- Bad coding practices
- Long methods
- Unused variables
- Style violations

Detekt works **without running the app**. It just analyzes your Kotlin codebase and gives suggestions for improvement.

9) Why should we use Detekt in Android projects?

You should use Detekt because:

- It helps **maintain clean and consistent Kotlin code**.
- It finds **bugs and performance issues early**.

Created by Anand Gaur

- It supports **custom rules** – you can define your own coding standards.
- It integrates easily with **CI tools** like GitHub Actions or Jenkins.
- It supports **report generation** (HTML, XML) for analysis.

10) How do you set up Detekt in an Android project?

Here's a simple way to set up Detekt:

1. Add the Detekt plugin in your root `build.gradle`:

```
plugins {
    id("io.gitlab.arturbosch.detekt") version "1.23.0"
}
```

2. Apply it in your module:

```
apply plugin: "io.gitlab.arturbosch.detekt"
```

3. Run Detekt with:

```
./gradlew detekt
```

It will analyze your Kotlin code and generate a report in the `build/reports/detekt/` folder.

11) Can we integrate Detekt in CI/CD pipelines?

Yes, Detekt is easily integrated into CI/CD tools like GitHub Actions or Jenkins.

In CI:

- Run Detekt as a part of your build process.
- Fail the build if there are code violations.
- Generate a report so team members can review issues.

This enforces code quality across the team automatically.

12) How is Detekt different from Lint in Android?

- **Lint** checks both **Java and Kotlin** code and focuses on Android-specific issues (like layouts, permissions, etc.).
- **Detekt** is purely for **Kotlin code** and focuses more on **code smells, complexity, and style**.

Many teams use **both tools together** for complete code analysis.

13) What are build variants in Android, and how are they used in CI/CD?

Build variants allow you to create **different versions of your app** from the same codebase.

For example:

- `debug` – for internal testing
- `release` – for production
- You can also have `flavor` combinations like `freeDebug`, `paidRelease`

In CI/CD, build variants help to:

- **Test debug builds automatically**
- **Generate release APKs/AABs** for production
- Customize builds (different API keys, resources, or features)

Created by Anand Gaur

14) What is the difference between Continuous Delivery and Continuous Deployment?

- **Continuous Delivery**: The app is automatically tested and **ready to release**, but a human manually triggers the release (like uploading to Play Store).
- **Continuous Deployment**: Every successful build is **automatically released to production** without manual intervention.

In Android, most teams follow **Continuous Delivery**, not Deployment, because of the Google Play Store's manual checks and staged rollouts.

15) What are Git tags and how are they used in Android CI/CD?

A Git **tag** is a **label for a specific commit**. It is commonly used to **mark releases**, like:

```
git tag v1.0.0
git push origin v1.0.0
```

In CI/CD, tags are used to:

- Trigger **release builds only** when a tag is pushed.
- Keep track of **app version history**.
- Automatically publish versioned APKs/AABs.

16) What are the types of Git merge strategies?

When you merge branches, Git can do it in different ways:

- **Fast-forward**: The branch history moves forward cleanly without a merge commit.
- **Recursive (with merge commit)**: A new commit is created to combine both branches.
- **Squash and merge**: Combines all commits into one for a clean history.

Created by Anand Gaur

For Android projects, **squash and merge** is often used to **keep the main branch clean**.

17) What is `.gitignore` and why is it important?

`.gitignore` is a file where you list **files and folders Git should ignore**, like:

- `build/`
- `.idea/`
- `.gradle/`
- `local.properties`

This is important because:

- It prevents **unwanted files from being committed**.
- Keeps the repository **clean and portable**.
- Reduces chances of committing **API keys or secrets** by mistake.

18) What is Git rebase and how is it different from merge?

`git rebase` rewrites your branch's history to **apply commits on top of another branch**.

- `merge` creates a **merge commit**, preserving history.
- `rebase` gives a **linear history**, making it cleaner.

For example, in a feature branch, you can:

```
git rebase main
```

This makes your commits look like they came after the latest `main`, which is often better for **clean commit history**.

Created by Anand Gaur

19) What are custom rules in Detekt and when do you need them?

Detekt allows you to create **your own custom rules** using Kotlin code. This is useful when:

- You want to **enforce project-specific guidelines**, like naming conventions.
- You want to **ban certain functions or patterns**, like `!!` or `println`.

You can build a custom Detekt rule as a plugin and include it in your project to enforce your team's coding standards.

20) How can we improve CI/CD build performance in Android?

Ways to improve performance:

- **Enable Gradle caching** (both local and remote).
- Use **parallel execution** in Gradle (`org.gradle.parallel=true`).
- **Avoid unnecessary tasks** in CI (e.g., skip UI tests if not needed).
- Use **modularization** to build only the changed modules.
- Use **CI runners with more memory/CPU**.

Faster builds mean **faster feedback**, which is key in agile development.

21) What is the difference between `detekt` and `ktlint`?

- **detekt** focuses on **code smells**, complexity, best practices, and reports.
- **ktlint** focuses mainly on **code formatting** (indentation, line breaks, etc.).

Created by Anand Gaur

Many teams use **both tools together**:

- `ktlint` for formatting
- `detekt` for code quality

They can also be automated via pre-commit hooks or CI pipelines.

22) What are CI/CD pipeline stages in Android projects?

A typical CI/CD pipeline has:

1. **Checkout** – Get the code from Git.
2. **Build** – Run Gradle build (assemble).
3. **Test** – Run unit and instrumentation tests.
4. **Lint/Static Analysis** – Run Detekt, Lint, ktlint.
5. **Sign APK/AAB** – Sign using keystore (for release).
6. **Distribute** – Upload to Firebase App Distribution or Play Store.
7. **Notify** – Send notification to Slack/Email after build.

Each stage ensures quality and automation from code to deployment.

23) What is the use of Gradle in CI/CD for Android?

Gradle is the **build system** used to:

- Compile Kotlin/Java code
- Merge resources
- Build APKs or AABs
- Run tests, Lint, Detekt
- Sign and align the APK

Created by Anand Gaur

- Manage dependencies

In CI/CD, the Gradle commands are triggered automatically to **build, test, and release** the app.

24) How do you secure secrets (like keystore or API keys) in Android CI/CD?

Secrets should never be committed to Git. Use:

- **GitHub Secrets** or **GitLab CI variables**
- **Encrypted files** (e.g., encrypted `keystore` file with a password)
- **.env files** with `.gitignore`
- **Gradle Properties** to inject secrets at runtime

Tools like **Gradle Play Publisher** support secure release signing using these methods.

25) How do you distribute Android builds to testers automatically?

You can use tools like:

- **Firebase App Distribution**
- **App Center (by Microsoft)**
- **Bitrise** or **GitHub Actions with Firebase CLI**

In CI/CD:

1. Generate signed APK/AAB using Gradle.
2. Use Firebase CLI or App Distribution plugin to upload the build.
3. Notify testers via email or Slack with download links.

This allows quick feedback and avoids sending builds manually.

Created by Anand Gaur

26) What are Gradle tasks used in CI pipelines?

Common Gradle tasks in Android CI/CD:

- `./gradlew clean` – Clean old build files.
- `./gradlew lint` – Run Lint checks.
- `./gradlew test` – Run unit tests.
- `./gradlew detekt` – Run Detekt analysis.
- `./gradlew assembleDebug` – Build debug APK.
- `./gradlew assembleRelease` – Build signed release APK.
- `./gradlew bundleRelease` – Build release AAB for Play Store.

These tasks form the core of Android automation workflows.

27) What is a Git conflict and how do you resolve it?

A Git conflict happens when **two branches change the same part of the code**, and Git doesn't know which one to keep.

You resolve it by:

1. Opening the conflicted file.
2. Looking for conflict markers:

```
<<<<<<< HEAD
Your code
=======
Incoming branch code
>>>>>>> feature
```

3. Choosing the correct code, deleting the markers.
4. Committing the resolved file.

Conflicts usually happen during **merge or rebase** operations.

Created by Anand Gaur

28) How can you automate versioning in Android CI/CD?

You can:

- Use a script to update `versionCode` and `versionName` based on:
    - Git tags (like `v1.2.3`)
    - Number of commits
    - Date or timestamp

Example (in `build.gradle`):

```
versionCode = 100 + gitCommitsCount
versionName = "1.0.${gitCommitsCount}"
```

This ensures every CI build has a **unique version** and reduces manual mistakes.

29) What are `pre-commit` hooks in Git and how are they used in Android projects?

Git `pre-commit` hooks are scripts that run **before committing code**.

In Android projects, you can use them to:

- Run `detekt` or `ktlint`
- Prevent commits with syntax errors
- Block commits if TODOs or debug logs are present

You set them in `.git/hooks/pre-commit` or use tools like **Husky** (with Kotlin JS or multiplatform).

They ensure **code quality is enforced even before pushing to CI**.

Created by Anand Gaur

30) What is a Detekt baseline and why do we use it?

A Detekt baseline is a file (`detekt-baseline.xml`) that stores **existing issues** in your project.

You use it when:

- You're adding Detekt to a legacy project with lots of issues.
- You want to **ignore current issues** and focus on **new code quality**.

Command to generate baseline:

```
./gradlew detektBaseline
```

Detekt will skip known issues in baseline while still checking for new violations.

31) What is caching in CI and why is it important for Android?

Caching stores data like:

- Gradle dependencies
- Build outputs
- Detekt or test results

Benefits:

- **Faster builds** because Gradle doesn't redownload or rebuild unchanged modules.
- **Saves CI build minutes** on paid platforms.
- **Improves performance** especially for large projects.

Most CI tools like GitHub Actions, Bitrise, or Jenkins support Gradle caching out of the box.

Created by Anand Gaur

32) How do you ensure code quality using CI/CD?

In CI/CD:

- Run **unit tests** and **instrumented tests**
- Integrate **Detekt**, **ktlint**, and **Android Lint**
- Perform **code coverage analysis** with tools like Jacoco
- Block merge if tests fail or coverage drops
- Use **static analysis gates** to fail CI on high-severity issues

This keeps your codebase healthy and prevents poor-quality code from going to production.

33) How do you set up GitHub Actions for Android CI/CD?

Basic `.github/workflows/android.yml` structure:

```
name: Android CI

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-java@v3
        with:
          java-version: '17'
      - name: Build with Gradle
        run: ./gradlew build
      - name: Run Detekt
        run: ./gradlew detekt
```

You can add more steps like uploading APK, sending Slack notifications, etc.

Anand Gaur

34) What is Git stash and when do you use it?

`git stash` temporarily saves your **uncommitted changes** without committing them.

Use it when:

- You want to switch branches but have **unfinished work**
- You want to pull latest code without losing local changes

Commands:

- `git stash` – Save current changes.
- `git stash pop` – Apply saved changes back.
- `git stash list` – See stashed items.

This is useful when **you need a clean working directory temporarily**.

35) How can you enforce Detekt rules for every commit in Android?

Ways to enforce:

- Add a **pre-commit hook** to run Detekt.
- Make Detekt run as a **mandatory CI step** (fail build if it fails).
- Configure Detekt as part of `./gradlew check`.

You can even add `detektCheck.dependsOn("check")` to ensure it runs during regular builds.

36) How do you handle signing APK/AAB in CI/CD securely?

In CI/CD, signing an app for release requires:

- **Keystore file**
- **Keystore password**

Created by Anand Gaur

- **Key alias and key password**

Best practices:

- **Do not commit keystore files** to Git.
- Store keystore and credentials as **encrypted CI/CD secrets** (like GitHub Secrets).
- Use Gradle config like:

```
signingConfigs {
    release {
        storeFile file(System.getenv("KEYSTORE_FILE"))
        storePassword System.getenv("KEYSTORE_PASSWORD")
        keyAlias System.getenv("KEY_ALIAS")
        keyPassword System.getenv("KEY_PASSWORD")
    }
}
```

This allows your CI pipeline to **sign builds automatically** without exposing credentials.

37) What is a CI/CD pipeline failure? How do you debug it?

A pipeline failure means **some step in the automated process (build/test/lint/deploy) failed**.

Debugging process:

- **Read logs** carefully from failed step.
- Identify if it's a **syntax error, test failure, dependency issue, or environment issue**.
- Reproduce locally (`./gradlew build` or `test`) to isolate the problem.
- Check if recent commits or merged branches introduced the issue.

CI pipelines should fail **early and loudly** to avoid releasing broken code.

38) What are Detekt rule sets, and how can you customize them?

Detekt has built-in rule sets:

- `style` – Enforces formatting rules.
- `complexity` – Detects overly complex code.
- `performance` – Optimizes code behavior.
- `potential-bugs` – Finds dangerous code.
- `comments` – Checks for documentation.

You can:

- Enable/disable rules in `detekt.yml`
- Set severity (`warning` or `error`)
- Write **custom rules** for project-specific checks

This flexibility helps maintain **team-wide code quality standards**.

39) What is semantic versioning and how do you apply it in Android CI/CD?

Semantic versioning follows format: `MAJOR.MINOR.PATCH`
For example: `2.5.1`

Usage in Android:

- Update `versionName` based on semantic versioning.
- Increase:
    - **MAJOR**: Breaking API change
    - **MINOR**: New features (backward compatible)

Created by Anand Gaur

- **PATCH**: Bug fixes

CI/CD can auto-increment PATCH versions based on commits or tags.

40) What's the difference between `assembleDebug` and `assembleRelease` in CI/CD?

- `assembleDebug` – Builds an APK for debugging:
  - Includes debug tools
  - Not signed for Play Store
- `assembleRelease` – Builds a **signed, optimized APK/AAB**:
  - No debug logs
  - Minified and obfuscated (via ProGuard or R8)
  - Used for production release

CI/CD pipelines run both depending on environment (testing vs production).

41) How do you test multiple modules using CI/CD in Android?

In modular apps:

- Use CI to run **individual module builds** (`:core:build`, `:feature:login:build`)
- Use `--continue` flag to build all even if one fails
- Use Gradle dependency analysis to detect what module changed and **build only that**

This **speeds up the pipeline** and reduces unnecessary builds.

Created by Anand Gaur

42) What is Git cherry-pick and when do you use it in Android projects?

`git cherry-pick` copies a commit from one branch to another.

Example:

```
git cherry-pick abc1234
```

Use cases:

- Bug fix on `main` that also needs to be applied to `release/1.0` branch
- Moving a specific change without merging the whole branch

It's helpful in **hotfix situations** during app development.

43) How do you trigger builds conditionally in CI/CD for Android?

Examples of conditional triggers:

- Only build when code is pushed to `main`:

```
on:
  push:
    branches: [main]
```

- Only build when files in `app/` change:

```
paths:
  - 'app/**'
```

- Only deploy if it's a release tag:

```
if: startsWith(github.ref, 'refs/tags/')
```

This avoids **wasting CI minutes** on irrelevant changes.

Created by Anand Gaur

44) How do you manage large CI workflows for Android in GitHub Actions?

Strategies:

- **Split into multiple jobs**: build, test, lint
- Use **matrix builds** for multiple flavors or SDK versions
- Use **reusable workflows** (call other YAML workflows)
- Use **job dependencies** (`needs:` keyword)

This improves **readability, parallelism, and reusability** of CI scripts.

45) What are some examples of bad practices in Git for Android teams?

- Committing directly to `main` without PR
- Not writing meaningful commit messages
- Ignoring `.gitignore` rules (committing `build/`, `.idea`)
- Pushing broken code that fails CI
- Not using branches for features or bug fixes

These can lead to **unstable main branches and poor collaboration**.

46) How do you analyze code complexity in Android using Detekt?

Detekt's `complexity` rules help detect:

- Long methods
- Nested loops
- Large classes

Created by Anand Gaur

- High cognitive complexity

You can customize thresholds in `detekt.yml`:

```
LongMethod:
  active: true
  threshold: 30
```

This helps maintain **readable and maintainable code**.

47) How can you roll back a CI/CD deployment in Android?

Steps:

- Revert the commit that triggered the release
- Rebuild previous version using Git tag (`v1.0.3`)
- Upload old AAB/APK to Play Console with higher version code
- Notify testers or users if needed

A good pipeline always keeps **previous release artifacts** for emergency rollbacks.

48) What is the difference between `push` and `pull request` triggers in CI/CD?

- `push` trigger: Pipeline runs on every commit to branch.
- `pull_request` trigger: Runs only when a PR is created or updated.

Example:

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

PR-based CI ensures **only reviewed code gets tested and merged**.

Created by Anand Gaur

49) How would you design a scalable CI/CD pipeline for a large Android project with multiple modules and multiple teams?

To design a scalable CI/CD pipeline:

- **Use modular builds**: Configure Gradle so only changed modules are built and tested. Use `gradle-project-cache` to improve speed.
- **Parallel jobs**: Split testing, linting, and building into separate parallel jobs to improve build times.
- **Matrix strategy**: Use CI matrix builds to test on different API levels, flavors, or device configurations.
- **Triggers**:
  - PRs → run lint, tests, and review gates
  - Push to main → trigger release build
- **Environment-specific workflows**: Separate workflows for `staging`, `QA`, and `production`.
- **Artifact versioning and caching**: Use artifact storage with semantic versioning (`1.0.0-rc01`) and Gradle caching.
- **Secret management**: Inject keys and credentials using GitHub Actions secrets or Vault.

This system should also include rollback, monitoring, and alerting for failed builds.

50) How do you ensure high-quality pull requests in a CI/CD setup?

- **Enforce CI checks** on every PR (build + unit test + lint + Detekt).
- Require **minimum code coverage** using Jacoco + Codecov.
- Integrate **static analysis tools** like Detekt and ktlint.

Created by Anand Gaur

- Require at least **one peer code review** before merge.
- Use **Danger or custom GitHub Actions** to add PR checks (e.g., PR description template, checklist).
- Add **pre-merge hooks** for schema validation or API compatibility checks.

This ensures **clean, secure, and production-ready code** enters the main branch.

51) How do you detect and prevent flaky tests in CI/CD?

- **Run tests multiple times** in CI using a test flakiness detection plugin or retry logic (`--rerun-failed`).
- Tag tests as `@FlakyTest` and isolate them.
- Separate UI (instrumentation) tests and run them on **emulators using Firebase Test Lab or cloud devices**.
- Use **test orchestration tools** like Android Test Orchestrator for clean state execution.
- Configure logs to detect **non-deterministic behavior** (e.g., relying on timing, UI thread delays).
- Fail build on flaky test patterns and enforce fixing them in sprints.

52) How can you detect architectural violations using static analysis in Android?

Use tools like:

- **Detekt + custom rules** to enforce layer boundaries (e.g., UI layer should not access Repository directly).
- **ArchUnit (JVM)** – A Java/Kotlin library to write **unit tests for architecture rules** (e.g., Activity should not depend on DAO).
- **Ktlint + annotations** to prevent misuse of APIs.

Created by Anand Gaur

You can write rules like:

```
val rule = slices()
    .matching("com.example..service..")
    .should().notBeAccessed().byAnyPackage("com.example..ui..")
```

This protects your codebase from **architecture erosion** over time.

53) How do you prevent secrets or API keys from leaking through Git or CI/CD?

- Add `local.properties`, `google-services.json`, `.env` to `.gitignore`.
- Use **Git pre-commit hooks** to detect keys using regex or tools like `gitleaks`, `truffleHog`.
- Store secrets in **GitHub Secrets, GitLab CI Variables, or HashiCorp Vault**.
- Use environment variables in Gradle:

```
buildConfigField("String", "API_KEY",
"\"${System.getenv("API_KEY")}\"")
```

- Automatically **scan commits in CI** for hardcoded keys and block merges if found.

Also rotate keys periodically and enable monitoring on usage.

54) What is Gradle remote build cache and how can it speed up Android CI/CD?

Gradle **remote build cache** stores the outputs of builds (compiled classes, dex, APKs) on a remote server. This lets multiple CI runners or developers **reuse previous builds**.

Benefits:

- **Dramatic reduction in build times**
- Ensures **build reproducibility**
- Reduces CI cost

Created By : Anand Gaur

Setup involves:

- Enabling remote cache in `gradle.properties`
- Using Gradle Enterprise or custom S3 cache
- Configuring project-specific cache keys for reliability

Ideal for **monorepos and large teams**.

55) How would you implement a multi-environment release strategy (e.g., QA, Staging, Prod) in Android?

1. Define **product flavors**:

```
flavorDimensions "env"
productFlavors {
    qa { applicationIdSuffix ".qa" }
    staging { applicationIdSuffix ".staging" }
    prod { applicationIdSuffix ".prod" }
}
```

2. Use **different build configs and API keys** per flavor via `BuildConfig`.
3. **In CI:**

- Trigger different jobs based on branch name or tag:

```
if: github.ref == 'refs/heads/staging'
```

4. Use **Play Store tracks**:

- QA → Internal testing
- Staging → Closed track
- Prod → Production track

Each build goes through its respective environment automatically.

56) How would you handle canary releases using CI/CD in Android?

1. Use **Google Play's staged rollout**:
    - Upload via CI to production with 10% rollout.
    - Monitor crash reports, ANRs.
    - Gradually increase rollout via CI/CD scripts or Play Developer API.
2. Tag builds in CI:

```
git tag v1.0.0-canary
```

3. Use a dedicated **canary flavor** with internal config flags.
4. Enable **Firebase Remote Config or Feature Flags** to control visibility.

This ensures **controlled rollout** and safer production deployments.

57) How do you write custom Detekt rules for Kotlin?

Steps:

1. Create a new Kotlin project.
2. Add Detekt dependency:

```
implementation("io.gitlab.arturbosch.detekt:detekt-api:1.23.0")
```

3. Extend `Rule` class:

```
class NoSystemOutPrintlnRule : Rule() {
    override val issue = Issue("NoSystemOut", Severity.CodeSmell, ...)
    override fun visitCallExpression(expression: KtCallExpression) {
        if (expression.text.contains("System.out.println")) {
            report(CodeSmell(issue, Entity.from(expression), "Avoid println"))
        }
    }
}
```

Created by Anand Gaur

4. Package and include in your project's Detekt config.

Custom rules help you enforce **team-specific code standards**.

58) How do you monitor and alert on failed CI/CD builds?

- Use **Slack notifications** in CI pipeline:

```
- name: Slack Notification
  uses: slackapi/slack-github-action@v1
```

- Use **GitHub status checks** on PRs.
- Integrate **Datadog**, **New Relic**, or **Prometheus** for build metrics.
- Set up **retry logic** with alerts after 2-3 failures.
- Tag owners for failing jobs in team-based ownership models.

This improves **developer visibility** and faster issue resolution.

59) How would you optimize test execution time in Android CI/CD?

- Use **test sharding**: Split tests into multiple devices or jobs.
- Run unit tests on JVM, only run critical UI tests in CI.
- Use **Gradle test filters**:

```
./gradlew test --tests "com.example.MyTestClass"
```

- Use **Android Test Orchestrator** to isolate test runs.
- Run **only changed module tests** using Gradle input/output tracking.

This can reduce test times from **30 mins to under 5 mins** for large projects.

Created by Anand Gaur

60) How do you prevent regressions in Android releases via CI/CD?

- Use **automated regression test suites** that run on every release candidate.
- Enforce **backward compatibility tests** for APIs or Room schema.
- Integrate **UI screenshot comparison testing** (e.g., Shot, Paparazzi).
- Use **version locking** for dependencies (`versions.toml` or `buildSrc`).
- Add **release checklists in CI** before production upload.

This makes sure **new features don't break existing functionality**.

Created by Anand Gaur

Thank You for Being a Part of This Journey

A big thank you once again for purchasing this **Android Developer Interview Handbook**.

I truly hope that after reading the questions and answers in this book, you not only found value but also felt more confident and prepared for your upcoming interviews. This handbook is designed to help take your **interview preparation and skillset to the next level** — making your journey smoother, sharper, and more focused.

Regular Updates – Stay Ahead!

This handbook already includes **500+ carefully selected questions**, and in the future, I'll keep adding more — especially based on:

- Latest Android updates
- Real-world interview scenarios
- Advanced architecture & Jetpack topics & DevOps & Code Quality

Whenever a new version is released, you'll automatically receive the **updated copy** on your **email and WhatsApp number**.

A Humble Request from the Author

This guide is the result of my **8+ years of real-world experience**, 1000+ interviews (taken and given), and countless hours of effort — all focused on helping developers like you grow confidently in their careers.

I have just **one humble request**:

👉 **Please do not share this PDF or any part of this handbook with others**, even if they're your close friends or colleagues.

Every page of this book has been crafted with deep thought, hard work, and passion. Your honesty and respect for that effort not only means the world to me — it helps me keep creating more valuable content for the Android community.

If someone you know wants access, please encourage them to support this work and get their own copy here:
🔗 **https://topmate.io/anand_gaur/1623062**

Created by Anand Gaur

🎓 Want to Learn More?

Check out my **Advance Android Masterclass Playlist** on YouTube:
▶️ Watch Now

I've also written **110+ detailed Android articles** on Medium — covering everything from beginner to expert level — in simple and easy-to-understand language:
📖 https://medium.com/@anandgaur2207

Need 1:1 Mentorship?

If you're looking for personalized guidance, interview tips, or a mock interview session You can **book a mentorship session with me on Topmate**.

I'll help you identify your weaknesses, turn them into strengths, and **prepare you to crack interviews confidently**.

Many developers I've mentored in the past year have successfully **switched to better companies** and significantly improved their careers.

👉 https://topmate.io/anand_gaur

🌐 Let's Stay Connected!

Please **follow, connect, and subscribe** — I regularly share tips, insights, and job-ready content:

🔗 **LinkedIn:** linkedin.com/in/anandgaur22
📖 **Medium:** medium.com/@anandgaur2207
📸 **Instagram:** instagram.com/tech.anandgaur
▶️ **YouTube:** youtube.com/@technicalanandgaur
💻 **GitHub:** github.com/anandgaur22
📱 **WhatsApp:** 919807407363

Wishing you all the best for your interviews and career ahead.
Keep learning, keep growing — and never stop preparing.
Because the stronger your foundation, the brighter your future!

– Anand Gaur
Mobile Tech Lead | Mentor | Interview Coach

Created by Anand Gaur

Testimonials

"The Android Developer Interview Handbook was an invaluable resource during my interview preparation. Clear, concise, and comprehensive—highly recommended!"

— Eva Das

"This is exactly what I was looking for. Precise, well-organized, and super helpful. Simplifies tough concepts and focuses on what really matters."

— Tikeshwar Sahu

"This handbook is just wow! It reveals hidden details behind concepts that many overlook. A must-read for every native Android developer."

— Dharmender Kumar

"Extremely helpful! Whether you're a beginner or experienced Android developer, this book offers great value. Perfect for job switchers or anyone wanting to strengthen their Android and architectural concepts."

— Sushmita Biswas

"Your Complete Guide to Cracking Android Interviews"

About Author

Anand Gaur is a Tech Lead with rich experience in mobile app development, specializing in Android, Kotlin, and Kotlin Multiplatform. He has mentored many developers and guided them to crack interviews at top IT companies.

You can find Anand at https://linktr.ee/anandgaur

ISBN : 978-93-343-7047-8

9 789334 370478

Digital Copy MRP: ₹ 699